JVH Systems

Generated by Doxygen 1.9.5

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

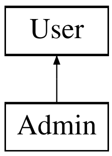
Class Documentation

4.1 Admin Class Reference

This class is the responsible for instantiating an Admin object to distinguish between Users and Admins.

```
#include <Admin.h>
```

Inheritance diagram for Admin:



Public Member Functions

- Admin (int id)

    Builds a new Admin object.

Static Public Member Functions

- static bool adminStatus (int id)

    Gets current object status.

Additional Inherited Members

4.1.1 Detailed Description

This class is the responsible for instantiating an Admin object to distinguish between Users and Admins.

Definition at line 17 of file Admin.h.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Admin()

```
Admin::Admin (
        int id )
```

Builds a new Admin object.

Parameters

id	Admin identificator

Definition at line 8 of file Admin.cpp.

```
00008                    : User(id){
00009
00010     this->isAdmin = true;
00011
00012 }
```

References User::isAdmin.

4.1.3 Member Function Documentation

4.1.3.1 adminStatus()

```
bool Admin::adminStatus (
        int id ) [static]
```

Gets current object status.

Parameters

id	User Identifier

Returns

(True) is Admin. (False) is Guest

Definition at line 15 of file Admin.cpp.

```
00015                            {
00016
00017     try{
00018         int* userData = find(id);
00019         return (userData[ADMIN_COLUMN] == ADMIN_IDENTIFIER);
00020
00021     // User not found
00022     }catch(UserNotFoundException &e){
00023         return false;
```

```
00024     }
00025 }
```

References User::ADMIN_COLUMN, User::ADMIN_IDENTIFIER, and User::find().

Referenced by Login::startLogin().

The documentation for this class was generated from the following files:

- Admin.h
- Admin.cpp

4.2 AI Class Reference

This class will be the responsible for interpreting global commands requested from the main class and checking the proper functioning of the devices. Thus, it will be implemented by the main class and can't live without Device class. Will be settled by the builder.

```
#include <AI.h>
```

Public Member Functions

- AI ()
  Creates a new Automatic Interface ready for controlling device and main program status.
- void startCollectingData ()
  Search for every DataDevice and fills all its data storage with measurements.
- void refreshData ()
  Refresh last data to every device built.
- void throwDemon ()
  Throws a process-like demon, which will be updating the DataDevice measurements while waiting for key-interruption.

Static Public Member Functions

- static void requestOption (std::string opt, bool isAdmin)
  Requests an option from main settings interface.

Private Types

- enum {
  TURNDEVICES = 1 , TURNSECURITY = 2 , CHECKSECURITY = 3 , MICROPHONE = 4 ,
  FORCEERROR = 5 , BUILDDEVICE = 6 , ADDUSER = 7 , REMOVEUSER = 8 ,
  RETURNSECURITY = 9 , FORCEBADALLOC = 10 , SHOWUSERS = 11 , MORE = 12 ,
  EXIT = -1 }

Static Private Member Functions

- static void turnDevices ()

  Turns all devices on/off.
- static void turnSecurity ()

  Turns all security on/off.
- static void checkSecurity ()

  Checks and print current security status.
- static void microphone ()

  Sends a message using the microphone.
- static void forceError ()

  Forces an error in all Security devices.
- static void buildDevice ()

  Calls the builder for starting a interface to build a new device in execution time.
- static void recoverSecurity ()

  Recover all security breaches to safe status.
- static void addUser ()

  Calls User class to add a new user.
- static void removeUser ()

  Calls User class to remove a existing user.
- static void forceBadAlloc ()

  Generates a bad_alloc exception in a safe environment.
- static void showAllUsers ()

  Calls to print all users registered.
- static void ensureAdminAccess (bool isAdmin)
- static void signalHandler (int signum)

Private Attributes

- int time_counter

  Used for counting number of time-stamps.

Static Private Attributes

- static std::string TAG = "AI"

  Name of class (for exception throwing)
- static bool abort = false

  Alerts if exit request has been detected.
- static int MAX_DATA = 30

  Sets maximum of data/measurements collected from Data Devices.
- static int SLEEPTIME = 1

  Sets time to sleep between interactions.

4.2.1 Detailed Description

This class will be the responsible for interpreting global commands requested from the main class and checking the proper functioning of the devices. Thus, it will be implemented by the main class and can't live without Device class. Will be settled by the builder.

Definition at line 24 of file AI.h.

4.2.2 Member Enumeration Documentation

4.2.2.1 anonymous enum

`anonymous enum [private]`

Enumerator

TURNDEVICES	
TURNSECURITY	
CHECKSECURITY	
MICROPHONE	
FORCEERROR	
BUILDDEVICE	
ADDUSER	
REMOVEUSER	
RETURNSECURITY	
FORCEBADALLOC	
SHOWUSERS	
MORE	
EXIT	

Definition at line 66 of file AI.h.

```
00066         {
00067         TURNDEVICES = 1,
00068         TURNSECURITY = 2,
00069         CHECKSECURITY = 3,
00070         MICROPHONE = 4,
00071         FORCEERROR = 5,
00072         BUILDDEVICE = 6,
00073         ADDUSER = 7,
00074         REMOVEUSER = 8,
00075         RETURNSECURITY = 9,
00076         FORCEBADALLOC = 10,
00077         SHOWUSERS = 11,
00078         MORE = 12,
00079         EXIT = -1
00080     };
```

4.2.3 Constructor & Destructor Documentation

4.2.3.1 AI()

`AI::AI ( )`

Creates a new Automatic Interface ready for controlling device and main program status.

Definition at line 25 of file AI.cpp.

```
00025        {
00026     this->time_counter = 0;
00027     signal(SIGINT, signalHandler);
00028
00029 }
```

References signalHandler(), and time_counter.

4.2.4 Member Function Documentation

4.2.4.1 addUser()

```
void AI::addUser ( )  [static], [private]
```

Calls User class to add a new user.

Definition at line 455 of file AI.cpp.

```
00455                   {
00456 
00457     int user, password;
00458     bool isAdmin;
00459     std::string entry;
00460 
00461     SimpleDisplay::cout("[AI] Insert a NIF id:  ");
00462     std::getline(std::cin, entry);
00463 
00464     try {
00465         user = std::stoi(entry);
00466     }catch (std::exception e){
00467         SimpleDisplay::cout("[AI] Input not valid");
00468         sleep(SLEEPTIME);
00469         return;
00470     }
00471 
00472     SimpleDisplay::cout("[AI] Insert a password:  ");
00473 
00474     try {
00475         password = std::stoi(entry);
00476     }catch (std::exception e) {
00477         SimpleDisplay::cout("[AI] Input not valid");
00478         sleep(SLEEPTIME);
00479         return;
00480     }
00481 
00482     std::getline(std::cin, entry);
00483     do {
00484         SimpleDisplay::cout("[AI] Give the user Admin Status?  [y/n]:  ");
00485         std::getline(std::cin, entry);
00486     }while(entry!= "y" && entry!= "n");
00487 
00488     isAdmin = (entry == "y" ?  1 :  0);
00489     User::addUser(user,password,isAdmin);
00490 
00491 }
```

References User::addUser(), SimpleDisplay::cout(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.2 buildDevice()

```
void AI::buildDevice ( )  [static], [private]
```

Calls the builder for starting a interface to build a new device in execution time.

Definition at line 382 of file AI.cpp.

```
00382                      {
00383 
00384     SimpleDisplay::execBuildName();
00385     std::string name;
00386     std::getline(std::cin,name);
00387     if (name == "EXIT") {
00388 
00389         SimpleDisplay::cout("\n[AI] Exit Request");
```

```
00390         sleep(SLEEPTIME);
00391         return;
00392     }
00393
00394     SimpleDisplay::execBuildType();
00395     std::string type;
00396     std::getline(std::cin,type);
00397
00398     if (type != "data" && type != "kernel"){
00399         SimpleDisplay::cout("\n[AI] Exit Request");
00400         sleep(SLEEPTIME);
00401         return;
00402     }
00403
00404     std::string generator;
00405     if (type == "data"){
00406         SimpleDisplay::execBuildGenerator(true);
00407         std::getline(std::cin,generator);
00408     }else{
00409         SimpleDisplay::execBuildGenerator(false);
00410         std::getline(std::cin,generator);
00411     }
00412
00413     SimpleDisplay::execBuildFinish();
00414     sleep(SLEEPTIME*2);
00415     SimpleDisplay::cout("[AI] Testing Building...\n");
00416     sleep(SLEEPTIME*2);
00417     SimpleDisplay::cout("[AI] Testing Device...\n");
00418     sleep(SLEEPTIME*2);
00419
00420     DeviceBuilder::clearAll();
00421     DeviceBuilder* newDevice = new DeviceBuilder(name);
00422     DataDevice::numSensors = 0;
00423     KernelDevice::numKerns = 0;
00424     Device::numDevices = 0;
00425     if (type == "data"){
00426         (*newDevice).setAsDataDevice();
00427         (*newDevice).setDataGenerator(generateRandomData);
00428         DeviceBuilder::buildAllDevices();
00429     }else{
00430         (*newDevice).setAsKernelDevice();
00431         (*newDevice).setSecurityGenerator(exampleTest);
00432         DeviceBuilder::buildAllDevices();
00433
00434     }
00435     SimpleDisplay::cout("[AI] Building SUCCESS\n");
00436     std::cout « "\nPress ENTER to start";
00437     std::cin.get();
00438     std::cin.clear();
00439
00440 }
```

References DeviceBuilder::buildAllDevices(), DeviceBuilder::clearAll(), SimpleDisplay::cout(), exampleTest(), SimpleDisplay::execBuildFinish(), SimpleDisplay::execBuildGenerator(), SimpleDisplay::execBuildName(), SimpleDisplay::execBuildType(), generateRandomData(), Device::numDevices, KernelDevice::numKerns, DataDevice::numSensors, and SLEEPTIME.

Referenced by requestOption().

4.2.4.3 checkSecurity()

```
void AI::checkSecurity ( )  [static], [private]
```

Checks and print current security status.

Definition at line 319 of file AI.cpp.

```
00319                         {
00320
00321     for (int i = 0; i < KernelDevice::numKerns; i++) {
00322
00323         // Acessing Device
00324         KernelDevice* k = &KernelDevice::allKernelDevices[i];
00325
00326         // Displaying chart
```

```
        SimpleDisplay::cout("[AI] [" + k->getName() + "] ");

        if (~*(k))
            SimpleDisplay::cout("[RUNNING]: ");
        else
            SimpleDisplay::cout("[DISCONNECTED]: ");

        if (!k->securityHasBeenBroken)
            SimpleDisplay::cout("Security status correct");
        else
            SimpleDisplay::cout("[WARNING] Security FAILURE! Please check the logs.");

        SimpleDisplay::cout("\n");

    }
    // ----- Exiting...  -----
    std::string none;
    SimpleDisplay::cout("Press ENTER to continue");
    getline(std::cin,none);

}
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), Device::getName(), KernelDevice::numKerns, and KernelDevice::securityHasBeenBroken.

Referenced by requestOption().

4.2.4.4 ensureAdminAccess()

```
void AI::ensureAdminAccess (
            bool isAdmin )  [static], [private]
```

Ensures Administrator Access to a function

Parameters

isAdmin	previous status of user. (True) is Admin. False (is Guest).

Exceptions

InsufficientPermissionsException	Administrator Authentication Failed

Definition at line 240 of file AI.cpp.

```
                                              {

    // ----- Checking Admin Status -----
    if (!isAdmin) {
        SimpleDisplay::cout("THIS ACTION REQUIRES ADMINISTRATOR ACCESS");
        sleep(SLEEPTIME*2);
        throw InsufficientPermissionsException(TAG);
    }

    // ----- Entering a console and requesting password -----
    try {
        std::string message;
        SimpleDisplay::printRequestPassword();
        SimpleDisplay::cout("Keyboard:  ");
        std::getline(std::cin,message);

        // Checking input
        if (message != "0")
            throw InsufficientPermissionsException(TAG);

        // Invalid Input
        }catch(std::exception e){

```

```
00263             SimpleDisplay::cout("INVALID PASSWORD");
00264             sleep(SLEEPTIME);
00265             throw InsufficientPermissionsException(TAG);
00266         }
00267
00268 }
```

References SimpleDisplay::cout(), SimpleDisplay::printRequestPassword(), SLEEPTIME, and TAG.

Referenced by requestOption().

4.2.4.5 forceBadAlloc()

`void AI::forceBadAlloc ( )  [static], [private]`

Generates a bad_alloc exception in a safe environment.

Definition at line 516 of file AI.cpp.

```
00516                         {
00517
00518     SimpleDisplay::cout("\nBuilding Secure Environment...");
00519     sleep(SLEEPTIME);
00520     SimpleDisplay::cout("\nGenerating Error...");
00521     try {
00522         std::vector<int> vector;
00523         for (int i = 0; i > -1; i++)
00524             vector.push_back(i);
00525     }catch(std::exception &e) {
00526         SimpleDisplay::cout("\n[WARNING] Environment Failure:  \n-> what():  ");
00527         SimpleDisplay::cout(e.what());
00528         SimpleDisplay::cout("\nReturning...");
00529         sleep(SLEEPTIME*4);
00530     }
00531 }
```

References SimpleDisplay::cout(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.6 forceError()

`void AI::forceError ( )  [static], [private]`

Forces an error in all Security devices.

Note

Error can be restored with recoverSecurity()

Definition at line 369 of file AI.cpp.

```
00369                     {
00370
00371     SimpleDisplay::printAIInterface();
00372     SimpleDisplay::cout("Forcing FAILURE in all security devices");
00373
00374     for (int i = 0; i < KernelDevice::numKerns; i++) {
00375         KernelDevice::allKernelDevices[i].securityHasBeenBroken = true;
00376     }
00377     SimpleDisplay::currentDisplay->setSecurityStatus(false);
00378
00379     sleep(SLEEPTIME*2);
00380 }
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, KernelDevice::numKerns, SimpleDisplay::printAIInterface(), KernelDevice::securityHasBeenBroken, SimpleDisplay::setSecurityStatus(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.7 microphone()

```
void AI::microphone ( ) [static], [private]
```

Sends a message using the microphone.

Definition at line 349 of file AI.cpp.

```
00349                       {
00350 
00351 
00352     try {
00353         std::string message;
00354 
00355         SimpleDisplay::cout("\nEnter a message:  ");
00356         getline(std::cin, message);
00357 
00358         SimpleDisplay::cout("[AI] SENDING MESSAGE:\n");
00359         SimpleDisplay::cout("|| -> " + message);
00360 
00361         sleep(SLEEPTIME);
00362 
00363     }catch (std::exception &e){
00364         SimpleDisplay::cout("Input not valid");
00365     }
00366 
00367 }
```

References SimpleDisplay::cout(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.8 recoverSecurity()

```
void AI::recoverSecurity ( ) [static], [private]
```

Recover all security breaches to safe status.

Definition at line 442 of file AI.cpp.

```
00442                            {
00443 
00444     SimpleDisplay::printAIInterface();
00445     SimpleDisplay::cout("Recovering all Devices to Secure Status...");
00446 
00447     for (int i = 0; i < KernelDevice::numKerns; i++) {
00448         KernelDevice::allKernelDevices[i].securityHasBeenBroken = false;
00449     }
00450     SimpleDisplay::currentDisplay->setSecurityStatus(true);
00451 
00452     sleep(SLEEPTIME*2);
00453 }
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, KernelDevice::numKerns, SimpleDisplay::printAIInterface(), KernelDevice::securityHasBeenBroken, SimpleDisplay::setSecurityStatus(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.9 refreshData()

```
void AI::refreshData ( )
```

Refresh last data to every device built.

Definition at line 52 of file AI.cpp.

```
00052                          {
00053
00054     // ----- Declarations ------
00055     int numDevices = DataDevice::getNumSensors();
00056
00057     // ----- Refreshing every data device one time ------
00058     for (int i = 0; i < DataDevice::numSensors; i++) {
00059
00060         DataDevice* target = &DataDevice::allDataDevices[i];
00061
00062         // Check if it's active
00063         if (!~(*target)) continue;
00064
00065         // Check if it is full
00066         if (target->collectedData.size() >= MAX_DATA) {
00067
00068             // Remove oldest measurement
00069             target->collectedData.erase(target->collectedData.begin());
00070
00071             // Remove oldest time
00072             target->collectedTimes.erase(target->collectedTimes.begin());
00073
00074         }
00075         // Append new measurement and time
00076         target->appendNewData();
00077     }
00078
00079     // ----- Refreshing every kernel device one time -----
00080     for (int i = 0; i < KernelDevice::numKerns; i++) {
00081
00082         KernelDevice* k = &KernelDevice::allKernelDevices[i];
00083         // Check if it's active
00084         if (!~(*k)) continue;
00085
00086         if (!(*k).securityGenerator() || k->securityHasBeenBroken) {
00087             k->securityHasBeenBroken = true;
00088             SimpleDisplay::currentDisplay->setSecurityStatus(false);
00089         }
00090     }
00091
00092 }
```

References DataDevice::allDataDevices, KernelDevice::allKernelDevices, DataDevice::appendNewData(), DataDevice::collectedData, DataDevice::collectedTimes, SimpleDisplay::currentDisplay, DataDevice::getNumSensors(), MAX_DATA, KernelDevice::numKerns, DataDevice::numSensors, KernelDevice::securityHasBeenBroken, and SimpleDisplay::setSecurityStatus().

Referenced by main(), and throwDemon().

4.2.4.10 removeUser()

```
void AI::removeUser ( )  [static], [private]
```

Calls User class to remove a existing user.

Definition at line 493 of file AI.cpp.

```
00493                         {
00494
00495     std::string id, confirmation;
00496
00497     SimpleDisplay::cout("[" + TAG + "] Insert the id of the user to be removed:  ");
00498     std::getline(std::cin,id);
00499     do {
00500         SimpleDisplay::cout("[" + TAG + "] Are you sure to remove " + id
```

```
        + " from the User list?\nThis can't be undone [y/n]:");
        std::getline(std::cin,confirmation);
    }while(confirmation != "y" && confirmation != "n");

    if (confirmation == "y") {
        try {
            User::rmUser(std::stoi(id));
        } catch (InvalidUserException &e) {
            SimpleDisplay::cout("[" + TAG + "] User requested not found");
        }
    }
    sleep(SLEEPTIME);

}
```

References SimpleDisplay::cout(), User::rmUser(), SLEEPTIME, and TAG.

Referenced by requestOption().

4.2.4.11 requestOption()

```
void AI::requestOption (
            std::string opt,
            bool isAdmin )  [static]
```

Requests an option from main settings interface.

Parameters

opt	Option to select
isAdmin	Admin status of current user

Definition at line 134 of file AI.cpp.

```
00134                                                         {
00135     try {
00136         int num = atoi(opt.c_str());
00137 
00138         switch (num) {
00139 
00140             case TURNDEVICES:
00141                 turnDevices();
00142                 break;
00143 
00144             case TURNSECURITY:
00145                 turnSecurity();
00146                 break;
00147 
00148             case CHECKSECURITY:
00149                 checkSecurity();
00150                 break;
00151 
00152             case MICROPHONE:
00153                 microphone();
00154                 break;
00155 
00156             case FORCEERROR:  {
00157                 try {
00158                     AI::ensureAdminAccess(isAdmin);
00159                 }catch (InsufficientPermissionsException &e){
00160                     break;
00161                 }
00162 
00163                 forceError();
00164                 break;
00165             }
00166 
00167             case BUILDDEVICE:{
00168                 try {
00169                     AI::ensureAdminAccess(isAdmin);
```

```
                }catch (InsufficientPermissionsException &e){
                    break;
                }

                buildDevice();
                break;
            }

            case ADDUSER:{
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch(InsufficientPermissionsException &e){
                    break;
                }
                addUser();
                break;
            }

            case REMOVEUSER:{
                try{
                    AI::ensureAdminAccess(isAdmin);
                }catch(InsufficientPermissionsException &e){
                    break;
                }
                removeUser();
                break;
            }

            case RETURNSECURITY:  {
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch (InsufficientPermissionsException &e){
                    break;
                }
                recoverSecurity();
                break;
            }

            case FORCEBADALLOC:{
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch (InsufficientPermissionsException &e){
                    break;
                }
                forceBadAlloc();
                break;
            }

            case SHOWUSERS:  {
                try {
                    AI::ensureAdminAccess(isAdmin);
                } catch (InsufficientPermissionsException &e){
                    break;
                }
                showAllUsers();
                break;
            }

            case EXIT:
                break;
        }


    }catch (std::exception &e){
        SimpleDisplay::cout("Option NOT valid");
        sleep(1);
    }

}
```

References ADDUSER, addUser(), BUILDDEVICE, buildDevice(), CHECKSECURITY, checkSecurity(), SimpleDisplay::cout(), ensureAdminAccess(), EXIT, FORCEBADALLOC, forceBadAlloc(), FORCEERROR, forceError(), MICROPHONE, microphone(), recoverSecurity(), REMOVEUSER, removeUser(), RETURNSECURITY, showAllUsers(), SHOWUSERS, TURNDEVICES, turnDevices(), TURNSECURITY, and turnSecurity().

Referenced by main().

4.2.4.12 showAllUsers()

```
void AI::showAllUsers ( )  [static], [private]
```

Calls to print all users registered.

Definition at line 550 of file AI.cpp.

```
00550                 {
00551     User::printUsers();
00552
00553     std::cout « "\nPress ENTER to return";
00554     std::cin.get();
00555
00556     //  Clear bad inputs
00557     std::cin.clear();
00558
00559 }
```

References User::printUsers().

Referenced by requestOption().

4.2.4.13 signalHandler()

```
void AI::signalHandler (
            int signum )  [static], [private]
```

Function to end the program if the exit call is raised

Parameters

signum	Exit call

Warning

Function should not be used directly. Will be controlled by AI

Definition at line 533 of file AI.cpp.

```
00533                            {
00534
00535     // ----- Save all data -----
00536     User::saveUserData();
00537
00538     // ----- Send abortation signal -----
00539     abort = true;
00540     SimpleDisplay::security_abort = true;
00541
00542     // ----- Display EXIT CODE -----
00543     std::cout « "[" + TAG + "] Exit call detected.  Exiting JVH Systems...";
00544     sleep(SLEEPTIME);
00545     std::cout « "\n[" + TAG + "] Thank you for trusting us!\n\n";
00546     exit(0);
00547
00548 }
```

References abort, User::saveUserData(), SimpleDisplay::security_abort, SLEEPTIME, and TAG.

Referenced by AI().

4.2.4.14 startCollectingData()

```
void AI::startCollectingData ( )
```

Search for every DataDevice and fills all its data storage with measurements.

Warning

All devices must be built before using this function.

Definition at line 32 of file AI.cpp.

```
00032                               {
00033
00034     // ----- Intializing ------
00035     std::cout « "[AI] Collecting Data" « std::endl;
00036     int dataCounter = 0;
00037
00038     // ----- Filling every device ------
00039     int numDevices = DataDevice::getNumSensors();
00040     for (int device = 0; device < numDevices; device++)
00041         for (int data = 0; data < MAX_DATA; data++) {
00042             DataDevice::allDataDevices[device].appendNewData();
00043             dataCounter++;
00044         }
00045
00046     // ----- Sending Output ------
00047     std::cout « "[AI] Collected " « dataCounter « " measures" « std::endl;
00048 }
```

References DataDevice::allDataDevices, DataDevice::appendNewData(), DataDevice::getNumSensors(), and MAX_DATA.

Referenced by main().

4.2.4.15 throwDemon()

```
void AI::throwDemon ( )
```

Throws a process-like demon, which will be updating the DataDevice measurements while waiting for key-interruption.

Definition at line 95 of file AI.cpp.

```
00095                      {
00096
00097 #ifdef _WIN32
00098
00099     while (true) {
00100         try {
00101
00102             if (kbhit()) break;
00103             if (abort){
00104                 SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1
      instead.\n");
00105                 exit(0);
00106             }
00107             refreshData();
00108             sleep(SLEEPTIME);
00109
00110         } catch (std::exception &e) {
00111             sleep(SLEEPTIME);
00112             continue;
00113         }
00114     }
00115 #else
00116     while (true){
00117     try{
00118
00119         if (kb::kbhit()) break;
```

```
00120         if (abort){
00121             SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1 instead.\n");
00122             exit(0);
00123         }
00124         refreshData();
00125         sleep(SLEEPTIME);
00126 
00127     }catch (std::exception e){
00128         sleep(SLEEPTIME);
00129     }
00130 }
00131 #endif
00132 }
```

References abort, SimpleDisplay::cout(), kb::kbhit(), refreshData(), and SLEEPTIME.

Referenced by main().

4.2.4.16 turnDevices()

`void AI::turnDevices ( )  [static], [private]`

Turns all devices on/off.

Definition at line 270 of file AI.cpp.

```
00270                           {
00271 
00272     // ----- Instantiating IODi for easy device control -----
00273     IODi iodi;
00274 
00275     // ----- Turning all Devices ON/OFF -----
00276     for (int i = 0; i < DataDevice::numSensors; i++) {
00277         iodi.setCurrentDevice(i);
00278 
00279         if (~DataDevice::allDataDevices[i])
00280             iodi.requestSkill("turnoff");
00281         else
00282             iodi.requestSkill("turnon");
00283 
00284     }
00285 
00286     // ----- Exiting...  -----
00287     std::string none;
00288     SimpleDisplay::cout("Press ENTER to continue");
00289     getline(std::cin,none);
00290 }
```

References DataDevice::allDataDevices, SimpleDisplay::cout(), DataDevice::numSensors, IODi::requestSkill(), and IODi::setCurrentDevice().

Referenced by requestOption().

4.2.4.17 turnSecurity()

`void AI::turnSecurity ( )  [static], [private]`

Turns all security on/off.

Definition at line 292 of file AI.cpp.

```
00292                            {
00293 
00294     for (int i = 0; i < KernelDevice::numKerns; i++) {
00295 
```

```
        // Accessing Device
        KernelDevice *k = &KernelDevice::allKernelDevices[i];

        // Checking Status
        k->isActive = !~(*k);

        // Turning ON/OFF
        if (~(*k)) {
            SimpleDisplay::cout(k->getName() + " turned ON\n");
            SimpleDisplay::currentDisplay->setSecurityConnected(true);
        }
        else {
            SimpleDisplay::cout(k->getName() + " turned OFF\n");
            SimpleDisplay::currentDisplay->setSecurityConnected(false);
        }
    }

    // ----- Exiting -----
    std::string none;
    SimpleDisplay::cout("Press ENTER to continue");
    getline(std::cin,none);
}
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, Device::getName(), Device::isActive, KernelDevice::numKerns, and SimpleDisplay::setSecurityConnected().

Referenced by requestOption().

4.2.5 Member Data Documentation

4.2.5.1 abort

`bool AI::abort = false [static], [private]`

Alerts if exit request has been detected.

Definition at line 137 of file AI.h.

Referenced by signalHandler(), and throwDemon().

4.2.5.2 MAX_DATA

`int AI::MAX_DATA = 30 [static], [private]`

Sets maximum of data/measurements collected from Data Devices.

Definition at line 139 of file AI.h.

Referenced by refreshData(), and startCollectingData().

4.2.5.3 SLEEPTIME

`int AI::SLEEPTIME = 1 [static], [private]`

Sets time to sleep between interactions.

Definition at line 141 of file AI.h.

Referenced by addUser(), buildDevice(), ensureAdminAccess(), forceBadAlloc(), forceError(), microphone(), recoverSecurity(), removeUser(), signalHandler(), and throwDemon().

4.2.5.4 TAG

`std::string AI::TAG = "AI" [static], [private]`

Name of class (for exception throwing)

Definition at line 135 of file AI.h.

Referenced by ensureAdminAccess(), removeUser(), and signalHandler().

4.2.5.5 time_counter

`int AI::time_counter [private]`

Used for counting number of time-stamps.

Definition at line 143 of file AI.h.

Referenced by AI().

The documentation for this class was generated from the following files:

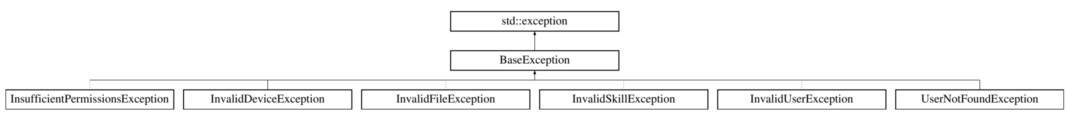
- AI.h
- AI.cpp

4.3 BaseException Class Reference

This class establishes a basic format for JVH exceptions.

```
#include <BaseException.h>
```

Inheritance diagram for BaseException:



Public Member Functions

- BaseException ()
  Builds a new BaseException.
- const char * what ()
  Prints the cause of the exception with the tag of the owner.

Protected Attributes

- std::string tag
  Owner/Causant of the exception.
- std::string exception
  Description of the exception.

4.3.1 Detailed Description

This class establishes a basic format for JVH exceptions.

Definition at line 16 of file BaseException.h.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 BaseException()

```
BaseException::BaseException ( )
```

Builds a new BaseException.

Definition at line 2 of file BaseException.cpp.

```
00002 {}
```

4.3.3 Member Function Documentation

4.3.3.1 what()

```
const char * BaseException::what ( )
```

Prints the cause of the exception with the tag of the owner.

Returns

Exception info

Definition at line 4 of file BaseException.cpp.

```
00004                                  {
00005
00006     return ("[" + tag + "] " + exception).c_str();
00007
00008 }
```

References exception, and tag.

4.3.4 Member Data Documentation

4.3.4.1 exception

```
std::string BaseException::exception [protected]
```

Description of the exception.

Definition at line 38 of file BaseException.h.

Referenced by InsufficientPermissionsException::InsufficientPermissionsException(), InvalidDeviceException::InvalidDeviceException
InvalidFileException::InvalidFileException(), InvalidSkillException::InvalidSkillException(), InvalidUserException::InvalidUserException
UserNotFoundException::UserNotFoundException(), and what().

4.3.4.2 tag

```
std::string BaseException::tag [protected]
```

Owner/Causant of the exception.

Definition at line 36 of file BaseException.h.

Referenced by InsufficientPermissionsException::InsufficientPermissionsException(), InvalidDeviceException::InvalidDeviceException
InvalidFileException::InvalidFileException(), InvalidSkillException::InvalidSkillException(), InvalidUserException::InvalidUserException
UserNotFoundException::UserNotFoundException(), and what().

The documentation for this class was generated from the following files:

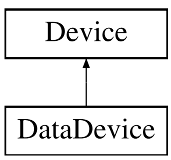
- BaseException.h
- BaseException.cpp

4.4 DataDevice Class Reference

This class will contain specific properties of output devices and a set of functionalities for managing and checking their data. It inherits properties from Device class.

```
#include <DataDevice.h>
```

Inheritance diagram for DataDevice:



Public Member Functions

- DataDevice ()

  Creates a New DataDevice. It uses invalid properties by default. Not recommended.
- DataDevice (std::string name, int id, std::vector< void(∗)()> ∗skill, std::string ∗skillNames)

  Creates a new DataDevice and assign its basic properties.
- void appendNewData ()

  Collect new measurement from main data generator and saves it in data vectors.

Static Public Member Functions

- static int getNumSensors ()

  Returns the current number of DataDevices detected.

Static Public Attributes

- static DataDevice ∗ allDataDevices

  Pointers to all Data Devices built.
- static std::string ∗ allDataDeviceNames

  Pointers to all Data Device names.

Private Member Functions

- void setLimit (int ∗limitArray)

  Sets a Warning limit for the device. This limit will be supervised by AI.

Static Private Member Functions

- static void setAllDataDevices (DataDevice ∗allDevices)

  Sets all pointers to DataDevices. Used for general Data Device control.

Private Attributes

- std::vector< int > collectedData

  vector with all measurements collected from the sensor
- std::vector< std::string > collectedTimes

  Vector with all time-stamps of the measurements.
- int(∗ dataGenerator)()

  Main generator of measurements/data of the sensor.
- bool needsLimit = false

  Distinguish between Data Devices which uses or not Limits.
- int ∗ limit

  Limit to control by AI.

Static Private Attributes

- static int numSensors = 0

    Number of DataDevices stablished.
- static int numSensorsActive = 0

    Number of DataDevices turned on.

Friends

- class DeviceBuilder
- class AI
- class IODi

Additional Inherited Members

4.4.1 Detailed Description

This class will contain specific properties of output devices and a set of functionalities for managing and checking their data. It inherits properties from Device class.

Definition at line 16 of file DataDevice.h.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 DataDevice() [1/2]

```
DataDevice::DataDevice ( )
```

Creates a New DataDevice. It uses invalid properties by default. Not recommended.

Definition at line 24 of file DataDevice.cpp.

```
00024                             {
00025     this->name = "";
00026     this->id = INVALID_DEVICE;
00027     this->isDataDevice = true;
00028
00029 };
```

References Device::INVALID_DEVICE, Device::isDataDevice, and Device::name.

4.4.2.2 DataDevice() [2/2]

```
DataDevice::DataDevice (
            std::string name,
            int id,
            std::vector< void(*)()> * skill,
            std::string * skillNames )
```

Creates a new DataDevice and assign its basic properties.

Parameters

name	public name for user
id	private identifier for AI
skillVector	Vector with all skills implemented in the devices as pointers to functions
skillNames	Names for the skills implemented.

Note

Skills and skill-names must be in the same order so the kernel can show them to the user.

Definition at line 32 of file DataDevice.cpp.

```
00032
        {
00033
00034     // ----- Counting Devices ------
00035     std::cout « "[DataDevice] New Device Detected:  0x" « id « std::endl;
00036     numSensors++;
00037
00038     // ----- Property assignation ------
00039     this->name = name;
00040     this->id = id;
00041     this->skills = skillVector;
00042     this->skillNames = skillNames;
00043     this->isDataDevice = true;
00044     this->numSkills = skillVector->size();
00045     this->isActive = true;
00046 }
```

References Device::id, Device::isActive, Device::isDataDevice, Device::name, numSensors, Device::numSkills, Device::skillNames, and Device::skills.

4.4.3 Member Function Documentation

4.4.3.1 appendNewData()

```
void DataDevice::appendNewData ( )
```

Collect new measurement from main data generator and saves it in data vectors.

Definition at line 55 of file DataDevice.cpp.

```
00055                                      {
00056     auto clock = std::chrono::system_clock::now();
00057     time_t time = std::chrono::system_clock::to_time_t(clock);
00058     std::string time_stamp = std::ctime(&time);
00059     this->collectedData.push_back(this->dataGenerator());
00060     this->collectedTimes.push_back(time_stamp);
00061 }
```

References collectedData, collectedTimes, and dataGenerator.

Referenced by AI::refreshData(), and AI::startCollectingData().

4.4.3.2 getNumSensors()

```
int DataDevice::getNumSensors ( ) [static]
```

Returns the current number of DataDevices detected.

Returns

Number of detected Sensors

Definition at line 49 of file DataDevice.cpp.

```
00049                            {
00050     return numSensors;
00051 };
```

References numSensors.

Referenced by AI::refreshData(), IODi::requestNumberOfDataDevices(), IODi::setCurrentDevice(), and AI::startCollectingData().

4.4.3.3 setAllDataDevices()

```
void DataDevice::setAllDataDevices (
             DataDevice * allDevices ) [static], [private]
```

Sets all pointers to DataDevices. Used for general Data Device control.

Warning

It can cause conflict with other processes, recommended only with builder implementation.

Definition at line 76 of file DataDevice.cpp.

```
00076                                                        {
00077     allDataDevices = allDevices;
00078     allDataDeviceNames = new std::string[numSensors];
00079 
00080     for (int i = 0; i < numSensors; i++){
00081         allDataDeviceNames[i] = allDevices[i].getName();
00082     }
00083 }
```

References allDataDeviceNames, allDataDevices, Device::allDevices, Device::getName(), and numSensors.

Referenced by DeviceBuilder::buildAllDevices().

4.4.3.4 setLimit()

```
void DataDevice::setLimit (
             int * limitArray ) [private]
```

Sets a Warning limit for the device. This limit will be supervised by AI.

Warning

It can cause conflict with other processes, recommended only with builder implementation

Parameters

limitArray	array of values to supervise.

Definition at line 69 of file DataDevice.cpp.

```
00069                                         {
00070     this->limit = limitArray;
00071     this->needsLimit = true;
00072 }
```

References limit, and needsLimit.

4.4.4 Friends And Related Function Documentation

4.4.4.1 AI

`friend class AI  [friend]`

Definition at line 20 of file DataDevice.h.

4.4.4.2 DeviceBuilder

`friend class DeviceBuilder  [friend]`

Definition at line 19 of file DataDevice.h.

4.4.4.3 IODi

`friend class IODi  [friend]`

Definition at line 21 of file DataDevice.h.

4.4.5 Member Data Documentation

4.4.5.1 allDataDeviceNames

`std::string * DataDevice::allDataDeviceNames  [static]`

Pointers to all Data Device names.

Definition at line 59 of file DataDevice.h.

Referenced by IODi::requestDeviceNames(), setAllDataDevices(), and IODi::setCurrentDevice().

4.4.5.2 allDataDevices

`DataDevice * DataDevice::allDataDevices [static]`

Pointers to all Data Devices built.

Definition at line 57 of file DataDevice.h.

Referenced by AI::refreshData(), IODi::requestAllStatus(), IODi::requestDeviceSkills(), IODi::requestNumberOfSkills(), IODi::requestSkill(), setAllDataDevices(), AI::startCollectingData(), and AI::turnDevices().

4.4.5.3 collectedData

`std::vector<int> DataDevice::collectedData [private]`

vector with all measurements collected from the sensor

Definition at line 87 of file DataDevice.h.

Referenced by appendNewData(), AI::refreshData(), and IODi::requestSkill().

4.4.5.4 collectedTimes

`std::vector<std::string> DataDevice::collectedTimes [private]`

Vector with all time-stamps of the measurements.

Definition at line 89 of file DataDevice.h.

Referenced by appendNewData(), AI::refreshData(), and IODi::requestSkill().

4.4.5.5 dataGenerator

`int(* DataDevice::dataGenerator) () [private]`

Main generator of measurements/data of the sensor.

Definition at line 91 of file DataDevice.h.

Referenced by appendNewData().

4.4.5.6 limit

```
int* DataDevice::limit [private]
```

Limit to control by AI.

Definition at line 95 of file DataDevice.h.

Referenced by setLimit().

4.4.5.7 needsLimit

```
bool DataDevice::needsLimit = false [private]
```

Distinguish between Data Devices which uses or not Limits.

Definition at line 93 of file DataDevice.h.

Referenced by setLimit().

4.4.5.8 numSensors

```
int DataDevice::numSensors = 0 [static], [private]
```

Number of DataDevices stablished.

Definition at line 110 of file DataDevice.h.

Referenced by AI::buildDevice(), DataDevice(), getNumSensors(), AI::refreshData(), IODi::requestAllStatus(), setAllDataDevices(), and AI::turnDevices().

4.4.5.9 numSensorsActive

```
int DataDevice::numSensorsActive = 0 [static], [private]
```

Number of DataDevices turned on.

Definition at line 112 of file DataDevice.h.

The documentation for this class was generated from the following files:

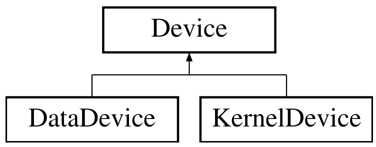
- DataDevice.h
- DataDevice.cpp

4.5 Device Class Reference

This class will contain every device as a general low-level definition, containing the skills, properties and a set of functionalities for managing all devices or modifying them individually. Every device will be built from the builder and the data managed will be managed from AI and IODi. This class will also be the base for constructing DataDevices and KernelDevices.

```
#include <Device.h>
```

Inheritance diagram for Device:



Public Member Functions

- Device ()

    Builds a New Device. Sets Invalid properties by default. Not recommended.
- Device (std::string deviceName, int deviceId)

    Creates a New Device and assign a identifier.
- void operator[ ] (int index)

    Uses a built-in device-skill calling it by its name.
- void operator[ ] (std::string skillName)

    Uses a built-in device-skill calling it by its position.
- bool operator∼ ()

    Checks if device is active.
- std::string getName ()

    Returns the name of the device.
- int getNumSkills ()

    Returns number of skills of the device.

Static Public Attributes

- static Device ∗ allDevices

    Pointer to array with all Devices built.

Protected Types

- enum { INVALID_DEVICE = -1 }

Protected Attributes

- bool isDataDevice

    Distinguish between Data Devices and other Devices.
- bool isActive

    (True). Is turned ON. (False). Is turned OFF.
- int id

    Kernel number of identification.
- int numSkills

    Number of skills detected for the device.
- std::string ∗ skillNames

    Pointers to names of each skill.
- std::string name

    Name of the device.
- std::vector< void(∗)()> ∗ skills

    Pointer to all functions (skills) build.

Static Private Member Functions

- static void setAllDevices (Device ∗allNewDevices, std::string ∗allNewDeviceNames, int numDevices)

    Sets all pointers to Devices. Used to general device control, recommended only for DeviceBuilder.
- static void setDeviceCounter (int num)

    Sets number of Detected Devices. Used to general device control, recommended only for DeviceBuilder.

Static Private Attributes

- static std::string TAG = "Device"

    Name of class (for exception throwing)
- static std::string ∗ allDeviceNames

    Pointers to every Device built.
- static int numDevices

    Number of Devices built.

Friends

- class DeviceBuilder
- class IODi
- class AI

4.5.1 Detailed Description

This class will contain every device as a general low-level definition, containing the skills, properties and a set of functionalities for managing all devices or modifying them individually. Every device will be built from the builder and the data managed will be managed from AI and IODi. This class will also be the base for constructing DataDevices and KernelDevices.

Definition at line 20 of file Device.h.

4.5.2 Member Enumeration Documentation

4.5.2.1 anonymous enum

```
anonymous enum [protected]
```

Enumerator

INVALID_DEVICE	

Definition at line 112 of file Device.h.
```
00112         {
00113         INVALID_DEVICE = -1  // Change to set invalid ID of devices
00114     };
```

4.5.3 Constructor & Destructor Documentation

4.5.3.1 Device() [1/2]

`Device::Device ( )`

Builds a New Device. Sets Invalid properties by default. Not recommended.

Definition at line 22 of file Device.cpp.
```
00022 :   name(""),id(INVALID_DEVICE){};
```

4.5.3.2 Device() [2/2]

```
Device::Device (
            std::string deviceName,
            int deviceId )
```

Creates a New Device and assign a identifier.

Parameters

name	Public name for device
id	Private identifier for device

Definition at line 25 of file Device.cpp.
```
00025 :   name(deviceName),id(deviceId){}
```

4.5.4 Member Function Documentation

4.5.4.1 getName()

`std::string Device::getName ( )`

Returns the name of the device.

Returns

Name of device

Definition at line 29 of file Device.cpp.

```
00029                                    {
00030     return this->name;
00031 }
```

References name.

Referenced by AI::checkSecurity(), IODi::requestSkill(), DataDevice::setAllDataDevices(), and AI::turnSecurity().

4.5.4.2 getNumSkills()

```
int Device::getNumSkills ( )
```

Returns number of skills of the device.

Returns

Number of skills

Definition at line 33 of file Device.cpp.

```
00033                                {
00034     return this->numSkills;
00035 }
```

References numSkills.

Referenced by IODi::requestDeviceSkills(), and IODi::requestNumberOfSkills().

4.5.4.3 operator[]() [1/2]

```
void Device::operator[] (
             int index )
```

Uses a built-in device-skill calling it by its name.

Parameters

skillName	Name of the skill

Exceptions

InvalidSkillException	Skill-index not valid/found

Definition at line 38 of file Device.cpp.

```
00038                                         {
```

```
    try{
        (*this->skills)[index]();

    }catch (std::exception){
        throw InvalidSkillException(TAG);
    }
}
```

References skills, and TAG.

4.5.4.4 operator[]() [2/2]

```
void Device::operator[] (
            std::string skillName )
```

Uses a built-in device-skill calling it by its position.

Parameters

index	Position of the skill (order of implementation)

Exceptions

InvalidSkillException	Skill-name not valid

Definition at line 48 of file Device.cpp.

```
                                          {
    // ----- Set number of skills ------
    int skillSize = this->numSkills;

    // ----- Find and use Skill ------
    for (int i = 0; i < skillSize; i++){
        if (skillName == skillNames[i])
            try{
                (*this->skills)[i]();
                break;
            }catch (std::exception){
                throw InvalidSkillException(TAG);
            }

    }
};
```

References numSkills, skillNames, skills, and TAG.

4.5.4.5 operator∼()

```
bool Device::operator∼ ( )
```

Checks if device is active.

Returns

Status of the device. (True) Active. (False) Inactive.

Definition at line 66 of file Device.cpp.

```
                         {
    return this->isActive;
}
```

References isActive.

4.5.4.6 setAllDevices()

```
void Device::setAllDevices (
            Device * allNewDevices,
            std::string * allNewDeviceNames,
            int numDevices )  [static], [private]
```

Sets all pointers to Devices. Used to general device control, recommended only for DeviceBuilder.

Parameters

allNewDevices	Array with pointers to all Devices
Array	with names for each Device
numDevices	Number of devices to be setted

Definition at line 76 of file Device.cpp.

```
00076                                                                                            {
00077
00078     // ----- Setting Device pointers and names ------
00079     std::cout « "[DEVICES] Collecting all devices:  " « std::endl;
00080     Device::allDevices = allNewDevices;
00081     Device::allDeviceNames = allNewDeviceNames;
00082     Device::numDevices = numDevices;
00083
00084     // ----- Log Success Device ------
00085     for (int i = 0; i < numDevices; i++){
00086         std::cout « "-> (SUCCESS) " « allNewDeviceNames[i] « std::endl;
00087     }
00088
00089 }
```

References allDeviceNames, allDevices, and numDevices.

Referenced by DeviceBuilder::buildAllDevices().

4.5.4.7 setDeviceCounter()

```
void Device::setDeviceCounter (
            int num )  [static], [private]
```

Sets number of Detected Devices. Used to general device control, recommended only for DeviceBuilder.

Parameters

num	Number of devices (total)

Definition at line 92 of file Device.cpp.

```
00092                                            {
00093     Device::numDevices = num;
00094 }
```

References numDevices.

4.5.5 Friends And Related Function Documentation

4.5.5.1 AI

```
friend class AI [friend]
```

Definition at line 24 of file Device.h.

4.5.5.2 DeviceBuilder

```
friend class DeviceBuilder [friend]
```

Definition at line 22 of file Device.h.

4.5.5.3 IODi

```
friend class IODi [friend]
```

Definition at line 23 of file Device.h.

4.5.6 Member Data Documentation

4.5.6.1 allDeviceNames

```
std::string * Device::allDeviceNames [static], [private]
```

Pointers to every Device built.

Definition at line 87 of file Device.h.

Referenced by setAllDevices().

4.5.6.2 allDevices

```
Device * Device::allDevices [static]
```

Pointer to array with all Devices built.

Definition at line 73 of file Device.h.

Referenced by DataDevice::setAllDataDevices(), setAllDevices(), and KernelDevice::setAllKernelDevices().

4.5.6.3 id

```
int Device::id [protected]
```

Kernel number of identification.

Definition at line 123 of file Device.h.

Referenced by DataDevice::DataDevice(), and KernelDevice::KernelDevice().

4.5.6.4 isActive

```
bool Device::isActive [protected]
```

(True). Is turned ON. (False). Is turned OFF.

Definition at line 121 of file Device.h.

Referenced by DataDevice::DataDevice(), KernelDevice::KernelDevice(), operator∼(), IODi::requestSkill(), and AI::turnSecurity().

4.5.6.5 isDataDevice

```
bool Device::isDataDevice [protected]
```

Distinguish between Data Devices and other Devices.

Definition at line 119 of file Device.h.

Referenced by DataDevice::DataDevice(), and KernelDevice::KernelDevice().

4.5.6.6 name

```
std::string Device::name [protected]
```

Name of the device.

Definition at line 130 of file Device.h.

Referenced by DataDevice::DataDevice(), getName(), and KernelDevice::KernelDevice().

4.5.6.7 numDevices

```
int Device::numDevices [static], [private]
```

Number of Devices built.

Definition at line 89 of file Device.h.

Referenced by AI::buildDevice(), setAllDevices(), and setDeviceCounter().

4.5.6.8 numSkills

```
int Device::numSkills [protected]
```

Number of skills detected for the device.

Definition at line 125 of file Device.h.

Referenced by DataDevice::DataDevice(), getNumSkills(), and operator[ ]().

4.5.6.9 skillNames

```
std::string* Device::skillNames [protected]
```

Pointers to names of each skill.

Warning

Must have the same order that numSkills

Definition at line 128 of file Device.h.

Referenced by DataDevice::DataDevice(), operator[ ](), IODi::requestDeviceSkills(), and IODi::requestSkill().

4.5.6.10 skills

```
std::vector<void(*)()>* Device::skills [protected]
```

Pointer to all functions (skills) build.

Definition at line 132 of file Device.h.

Referenced by DataDevice::DataDevice(), and operator[ ]().

4.5.6.11 TAG

```
std::string Device::TAG = "Device"  [static], [private]
```

Name of class (for exception throwing)

Definition at line 85 of file Device.h.

Referenced by operator[ ]().

The documentation for this class was generated from the following files:

- Device.h
- Device.cpp

4.6 DeviceBuilder Class Reference

This class lets a high-level programmer add easily more devices to the main program. You can use the implemented public functions wherever you need. For adding a new device you can follow this steps:

```
#include <DeviceBuilder.h>
```

Public Member Functions

- DeviceBuilder (std::string name="STD_DEVICE")

  Creates a temporal builder for specific device.
- void setSkill (void(∗skill)(), std::string skillName)

  Build one skill into the device Builder. All names and skills of each device will be ordered and used in kernel.
- void setLimit (int ∗newLimit)

  Builds limits into the device Builder. All limits will be controlled and implemented in execution by the kernel.
- void setAsDataDevice ()

  Sets type of Device as DataDevice.
- void setAsKernelDevice ()

  Sets type of Device as KernelDevice.
- void setDataGenerator (int(∗dataGenerator)())

  Sets the main data generator for DataDevices.
- void setSecurityGenerator (bool(∗securityGenerator)())

  Sets the security generator for KernelDevices.

Static Public Member Functions

- static void buildAllDevices ()

  Builds every builder instances.
- static SimpleDisplay ∗ buildDisplay ()

  Builds and returns a new SimpleDisplay object with properties according to builder devices.
- static void clearAll ()

  Kills every builder instances.

Private Member Functions

- void buildDKDevice ()

  Builds selected device as data or kernel device into the kernel.

Private Attributes

- std::string name = "STD Device"

  Name of Device.
- std::vector< void(∗)()> skillVector

  Skills of the Device.
- std::vector< std::string > skillNamesVector

  Skill-names of each skill.
- int(∗ dataGenerator)()

  Device's function to generate data.
- bool(∗ securityGenerator)()

  Kernel's Devices function to check security.
- int id

  Identifier of Device.
- int ∗ limit = nullptr

  Limits of the Device.
- bool isDataDevice

  Type of device. (True) Data Device. (False) Kernel Device.

Static Private Attributes

- static std::vector< DeviceBuilder ∗ > allBuilders

  Vector with all builders instantiated.
- static std::vector< Device > allDevices

  Vector with all Devices instantiated.
- static std::vector< DataDevice > allDataDevices

  Vector with all Data Devices instantiated.
- static std::vector< KernelDevice > allKernelDevices

  Vector with all Kernel Devices instantiated.
- static int deviceCounter

  Number of devices detected.

4.6.1 Detailed Description

This class lets a high-level programmer add easily more devices to the main program. You can use the implemented public functions wherever you need. For adding a new device you can follow this steps:

1. Instantiate a deviceBuilder with DeviceBuilder();
2. Set the type of device with setAsDataDevice() or setAsKernelDevice();
3. Set the main generator (function where the device extracts data) with setDataGenerator() or setSecurityGenerator()
4. (Optional) Add some skills (more functionalities) to your device with setSkill().
5. (Optional) Set a warning if measurement surpasses a limit with setLimit()
6. Repeat this process with as many devices as you need.
7. Use buildAllDevices() to build every instance.

Note

This class must be inited by the main class and will configure 3 classes. First, the builder will set every input device into the main class and set their corresponding global commands. Then it will build every output device into the Device class assigning the high-level functionalities and properties. Finally, the builder will assign AI properties of devices into the AI class if requested. When the relationships between high and low layer are established, the builder can be destroyed.

Definition at line 34 of file DeviceBuilder.h.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 DeviceBuilder()

```
DeviceBuilder::DeviceBuilder (
              std::string name = "STD_DEVICE" )
```

Creates a temporal builder for specific device.

Parameters

name	Name for the device.

Note

ID is assigned by kernel.

Warning

Device must be set as kernel or data device for being able to be built.

Definition at line 96 of file DeviceBuilder.cpp.

```
00096                                            {
00097 
00098     // ----- Name Assign ------
00099     std::cout « "[BUILDER] Building " « name « std::endl;
00100     this->name = name;
00101 
00102     // ----- Storing Builder Pointer------
00103     DeviceBuilder* devicePointer = this;
00104     allBuilders.push_back(devicePointer);
00105 
00106     // ----- Settling Device ID ------
00107     enum {FANCY_ID = 55324,FANCY_SUM = 37};
00108     //Shhh..  It looks cooler with strange nums
00109     this->id = deviceCounter + FANCY_ID;
00110     deviceCounter += FANCY_SUM;
00111 
00112 };
```

References allBuilders, deviceCounter, and name.

4.6.3 Member Function Documentation

4.6.3.1 buildAllDevices()

```
void DeviceBuilder::buildAllDevices ( )  [static]
```

Builds every builder instances.

Warning

This function should only be called before starting or resetting the main program.

Definition at line 37 of file DeviceBuilder.cpp.

```
00037                                           {
00038 
00039     // ----- Initializing ------
00040     std::cout « "[BUILDER] Sending " « allBuilders.size() « " Devices to Kernel" « std::endl;
00041 
00042     // ----- Declarations ------
00043     std::string *deviceNames = new std::string[allBuilders.size()]; // Array to Store Device names
00044     int numBuilders = allBuilders.size();
00045 
00046     // ----- Building All Devices Saved as Kernel or Data Device ------
00047     for (int i = 0; i < numBuilders; i++){
00048 
00049         // Saving device name in array
00050         deviceNames[i] = (*allBuilders[i]).name;
00051 
00052         // Building as Data or Kernel Device
00053         (*allBuilders[i]).buildDKDevice();
00054     }
00055 
00056     std::cout « "[BUILDER] Optimizing property access" « std::endl;
00057     // ----- All Devices Vector to Array ------
00058     Device *deviceArray = new Device[allDevices.size()];
00059     for (int i = 0; i < allDevices.size(); i++){
00060         deviceArray[i] = allDevices[i];
00061     }
00062 
00063     // ----- All DataDevices Vector to Array ------
00064     DataDevice *dataDeviceArray = new DataDevice[allDataDevices.size()];
00065     for (int i = 0; i < allDataDevices.size(); i++){
00066         dataDeviceArray[i] = allDataDevices[i];
00067     }
00068 
00069 
00070     // ----- All KernelDevices Vector to Array ------
00071     KernelDevice *kernelDeviceArray = new KernelDevice[allKernelDevices.size()];
00072     for (int i = 0; i < allKernelDevices.size(); i++){
00073         kernelDeviceArray[i] = allKernelDevices[i];
00074     }
00075 
00076     // ----- Storing Arrays ------
00077     std::cout « "[BUILDER] Storing General properties to Kernel..." « std::endl;
00078     Device::setAllDevices(deviceArray, deviceNames,numBuilders);
00079     DataDevice::setAllDataDevices(dataDeviceArray);
00080     KernelDevice::setAllKernelDevices(kernelDeviceArray);
00081 
00082 }
```

References allBuilders, allDataDevices, allDevices, allKernelDevices, buildDKDevice(), name, DataDevice::setAllDataDevices(), Device::setAllDevices(), and KernelDevice::setAllKernelDevices().

Referenced by AI::buildDevice(), and main().

4.6.3.2 buildDisplay()

```
SimpleDisplay * DeviceBuilder::buildDisplay ( )  [static]
```

Builds and returns a new SimpleDisplay object with properties according to builder devices.

Returns

Display with properties

Note

Must be used after buildAllDevices()

Definition at line 24 of file DeviceBuilder.cpp.

```
                                    {

    SimpleDisplay* display = new SimpleDisplay();
    display->setDataDevices(allDataDevices.size());
    display->setKernelDevices(allKernelDevices.size());
    display->setSecurityConnected(true);
    display->setSecurityStatus(true);
    return display;

}
```

References allDataDevices, allKernelDevices, SimpleDisplay::setDataDevices(), SimpleDisplay::setKernelDevices(), SimpleDisplay::setSecurityConnected(), and SimpleDisplay::setSecurityStatus().

Referenced by main().

4.6.3.3 buildDKDevice()

```
void DeviceBuilder::buildDKDevice ( ) [private]
```

Builds selected device as data or kernel device into the kernel.

Warning

This will NOT kill the builder.

All properties must be settled before using.

Definition at line 153 of file DeviceBuilder.cpp.

```
                              {

    // ----- Skills to Dynamic Vector (so it doesn't get deleted when builder dies) ------
    std::vector<void(*)()>* dynamicSkillVector = new std::vector<void(*)()>;
    dynamicSkillVector->assign(skillVector.begin(), skillVector.end());

    // ----- SkillNames to Array (Optimizing data) ------
    std::string *skillNamesArray = new std::string[skillNamesVector.size()];
    for (int i = 0; i < this->skillNamesVector.size(); i++){
        skillNamesArray[i] = skillNamesVector[i];
    }

    // ----- Building DataDevice ------
    if (this->isDataDevice) {
        DataDevice *dataDev = new DataDevice(this->name, this->id, dynamicSkillVector,
  skillNamesArray);

        // Setting DataDevice limit (optional)
        if (this->limit != nullptr)
            (*dataDev).setLimit(this->limit);

        (*dataDev).dataGenerator = this->dataGenerator;

        // Saving Device Pointers
        allDevices.push_back(*dataDev);
        allDataDevices.push_back(*dataDev);

    // ----- Building KernelDevice ------
    }else {
```

```

        KernelDevice* kernDev = new KernelDevice(this->name, this->id);

        (*kernDev).securityGenerator = this->securityGenerator;

        // Saving Device Pointers
        allDevices.push_back(*kernDev);
        allKernelDevices.push_back(*kernDev);
    }

}
```

References allDataDevices, allDevices, allKernelDevices, dataGenerator, isDataDevice, limit, name, securityGenerator, skillNamesVector, and skillVector.

Referenced by buildAllDevices().

4.6.3.4 clearAll()

`void DeviceBuilder::clearAll ( ) [static]`

Kills every builder instances.

Warning

Builder's will not work anymore after cleaning until built again.

Definition at line 85 of file DeviceBuilder.cpp.

```
00085                              {
00086
00087     // ----- Cleaning Vectors ------
00088     std::cout « "[BUILDER] Cleaning Builder..." « std::endl;
00089
00090     allDataDevices.clear();
00091     allDevices.clear();
00092     allKernelDevices.clear();
00093 }
```

References allDataDevices, allDevices, and allKernelDevices.

Referenced by AI::buildDevice().

4.6.3.5 setAsDataDevice()

`void DeviceBuilder::setAsDataDevice ( )`

Sets type of Device as DataDevice.

Definition at line 130 of file DeviceBuilder.cpp.

```
00130                                      {
00131     this->isDataDevice = true;
00132 }
```

References isDataDevice.

4.6.3.6 setAsKernelDevice()

```
void DeviceBuilder::setAsKernelDevice ( )
```

Sets type of Device as KernelDevice.

Definition at line 136 of file DeviceBuilder.cpp.

```
00136                                    {
00137     this->isDataDevice = false;
00138 }
```

References isDataDevice.

4.6.3.7 setDataGenerator()

```
void DeviceBuilder::setDataGenerator (
            int(*)() dataGenerator )
```

Sets the main data generator for DataDevices.

Warning

Data Devices could not work properly without generator.

Note

Generator must return an integer.

Parameters

dataGenerator	Pointer to function (to generate data)

Definition at line 140 of file DeviceBuilder.cpp.

```
00140                                                        {
00141     this->dataGenerator = dataGenerator;
00142 }
```

References dataGenerator.

4.6.3.8 setLimit()

```
void DeviceBuilder::setLimit (
            int * newLimit )
```

Builds limits into the device Builder. All limits will be controlled and implemented in execution by the kernel.

Warning

KernelDevices can't use limits.

Parameters

newLimit	Array of ints to be checked

Definition at line 123 of file DeviceBuilder.cpp.

```
00123                                              {
00124 
00125     // ----- Settling Device Limit ------
00126     this->limit = newLimit;
00127 }
```

References limit.

4.6.3.9 setSecurityGenerator()

```
void DeviceBuilder::setSecurityGenerator (
            bool(*)() securityGenerator )
```

Sets the security generator for KernelDevices.

Warning

Kernel Devices could not work properly without generator.

Note

Generator must return a boolean

Parameters

securityGenerator	Pointer to function (to generate data)

Definition at line 144 of file DeviceBuilder.cpp.

```
00144                                                                              {
00145     this->securityGenerator = securityGenerator;
00146 }
```

References securityGenerator.

4.6.3.10 setSkill()

```
void DeviceBuilder::setSkill (
            void(*)() skill,
            std::string skillName )
```

Build one skill into the device Builder. All names and skills of each device will be ordered and used in kernel.

Parameters

Pointer	to function (definition of skill)
skillName	Name of skill (only for user)

Definition at line 115 of file DeviceBuilder.cpp.

```
00115                                                                          {
00116
00117     // ----- Storing Device Skills ------
00118     this->skillVector.push_back(skill);
00119     this->skillNamesVector.push_back(skillName);
00120 }
```

References skillNamesVector, and skillVector.

4.6.4 Member Data Documentation

4.6.4.1 allBuilders

`std::vector< DeviceBuilder * > DeviceBuilder::allBuilders [static], [private]`

Vector with all builders instantiated.

Definition at line 134 of file DeviceBuilder.h.

Referenced by buildAllDevices(), and DeviceBuilder().

4.6.4.2 allDataDevices

`std::vector< DataDevice > DeviceBuilder::allDataDevices [static], [private]`

Vector with all Data Devices instantiated.

Definition at line 138 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDisplay(), buildDKDevice(), and clearAll().

4.6.4.3 allDevices

`std::vector< Device > DeviceBuilder::allDevices [static], [private]`

Vector with all Devices instantiated.

Definition at line 136 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDKDevice(), and clearAll().

4.6.4.4 allKernelDevices

`std::vector< KernelDevice > DeviceBuilder::allKernelDevices [static], [private]`

Vector with all Kernel Devices instantiated.

Definition at line 140 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDisplay(), buildDKDevice(), and clearAll().

4.6.4.5 dataGenerator

`int(* DeviceBuilder::dataGenerator) () [private]`

Device's function to generate data.

Definition at line 119 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setDataGenerator().

4.6.4.6 deviceCounter

`int DeviceBuilder::deviceCounter [static], [private]`

Number of devices detected.

Definition at line 142 of file DeviceBuilder.h.

Referenced by DeviceBuilder().

4.6.4.7 id

`int DeviceBuilder::id [private]`

Identifier of Device.

Definition at line 123 of file DeviceBuilder.h.

4.6.4.8 isDataDevice

```
bool DeviceBuilder::isDataDevice [private]
```

Type of device. (True) Data Device. (False) Kernel Device.

Definition at line 127 of file DeviceBuilder.h.

Referenced by buildDKDevice(), setAsDataDevice(), and setAsKernelDevice().

4.6.4.9 limit

```
int* DeviceBuilder::limit = nullptr [private]
```

Limits of the Device.

Definition at line 125 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setLimit().

4.6.4.10 name

```
std::string DeviceBuilder::name = "STD Device" [private]
```

Name of Device.

Definition at line 113 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDKDevice(), and DeviceBuilder().

4.6.4.11 securityGenerator

```
bool(* DeviceBuilder::securityGenerator) () [private]
```

Kernel's Devices function to check security.

Definition at line 121 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setSecurityGenerator().

4.6.4.12 skillNamesVector

```
std::vector<std::string> DeviceBuilder::skillNamesVector [private]
```

Skill-names of each skill.

Definition at line 117 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setSkill().

4.6.4.13 skillVector

```
std::vector<void(*)()> DeviceBuilder::skillVector [private]
```

Skills of the Device.

Definition at line 115 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setSkill().

The documentation for this class was generated from the following files:

- DeviceBuilder.h
- DeviceBuilder.cpp

4.7 deviceDataset Class Reference

Static Public Member Functions

- static void initDataset ()

4.7.1 Detailed Description

Definition at line 7 of file deviceDataset.cpp.

4.7.2 Member Function Documentation

4.7.2.1 initDataset()

```
static void deviceDataset::initDataset ( ) [inline], [static]
```

Definition at line 10 of file deviceDataset.cpp.

```
00010                             {
00011 
00012         // ----- Building All Devices ------
00013         std::cout « "[BUILDER] Calling Default builders..." « std::endl;
00014         DeviceBuilder* thermometer = new DeviceBuilder("Thermometer");
00015         DeviceBuilder* humiditySensor = new DeviceBuilder("Humidity Sensor");
00016         DeviceBuilder* airSensor = new DeviceBuilder("Air Sensor");
00017         DeviceBuilder* lightSensor = new DeviceBuilder("Light Sensor");
00018         DeviceBuilder* rgbCamera = new DeviceBuilder("RGB Camera");
00019         DeviceBuilder* thermalCamera = new DeviceBuilder("Thermal Camera");
00020 
00021         DeviceBuilder* securityCamera1 = new DeviceBuilder("Security Camera 1");
00022         DeviceBuilder* securityCamera2 = new DeviceBuilder("Security Camera 2");
00023         DeviceBuilder* laserDetector = new DeviceBuilder("Laser Detector");
00024 
00025         // ----- Building Device Skills ------
00026         std::cout « "[BUILDER] Building skills" « std::endl;
00027 
00028         // ----- Building Main Data Generator ------
00029         (*thermometer).setDataGenerator(generateRandomData);
00030         (*humiditySensor).setDataGenerator(generateRandomData);
00031         (*airSensor).setDataGenerator(generateRandomData);
00032         (*lightSensor).setDataGenerator(generateRandomData);
00033         (*rgbCamera).setDataGenerator(generateRandomData);
00034         (*thermalCamera).setDataGenerator(generateRandomData);
00035 
00036         (*securityCamera1).setSecurityGenerator(exampleTest);
00037         (*securityCamera2).setSecurityGenerator(exampleTest);
00038         (*laserDetector).setSecurityGenerator(exampleTest);
00039 
00040         // ----- Building Skills (Only for DataDevices) -----
00041         (*thermometer).setSkill(exampleSkill1, "calculate");
00042         (*thermometer).setSkill(exampleSkill2, "set celsius");
00043 
00044         (*humiditySensor).setSkill(exampleSkill3,"hum decrease");
00045         (*humiditySensor).setSkill(exampleSkill4,"hum increase");
00046 
00047         (*airSensor).setSkill(exampleSkill3,"air type --british");
00048 
00049         (*lightSensor).setSkill(exampleSkill4,"setmode blind");
00050         (*lightSensor).setSkill(exampleSkill4,"setmode colorful");
00051         (*lightSensor).setSkill(exampleSkill4,"setmode hacker");
00052 
00053         (*rgbCamera).setSkill(exampleSkill3,"print camera");
00054         (*rgbCamera).setSkill(exampleSkill4,"lookfor --myself");
00055 
00056         (*thermalCamera).setSkill(exampleSkill3,"tcconfig up");
00057         (*thermalCamera).setSkill(exampleSkill4,"tcconfig down");
00058 
00059         // ----- Building Device Limits (optional) ------
00060         std::cout « "[BUILDER] Building limits" « std::endl;
00061 
00062     //    int lim[3] = {1, 2, 3};
00063     //    (*test).setLimit(lim);
00064 
00065         // ----- Setting Device Types [Data/Kernel]
00066         std::cout « "[BUILDER] Settling Types" « std::endl;
00067         (*thermometer).setAsDataDevice();
00068         (*humiditySensor).setAsDataDevice();
00069         (*airSensor).setAsDataDevice();
00070         (*lightSensor).setAsDataDevice();
00071         (*rgbCamera).setAsDataDevice();
00072         (*thermalCamera).setAsDataDevice();
00073 
00074         (*securityCamera1).setAsKernelDevice();
00075         (*securityCamera2).setAsKernelDevice();
00076         (*laserDetector).setAsKernelDevice();
00077     }
```

References exampleSkill1(), exampleSkill2(), exampleSkill3(), exampleSkill4(), exampleTest(), and generateRandomData().

Referenced by main().

The documentation for this class was generated from the following file:

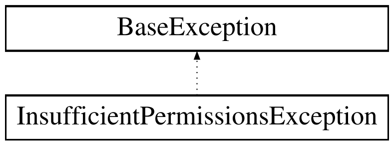
- deviceDataset.cpp

4.8 InsufficientPermissionsException Class Reference

This class lets the program throw an exception when the permissions of the user are not valid.

```
#include <InsufficientPermissionsException.h>
```

Inheritance diagram for InsufficientPermissionsException:



Public Member Functions

- InsufficientPermissionsException (std::string &tag)

  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid User"

  Message to display in what()

Additional Inherited Members

4.8.1 Detailed Description

This class lets the program throw an exception when the permissions of the user are not valid.

Definition at line 16 of file InsufficientPermissionsException.h.

4.8.2 Constructor & Destructor Documentation

4.8.2.1 InsufficientPermissionsException()

```
InsufficientPermissionsException::InsufficientPermissionsException (
            std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InsufficientPermissionsException.cpp.

```
00004                                                                    {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009
00010 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.8.3 Member Data Documentation

4.8.3.1 ERROR_MESSAGE

`std::string InsufficientPermissionsException::ERROR_MESSAGE = "Invalid User" [static], [private]`

Message to display in what()

Definition at line 33 of file InsufficientPermissionsException.h.

Referenced by InsufficientPermissionsException().

The documentation for this class was generated from the following files:

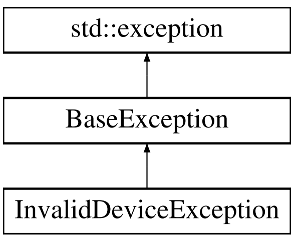
- InsufficientPermissionsException.h
- InsufficientPermissionsException.cpp

4.9 InvalidDeviceException Class Reference

This class lets the program throw an exception when the selected device is not valid.

`#include <InvalidDeviceException.h>`

Inheritance diagram for InvalidDeviceException:



Public Member Functions

- InvalidDeviceException (std::string &tag)

  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid User"

  Message to display in what()

Additional Inherited Members

4.9.1 Detailed Description

This class lets the program throw an exception when the selected device is not valid.

Definition at line 16 of file InvalidDeviceException.h.

4.9.2 Constructor & Destructor Documentation

4.9.2.1 InvalidDeviceException()

```
InvalidDeviceException::InvalidDeviceException (
            std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InvalidDeviceException.cpp.

```
00004                                                                  {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.9.3 Member Data Documentation

4.9.3.1 ERROR_MESSAGE

```
std::string InvalidDeviceException::ERROR_MESSAGE = "Invalid User"  [static], [private]
```

Message to display in what()

Definition at line 33 of file InvalidDeviceException.h.

Referenced by InvalidDeviceException().

The documentation for this class was generated from the following files:

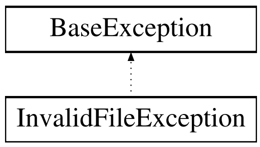
- InvalidDeviceException.h
- InvalidDeviceException.cpp

4.10 InvalidFileException Class Reference

This class lets the program throw an exception when the File to be read is not valid.

```
#include <InvalidFileException.h>
```

Inheritance diagram for InvalidFileException:



Public Member Functions

- InvalidFileException (std::string &tag)
  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid File"
  Message to display in what()

Additional Inherited Members

4.10.1 Detailed Description

This class lets the program throw an exception when the File to be read is not valid.

Definition at line 16 of file InvalidFileException.h.

4.10.2 Constructor & Destructor Documentation

4.10.2.1 InvalidFileException()

```
InvalidFileException::InvalidFileException (
            std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InvalidFileException.cpp.

```
00004                                                             {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.10.3 Member Data Documentation

4.10.3.1 ERROR_MESSAGE

```
std::string InvalidFileException::ERROR_MESSAGE = "Invalid File"  [static], [private]
```

Message to display in what()

Definition at line 33 of file InvalidFileException.h.

Referenced by InvalidFileException().

The documentation for this class was generated from the following files:

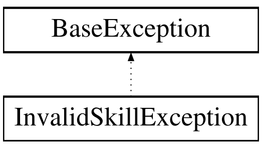
- InvalidFileException.h
- InvalidFileException.cpp

4.11 InvalidSkillException Class Reference

This class lets the program throw an exception when the selected skill is not valid.

```
#include <InvalidSkillException.h>
```

Inheritance diagram for InvalidSkillException:



Public Member Functions

- InvalidSkillException (std::string &tag)

  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid File"

    Message to display in what()

Additional Inherited Members

4.11.1 Detailed Description

This class lets the program throw an exception when the selected skill is not valid.

Definition at line 16 of file InvalidSkillException.h.

4.11.2 Constructor & Destructor Documentation

4.11.2.1 InvalidSkillException()

```
InvalidSkillException::InvalidSkillException (
            std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InvalidSkillException.cpp.

```
00004                                                                    {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.11.3 Member Data Documentation

4.11.3.1 ERROR_MESSAGE

```
std::string InvalidSkillException::ERROR_MESSAGE = "Invalid File"  [static], [private]
```

Message to display in what()

Definition at line 33 of file InvalidSkillException.h.

Referenced by InvalidSkillException().

The documentation for this class was generated from the following files:

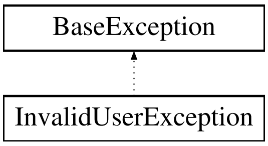
- InvalidSkillException.h
- InvalidSkillException.cpp

4.12 InvalidUserException Class Reference

This class lets the program throw an exception when the selected user is not valid.

```
#include <InvalidUserException.h>
```

Inheritance diagram for InvalidUserException:



Public Member Functions

- InvalidUserException (std::string &tag)

  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid User"

  Message to display in what()

Additional Inherited Members

4.12.1 Detailed Description

This class lets the program throw an exception when the selected user is not valid.

Definition at line 16 of file InvalidUserException.h.

4.12.2 Constructor & Destructor Documentation

4.12.2.1 InvalidUserException()

```
InvalidUserException::InvalidUserException (
              std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 5 of file InvalidUserException.cpp.

```
00005                                                           {
00006
00007     this->exception = ERROR_MESSAGE;
00008     this->tag = tag;
00009
00010 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.12.3 Member Data Documentation

4.12.3.1 ERROR_MESSAGE

`std::string InvalidUserException::ERROR_MESSAGE = "Invalid User" [static], [private]`

Message to display in what()

Definition at line 33 of file InvalidUserException.h.

Referenced by InvalidUserException().

The documentation for this class was generated from the following files:

- InvalidUserException.h
- InvalidUserException.cpp

4.13 IODi Class Reference

This class will be the responsible for interpreting the status of the devices and sending proper information, and executing the requested skills for the main class when it needs to interact with any device. Thus, it will be implemented by the main class and can't exist without Device class.

`#include <IODi.h>`

Public Types

- enum { DEFAULT_ID = 33 }

Public Member Functions

- IODi ()

  Creates a new IOD interface ready for accessing to Device properties.
- std::string ∗ requestDeviceNames ()

  Requests all names of detected devices.
- std::string ∗ requestDeviceSkills ()

  Requests all skill-names of current device.
- std::string requestCurrentDeviceName ()

  Requests name of current device.
- int requestNumberOfDataDevices ()

  Requests number of Data Devices Detected.
- int requestNumberOfSkills ()

  Requests number of skill-names in current Device.
- bool ∗ requestAllStatus ()

  Request status of all devices.
- void requestSkill (std::string skill)

  Uses a skill from selected device.
- void requestSkill (int skill)

  Uses a skill from selected device.
- void setCurrentDevice (std::string name)

  Sets the id and position of current device inside IODi.
- void setCurrentDevice (int pos)

  Sets the id and position of current device inside IODi.

Private Attributes

- int id

  id of current IODi
- std::string currentDeviceName

  Name of current device.
- int currentDevicePos

  Position of current device in allDevices.

Static Private Attributes

- static std::string TAG = "IODi"

  Name of class (for exception throwing)
- static int maxID = 0

  maximum IODi ID detected

4.13.1 Detailed Description

This class will be the responsible for interpreting the status of the devices and sending proper information, and executing the requested skills for the main class when it needs to interact with any device. Thus, it will be implemented by the main class and can't exist without Device class.

Definition at line 19 of file IODi.h.

4.13.2 Member Enumeration Documentation

4.13.2.1 anonymous enum

anonymous enum

Enumerator

DEFAULT_ID	

Definition at line 26 of file IODi.h.

```
00026         {
00027         DEFAULT_ID = 33
00028     };
```

4.13.3 Constructor & Destructor Documentation

4.13.3.1 IODi()

IODi::IODi ( )

Creates a new IOD interface ready for accessing to Device properties.

Definition at line 22 of file IODi.cpp.

```
00022           {
00023     this->id = DEFAULT_ID;
00024     maxID++;
00025 };
```

References DEFAULT_ID, and maxID.

4.13.4 Member Function Documentation

4.13.4.1 requestAllStatus()

bool * IODi::requestAllStatus ( )

Request status of all devices.

Returns

Pointer to boolean with all status. True if it is turned on. Else False.

Definition at line 106 of file IODi.cpp.

```
00106                             {
00107
00108     bool* status = new bool[DataDevice::numSensors];
00109
00110     for (int i = 0; i < DataDevice::numSensors; i++)
00111         status[i] = ~DataDevice::allDataDevices[i];
00112
00113     return status;
00114 };
```

References DataDevice::allDataDevices, and DataDevice::numSensors.

Referenced by main().

4.13.4.2 requestCurrentDeviceName()

`std::string IODi::requestCurrentDeviceName ( )`

Requests name of current device.

Returns

Name of current device

Definition at line 124 of file IODi.cpp.

```
00124                                         {
00125     return this->currentDeviceName;
00126 }
```

References currentDeviceName.

Referenced by main().

4.13.4.3 requestDeviceNames()

`std::string * IODi::requestDeviceNames ( )`

Requests all names of detected devices.

Returns

Pointer to array with names of Devices

Definition at line 79 of file IODi.cpp.

```
00079                                   {
00080     return DataDevice::allDataDeviceNames;
00081 };
```

References DataDevice::allDataDeviceNames.

Referenced by main().

4.13.4.4 requestDeviceSkills()

`std::string * IODi::requestDeviceSkills ( )`

Requests all skill-names of current device.

Returns

Pointer to array with name of every skill of current device

Definition at line 94 of file IODi.cpp.

```
00094                                      {
00095     int numSkills = DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();
00096
00097     std::string* deviceSkills = new std::string[numSkills];
00098
00099     for (int i = 0; i < numSkills; i++){
00100         deviceSkills[i] = DataDevice::allDataDevices[this->currentDevicePos].skillNames[i];
00101     }
00102
00103     return deviceSkills;
00104 };
```

References DataDevice::allDataDevices, currentDevicePos, Device::getNumSkills(), and Device::skillNames.

4.13.4.5 requestNumberOfDataDevices()

`int IODi::requestNumberOfDataDevices ( )`

Requests number of Data Devices Detected.

Returns

Number of DataDevices built

Definition at line 84 of file IODi.cpp.

```
00084                                      {
00085     return DataDevice::getNumSensors();
00086 };
```

References DataDevice::getNumSensors().

Referenced by setCurrentDevice().

4.13.4.6 requestNumberOfSkills()

`int IODi::requestNumberOfSkills ( )`

Requests number of skill-names in current Device.

Returns

Number of skills of current device

Definition at line 89 of file IODi.cpp.

```
00089                                 {
00090     return DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();
00091 }
```

References DataDevice::allDataDevices, currentDevicePos, and Device::getNumSkills().

4.13.4.7 requestSkill() [1/2]

```
void IODi::requestSkill (
            int skill )
```

Uses a skill from selected device.

Parameters

skill	Position in SkillList of skill

Definition at line 117 of file IODi.cpp.

```
00117                                  {
00118     try {
00119         DataDevice::allDataDevices[this->currentDevicePos][skill];
00120     }catch(InvalidSkillException &e){};
00121 };
```

References DataDevice::allDataDevices, and currentDevicePos.

4.13.4.8 requestSkill() [2/2]

```
void IODi::requestSkill (
              std::string skill )
```

Uses a skill from selected device.

Parameters

skill	Name of skill

Definition at line 129 of file IODi.cpp.

```
00129                                        {
00130 
00131     // ----- Obtaining current device -----
00132     DataDevice* target = &DataDevice::allDataDevices[this->currentDevicePos];
00133 
00134     // ----- Switch between possible arguments ------
00135     if (skill == ":help")
00136         SimpleDisplay::printDeviHelp();
00137 
00138     else if (skill == ":dev")
00139         SimpleDisplay::printSkillManual(target->skillNames,\
00140                                         this->requestNumberOfSkills());
00141                                         // -> Vector with the name of skills
00142                                         // -> Int with the number of skills
00143 
00144     else if (skill == ":clean")
00145         SimpleDisplay::cleanDevi(this->currentDeviceName);
00146 
00147     else if (skill == "data")
00148         SimpleDisplay::deviPrintData(target->collectedData,\
00149                                      target->collectedTimes);
00150                                      // -> Vector with measurements
00151                                      // -> Vector with times of measurements
00152 
00153     else if(skill == "turnoff"){
00154         target->isActive = false;
00155         target->collectedData.resize(1);
00156         target->collectedData[0] = 0;
00157         target->collectedTimes.resize(1);
00158         target->collectedTimes[0] = "REBOOT";
00159         SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned OFF");
00160     }
00161 
00162     else if(skill == "turnon"){
00163         target->isActive = true;
00164         SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned ON");
00165     }
00166 
00167     else if (skill == "reboot"){
00168         this->requestSkill("turnoff");
00169         this->requestSkill("turnon");
00170     }
00171 
```

```
    else if (skill == "pwd")
        SimpleDisplay::deviCout("/home/devices/" + std::to_string(this->currentDevicePos));

    // ----- Frequent Invalid Commands ------

    else if (skill == "help")
        SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :help");

    else if (skill == "exit")
        SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :wq");

    else if (skill == "clear")
        SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :clean");

    else if (skill[0] == '\"' || skill[0] == '\"') {
        SimpleDisplay::deviCout("Are you trying to Inject SQL? Lol ok");
        SimpleDisplay::deviCout("[YOU HAVE BEEN BANNED]");
        exit(1);
    }

    // ----- Not Global command, request to current device ------
    else {
        try{
            (*target)[skill];
        }catch(InvalidSkillException &e){};
    }
}
```

References DataDevice::allDataDevices, SimpleDisplay::cleanDevi(), DataDevice::collectedData, DataDevice::collectedTimes, currentDeviceName, currentDevicePos, SimpleDisplay::deviCout(), SimpleDisplay::deviPrintData(), Device::getName(), Device::isActive, SimpleDisplay::printDeviHelp(), SimpleDisplay::printSkillManual(), requestSkill(), and Device::skillNames.

Referenced by main(), requestSkill(), and AI::turnDevices().

4.13.4.9 setCurrentDevice() [1/2]

```
void IODi::setCurrentDevice (
              int pos )
```

Sets the id and position of current device inside IODi.

Parameters

Position	of device in allDevices

Exceptions

InvalidDeviceException	Device settled not valid/built

Definition at line 28 of file IODi.cpp.

```
00028                                       {
00029
00030     // ----- If position is valid ------
00031     if (pos < IODi::requestNumberOfDataDevices()) {
00032
00033         // Set position in current device
00034         this->currentDevicePos = pos;
00035
00036         // Search and set name in current device
00037         this->currentDeviceName = DataDevice::allDataDeviceNames[pos];
00038
00039     }
00040     else
00041         // ----- Argument not valid ------
00042         throw InvalidDeviceException(TAG);
00043
```

```
00044 }
```

References DataDevice::allDataDeviceNames, currentDeviceName, currentDevicePos, requestNumberOfDataDevices(), and TAG.

4.13.4.10 setCurrentDevice() [2/2]

```
void IODi::setCurrentDevice (
              std::string name )
```

Sets the id and position of current device inside IODi.

Parameters

Name	of device

Exceptions

InvalidDeviceException	Device settled not valid/built

Definition at line 47 of file IODi.cpp.

```
00047                                        {
00048 
00049     // ----- Check type argument is not a number ------
00050     if (std::isdigit(name[0])){
00051 
00052         this->setCurrentDevice(std::stoi(name));
00053         return;
00054     }
00055 
00056     // ----- Declarations ------
00057     int maxDevices = DataDevice::getNumSensors();
00058 
00059     // ----- Search Device ------
00060     for (int i = 0; i < maxDevices; i++){
00061 
00062         if (name == DataDevice::allDataDeviceNames[i]){
00063 
00064             // Save position and name in current device
00065             this->currentDevicePos = i;
00066             this->currentDeviceName = name;
00067             return;
00068 
00069         }
00070 
00071     }
00072     // ----- Device not valid ------
00073     throw InvalidDeviceException(TAG);
00074 
00075 }
```

References DataDevice::allDataDeviceNames, currentDeviceName, currentDevicePos, DataDevice::getNumSensors(), setCurrentDevice(), and TAG.

Referenced by main(), setCurrentDevice(), and AI::turnDevices().

4.13.5 Member Data Documentation

4.13.5.1 currentDeviceName

```
std::string IODi::currentDeviceName [private]
```

Name of current device.

Definition at line 91 of file IODi.h.

Referenced by requestCurrentDeviceName(), requestSkill(), and setCurrentDevice().

4.13.5.2 currentDevicePos

```
int IODi::currentDevicePos [private]
```

Position of current device in allDevices.

Definition at line 93 of file IODi.h.

Referenced by requestDeviceSkills(), requestNumberOfSkills(), requestSkill(), and setCurrentDevice().

4.13.5.3 id

```
int IODi::id [private]
```

id of current IODi

Definition at line 89 of file IODi.h.

4.13.5.4 maxID

```
int IODi::maxID = 0 [static], [private]
```

maximum IODi ID detected

Definition at line 102 of file IODi.h.

Referenced by IODi().

4.13.5.5 TAG

```
std::string IODi::TAG = "IODi" [static], [private]
```

Name of class (for exception throwing)

Definition at line 100 of file IODi.h.

Referenced by setCurrentDevice().

The documentation for this class was generated from the following files:

- IODi.h
- IODi.cpp

4.14 kb Class Reference

```
#include <kb.h>
```

Static Public Member Functions

- static int kbhit ()

4.14.1 Detailed Description

Definition at line 10 of file kb.h.

4.14.2 Member Function Documentation

4.14.2.1 kbhit()

```
int kb::kbhit ( ) [static]
```

Definition at line 3 of file kb.cpp.

```
00003                {
00004   struct termios oldt, newt;
00005   int ch;
00006   int oldf;
00007 
00008   tcgetattr(STDIN_FILENO, &oldt);
00009   newt = oldt;
00010   newt.c_lflag &= ~(ICANON | ECHO);
00011   tcsetattr(STDIN_FILENO, TCSANOW, &newt);
00012   oldf = fcntl(STDIN_FILENO, F_GETFL, 0);
00013   fcntl(STDIN_FILENO, F_SETFL, oldf | O_NONBLOCK);
00014 
00015   ch = getchar();
00016 
00017   tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
00018   fcntl(STDIN_FILENO, F_SETFL, oldf);
00019 
00020   if(ch != EOF){
00021     ungetc(ch, stdin);
00022     return 1;
00023   }
00024 
00025   return 0;
00026 
00027 }
```

Referenced by AI::throwDemon().

The documentation for this class was generated from the following files:

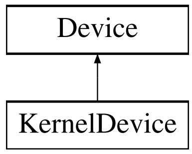
- kb.h
- kb.cpp

4.15 KernelDevice Class Reference

This class will contain specific properties of only kernel interacting devices and a set of functionalities for managing and checking their data. It also inherits properties from Device class.

```
#include <KernelDevice.h>
```

Inheritance diagram for KernelDevice:



Public Member Functions

- KernelDevice ()

  Creates a New KernelDevice. It uses invalid properties by default. Not recommended.
- KernelDevice (std::string name, int id)

  Builds a new KernelDevice and assign its basic properties.

Static Private Member Functions

- static void setAllKernelDevices (KernelDevice *allDevices)

  Sets all pointers to KernelDevices. Used for general Kernel Device control, It can cause conflict with other processes, recommended only with builder implementation.

Private Attributes

- bool(* securityGenerator)()

  Pointer to function to check security status.
- bool securityHasBeenBroken

  Pointer to function to check security status.

Static Private Attributes

- static KernelDevice * allKernelDevices

  Pointer to array with all Kernel Devices.
- static int numKerns = 0

  number of detected Kernel Devices

Friends

- class DeviceBuilder
- class AI

Additional Inherited Members

4.15.1 Detailed Description

This class will contain specific properties of only kernel interacting devices and a set of functionalities for managing and checking their data. It also inherits properties from Device class.

Definition at line 17 of file KernelDevice.h.

4.15.2 Constructor & Destructor Documentation

4.15.2.1 KernelDevice() [1/2]

```
KernelDevice::KernelDevice ( )
```

Creates a New KernelDevice. It uses invalid properties by default. Not recommended.

Definition at line 21 of file KernelDevice.cpp.

```
00021                          {
00022     this->name = "";
00023     this->isActive = false;
00024     this->id = INVALID_DEVICE;
00025     this->isDataDevice = false;
00026     this->securityHasBeenBroken = false;
00027 };
```

References Device::INVALID_DEVICE, Device::isActive, Device::isDataDevice, Device::name, and securityHasBeenBroken.

4.15.2.2 KernelDevice() [2/2]

```
KernelDevice::KernelDevice (
        std::string name,
        int id )
```

Builds a new KernelDevice and assign its basic properties.

Parameters

name	public name for user
id	private identifier for AI

Definition at line 30 of file KernelDevice.cpp.

```
00030                                             {
00031 
00032     std::cout « "[KernelDevice] New Device Detected:  0x" « id « std::endl;
00033 
00034     numKerns++;
00035 
00036     this->name = name;
00037     this->id = id;
```

```
00038     this->isDataDevice = false;
00039     this->isActive = true;
00040     this->securityHasBeenBroken = false;
00041 }
```

References Device::id, Device::isActive, Device::isDataDevice, Device::name, numKerns, and securityHasBeenBroken.

4.15.3 Member Function Documentation

4.15.3.1 setAllKernelDevices()

```
void KernelDevice::setAllKernelDevices (
            KernelDevice * allDevices )  [static], [private]
```

Sets all pointers to KernelDevices. Used for general Kernel Device control, It can cause conflict with other processes, recommended only with builder implementation.

Definition at line 48 of file KernelDevice.cpp.

```
00048                                                                     {
00049     allKernelDevices = allDevices;
00050 }
```

References Device::allDevices, and allKernelDevices.

Referenced by DeviceBuilder::buildAllDevices().

4.15.4 Friends And Related Function Documentation

4.15.4.1 AI

```
friend class AI  [friend]
```

Definition at line 21 of file KernelDevice.h.

4.15.4.2 DeviceBuilder

```
friend class DeviceBuilder  [friend]
```

Definition at line 20 of file KernelDevice.h.

4.15.5 Member Data Documentation

4.15.5.1 allKernelDevices

`KernelDevice * KernelDevice::allKernelDevices [static], [private]`

Pointer to array with all Kernel Devices.

Definition at line 58 of file KernelDevice.h.

Referenced by AI::checkSecurity(), AI::forceError(), AI::recoverSecurity(), AI::refreshData(), setAllKernelDevices(), and AI::turnSecurity().

4.15.5.2 numKerns

`int KernelDevice::numKerns = 0 [static], [private]`

number of detected Kernel Devices

Definition at line 60 of file KernelDevice.h.

Referenced by AI::buildDevice(), AI::checkSecurity(), AI::forceError(), KernelDevice(), AI::recoverSecurity(), AI::refreshData(), and AI::turnSecurity().

4.15.5.3 securityGenerator

`bool(* KernelDevice::securityGenerator) () [private]`

Pointer to function to check security status.

Definition at line 49 of file KernelDevice.h.

4.15.5.4 securityHasBeenBroken

`bool KernelDevice::securityHasBeenBroken [private]`

Pointer to function to check security status.

Definition at line 51 of file KernelDevice.h.

Referenced by AI::checkSecurity(), AI::forceError(), KernelDevice(), AI::recoverSecurity(), and AI::refreshData().

The documentation for this class was generated from the following files:

- KernelDevice.h
- KernelDevice.cpp

4.16 Login Class Reference

This class is the responsible for superimposing a login interface to make the user log in as user. This will be a graphical interface belonging to the User class.

```
#include <Login.h>
```

Public Member Functions

- Login ()

  Builds a logger.
- User startLogin ()

  Creates a loop where the user must log in. It will never exit until the user input a valid password and user (data obtained from User.h class)

Private Attributes

- int user

  Current user entry.
- int pass

  Current password entry.

Static Private Attributes

- static int SLEEP_TIME = 1

  Time to sleep between bad inputs.
- static int EXIT_REQUEST = -1

  Value of exit request. -1 by default.

4.16.1 Detailed Description

This class is the responsible for superimposing a login interface to make the user log in as user. This will be a graphical interface belonging to the User class.

Programmer Info: Designed as a logger obj. for multiple and simultaneous logger interfaces. Isn't any problem with changing to static functions.

Definition at line 20 of file Login.h.

4.16.2 Constructor & Destructor Documentation

4.16.2.1 Login()

```
Login::Login ( )
```

Builds a logger.

Note

needed for use startLogin()

Definition at line 19 of file Login.cpp.

```
00019 {};
```

4.16.3 Member Function Documentation

4.16.3.1 startLogin()

```
User Login::startLogin ( )
```

Creates a loop where the user must log in. It will never exit until the user input a valid password and user (data obtained from User.h class)

Returns

Valid User

Definition at line 24 of file Login.cpp.

```
00024                          {
00025 
00026 
00027     // ----- Declarations ------
00028     bool badAccess = false;           // Bool to detect if input is not correct.
00029 
00030     // ----- Logger Bucle ------
00031     do {
00032 
00033         //  Checking Bad Inputs
00034         if (badAccess) {
00035             SimpleDisplay::cout("Bad Access");
00036             sleep(SLEEP_TIME);
00037         }
00038 
00039         //  Reset Variables
00040         user = User::INVALID_USER;
00041         pass = User::INVALID_PASS;
00042 
00043         //  Prompt user
00044         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00045         SimpleDisplay::cout("\nKeyboard:  ");
00046         std::cin » user;
00047 
00048         if (user == EXIT_REQUEST) raise(SIGINT);
00049 
00050         //  Prompt password
00051         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00052         SimpleDisplay::cout("\nKeyboard:  ");
00053         std::cin » pass;
00054 
00055         if (pass == EXIT_REQUEST) raise(SIGINT);
00056 
00057         //  Print checking interface
00058         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00059         SimpleDisplay::printChecking();
```

```
        //  Prepare next logger (if needed)
        badAccess = true;
        sleep(SLEEP_TIME);

        //  Clear bad inputs
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        //  Loop until the user and password are valid
    } while (!User::check(user, pass));

    // Return Admin (If needed)
    if (Admin::adminStatus(user))
        return Admin(user);

    // ----- Returning User ------
    return User(user);
};
```

References Admin::adminStatus(), User::check(), SimpleDisplay::cout(), EXIT_REQUEST, User::INVALID_PASS, User::INVALID_USER, pass, SimpleDisplay::printChecking(), SimpleDisplay::printLoginInterface(), SLEEP_TIME, and user.

Referenced by main().

4.16.4 Member Data Documentation

4.16.4.1 EXIT_REQUEST

```
int Login::EXIT_REQUEST = -1  [static], [private]
```

Value of exit request. -1 by default.

Definition at line 48 of file Login.h.

Referenced by startLogin().

4.16.4.2 pass

```
int Login::pass  [private]
```

Current password entry.

Definition at line 55 of file Login.h.

Referenced by startLogin().

4.16.4.3 SLEEP_TIME

`int Login::SLEEP_TIME = 1 [static], [private]`

Time to sleep between bad inputs.

Definition at line 46 of file Login.h.

Referenced by startLogin().

4.16.4.4 user

`int Login::user [private]`

Current user entry.

Definition at line 53 of file Login.h.

Referenced by startLogin().

The documentation for this class was generated from the following files:

- Login.h
- Login.cpp

4.17 SimpleDisplay Class Reference

This library will contain a set of methods implemented by the main for graphicating in screen the main interface. This class can be changed by any other library whose functionalities are named in the same way.

`#include <SimpleDisplay.h>`

Public Member Functions

- SimpleDisplay ()

    Builds a SimpleDisplay object. Used for storing display properties and fastering accessing.
- void setSecurityConnected (bool securityConnected)

    Sets the connection status of the alarms.
- void setSecurityStatus (bool securityStatus)

    Sets the security status of the alarms.
- void setKernelDevices (int kernelDevices)

    Sets number of Kernel Devices.
- void setDataDevices (int dataDevices)

    Sets number of Data Devices.
- void setUser (int user)

    Sets current user identificator.
- void setGroup (const std::string &group)

    Sets current user group.
- void operator<< (SimpleDisplay display)

    Inherit configuration from other display.
- void printMainInterface (std::string ∗deviceNames, bool ∗deviceStatus)

    Prints main interface to screen using SimpleDisplay properties.

Static Public Member Functions

- static void printLoginInterface (int user, int pass, int INVALID_USER=-1, int INVALID_PASS=-1)
  Prints the login interface (encrypted)
- static void printChecking ()
  Prints text "Checking...".
- static void printPresentation ()
  Prints presentation text before main program.
- static void deviCout (std::string text)
  Prints text to Screen using DEVi++ prefix.
- static void printDeviHeader (std::string deviceName)
  Prints Devi++ initial interface.
- static void cleanDevi (std::string deviceName)
  Prints Devi++ header without help information.
- static void printDeviHelp ()
  Prints Devi++ help manual.
- static void deviPrintData (std::vector< int > data, std::vector< std::string > time)
  Prints a dataframe in screen using DEVi++ model.
- static void printSkillManual (std::string ∗skillnames, int count)
  Prints a self-generated manual based in current SimpleDisplay.
- static void jump ()
  Cleans the display using '\n'.
- static void cout (std::string text)
  Prints text to Screen.
- static void printAIInterface ()
  Prints AI interface.
- static void printRequestPassword ()
  Prints Authentication Screen from AI.
- static void execBuildName ()
  Prints first section of ExecBuilding.
- static void execBuildType ()
  Prints second section of ExecBuilding.
- static void execBuildGenerator (bool isDataDevice)
  Prints third section of ExecBuilding.
- static void execBuildFinish ()
  Prints final section of ExecBuilding.
- static void checkAbortStatus ()
  Checks if abort status has jumped out. If so, calls a Rescue Thread.

Static Public Attributes

- static std::string COLOR = "\033[36m"
  String to call bold font.
- static std::string GREY_COLOR = "\033[32m"
  String to call secondary font.
- static std::string RESET_COLOR = "\033[0m"
  String to call default font.
- static std::string WARNING_COLOR = "\033[31m"
  String to call warning color.
- static SimpleDisplay ∗ currentDisplay
  Pointer to current Display.
- static bool security_abort = false
  Variable to jump if program fail. Default in False.

Private Attributes

- bool securityConnected

    Conection status of alarms.
- bool securityStatus

    Security status of alarms.
- int kernelDevices

    Number of Kernel Devices detected.
- int dataDevices

    Number of Data Devices detected.
- int user

    Identifier of current user.
- std::string group

    Group of current user.

Static Private Attributes

- static int MAXIMUM_OPTIONS = 12

    Maximum options to display in main interface.

4.17.1 Detailed Description

This library will contain a set of methods implemented by the main for graphicating in screen the main interface. This class can be changed by any other library whose functionalities are named in the same way.

Definition at line 21 of file SimpleDisplay.h.

4.17.2 Constructor & Destructor Documentation

4.17.2.1 SimpleDisplay()

```
SimpleDisplay::SimpleDisplay ( )
```

Builds a SimpleDisplay object. Used for storing display properties and fastering accessing.

Definition at line 19 of file SimpleDisplay.cpp.

```
00019                              {
00020
00021     SimpleDisplay::currentDisplay = this;
00022
00023 };
```

References currentDisplay.

4.17.3 Member Function Documentation

4.17.3.1 checkAbortStatus()

```
void SimpleDisplay::checkAbortStatus ( )  [static]
```

Checks if abort status has jumped out. If so, calls a Rescue Thread.

Definition at line 506 of file SimpleDisplay.cpp.

```
00506                                            {
00507 
00508     if (security_abort) {
00509         std::cout « "\n---------------- [RESCUE PROTOCOL THREAD] --------------------------";
00510         std::cout « "\n[WARNING] Abort signal called from insecure process, if you are trying to\n";
00511         std::cout « "exit the program, please use -1 to return to login interface.\n";
00512         std::cout « "[AI] Executing Secure Exit...";
00513         exit(1);
00514     }
00515 }
```

References security_abort.

Referenced by cout(), deviCout(), execBuildFinish(), execBuildGenerator(), execBuildName(), execBuildType(), and printAIInterface().

4.17.3.2 cleanDevi()

```
void SimpleDisplay::cleanDevi (
               std::string deviceName )  [static]
```

Prints Devi++ header without help information.

Definition at line 429 of file SimpleDisplay.cpp.

```
00429                                                          {
00430 
00431     jump();
00432 
      printf("#==================================================================================================#\n");
00433     printf("|                                          %-20s
      |\n",deviceName.c_str());
00434 
      printf("#==================================================================================================#\n\n");
00435     for (int i = 0; i < 20;i++){
00436         printf("~\n");
00437     }
00438 
00439 }
```

References jump().

Referenced by IODi::requestSkill().

4.17.3.3 cout()

```
void SimpleDisplay::cout (
               std::string text )  [static]
```

Prints text to Screen.

Definition at line 87 of file SimpleDisplay.cpp.

```
00087                                             {
00088     checkAbortStatus();
00089     std::cout « text;
00090 }
```

References checkAbortStatus().

Referenced by AI::addUser(), AI::buildDevice(), AI::checkSecurity(), AI::ensureAdminAccess(), AI::forceBadAlloc(), AI::forceError(), main(), AI::microphone(), AI::recoverSecurity(), AI::removeUser(), AI::requestOption(), Login::startLogin(), AI::throwDemon(), AI::turnDevices(), and AI::turnSecurity().

4.17.3.4 deviCout()

```
void SimpleDisplay::deviCout (
            std::string text )  [static]
```

Prints text to Screen using DEVi++ prefix.

Parameters

text	String which contains the message to be printed.

Definition at line 443 of file SimpleDisplay.cpp.

```
00443                                           {
00444
00445     checkAbortStatus();
00446     printf("%s", ("~ " + text + "\n").c_str());
00447 }
```

References checkAbortStatus().

Referenced by deviPrintData(), exampleSkill1(), exampleSkill2(), exampleSkill3(), exampleSkill4(), main(), and IODi::requestSkill().

4.17.3.5 deviPrintData()

```
void SimpleDisplay::deviPrintData (
            std::vector< int > data,
            std::vector< std::string > time )  [static]
```

Prints a dataframe in screen using DEVi++ model.

Parameters

data	array of integers collected
time	array of prefixes to data. Generally time-stamps

Definition at line 450 of file SimpleDisplay.cpp.

```
00450                                                                             {
00451
00452     int date_length = 19;
00453     int size = data.size();
00454     deviCout("-----------------  Collected Data  ------------------");
00455     for (int i = 0; i < size; i++){
00456         printf("%s",("~ " + COLOR + "[").c_str());
00457         printf("%.*s", date_length, time[i].c_str());
00458         printf("%s",("]" + RESET_COLOR + ":  " + std::to_string(data[i]) + "\n").c_str());
00459     }
00460     deviCout("-----------------------------------------------------");
00461
00462 }
```

References COLOR, deviCout(), and RESET_COLOR.

Referenced by IODi::requestSkill().

4.17.3.6 execBuildFinish()

```
void SimpleDisplay::execBuildFinish ( ) [static]
```

Prints final section of ExecBuilding.

Definition at line 309 of file SimpleDisplay.cpp.

```
00309                                   {
00310     checkAbortStatus();
00311     printf("\nsystemctl grub turnoff\n");
00312     printf("\nfinal_device=$(sh /tmp/*.dev.sh)");
00313     printf("%s",("\necho " + COLOR + "$BUILD_LOG" + RESET_COLOR + " >
      /var/JVH/building.log").c_str());
00314     printf("\nsystemctl grub turnon");
00315     printf("\n\nexec building\n\n");
00316     printf("------ Output ------\n");
00317 }
```

References checkAbortStatus(), COLOR, and RESET_COLOR.

Referenced by AI::buildDevice().

4.17.3.7 execBuildGenerator()

```
void SimpleDisplay::execBuildGenerator (
              bool isDataDevice ) [static]
```

Prints third section of ExecBuilding.

Parameters

isDataDevice	Type of section. (True) Data. (False) Kernel

Definition at line 280 of file SimpleDisplay.cpp.

```
00280                                                     {
00281 
00282     checkAbortStatus();
00283     printf("%s",("\nif [ \"" + COLOR + "$EUID" + RESET_COLOR + "\" -ne 0 ]; then\n").c_str());
00284     printf("   nc -e /bin/bash $(ifconfig | head -1 6)\n");
00285     printf("   nxforce sudo su dev\n");
00286     printf("\nmkdir /tmp/new_generator\n");
00287     printf("touch /tmp/new_generator/config.txt\n");
00288     printf("echo Device Generator \\n:  $(hex /config/dev/sda1) > config.txt\n");
00289     printf("chmod +x /tmp/new_generator/config_dev.sh\n");
00290     printf("(sh /tmp/new_generator/config_dev.sh)\n");
00291     printf("%s",("\nlog "+ COLOR + "$EXECUTION" + RESET_COLOR + " complete\n\n").c_str());
00292 
00293     if (isDataDevice){
00294         printf("%s",(GREY_COLOR + "# Select one main generator for the data device.  This will be the
      function \n").c_str());
00295         printf("# responsible for generating all measurements.  We provided one example of data
      generators.  \n");
00296         printf("# Feel free for adding more by yourself in /src/exampleSkillsDataset.cpp\n");
00297         printf("# -> exampleGenerator1\n");
00298         printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
      ").c_str());
00299     }else{
00300         printf("%s",(GREY_COLOR + "# Select one main generator for the kernel device.  This will be
      the function \n").c_str());
00301         printf("# responsible for checking the security status.  We provided 1 example of main
      generator.  \n");
00302         printf("# Feel free for adding more by yourself in /src/exampleSecurityDataset.cpp\n");
00303         printf("# -> exampleSecurity1\n");
00304         printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
      ").c_str());
00305     }
```

```
00306
00307 }
```

References checkAbortStatus(), COLOR, GREY_COLOR, and RESET_COLOR.

Referenced by AI::buildDevice().

4.17.3.8 execBuildName()

`void SimpleDisplay::execBuildName ( )  [static]`

Prints first section of ExecBuilding.

Definition at line 247 of file SimpleDisplay.cpp.

```
00247                                   {
00248     checkAbortStatus();
00249     jump();
00250     printf("%s",(COLOR + "#!/bin/bash" + RESET_COLOR + "\n\n").c_str());
00251     printf("%s",(GREY_COLOR + "# ------------------------------ DEVICE BUILDER
      ---------------------------------------\n").c_str());
00252     printf("%s",(GREY_COLOR + "# Welcome to the Device Builder in Execution Time.  We will guide you
      for easily adding\n").c_str());
00253     printf("%s",(GREY_COLOR + "# more devices to JVH_Systems without modifying the program.  This tool
      is experimental\n").c_str());
00254     printf("%s",(GREY_COLOR + "# can cause SEVERAL DAMAGE to your program.  Please, if you have any
      doubt about the \n").c_str());
00255     printf("%s",(GREY_COLOR + "# building process, type EXIT (anytime) and read more in our
      github:\n").c_str());
00256     printf("%s",(GREY_COLOR + "# https://github.com/clases-julio/p6-setstl-usuarios-SamthinkGit\n\n" +
      RESET_COLOR).c_str());
00257     printf("%s",("cd " + COLOR + "$HOME" + RESET_COLOR + "/src/deviceDataset.cpp\n").c_str());
00258     printf("%s",("slot=find("+COLOR+"\'DeviceNameSlot\'"+RESET_COLOR+")\n\n").c_str());
00259     printf("if [ slot == \"VALID_SLOT\" ]; then\n\n");
00260     printf("%s",(GREY_COLOR + " # Set the name of your device after 'name=', you can use both simbols
      a-zA-Z0-9 and spaces.\n").c_str());
00261     printf("%s",(GREY_COLOR + " # When you have written the name press ENTER. PD: You don't need to
      use Quoation marks\n" + RESET_COLOR).c_str());
00262     printf("%s",(GREY_COLOR + " # Example:  «stablish name=My Device 2»\n" + RESET_COLOR).c_str());
00263     printf("   stablish name=");
00264 }
```

References checkAbortStatus(), COLOR, GREY_COLOR, jump(), and RESET_COLOR.

Referenced by AI::buildDevice().

4.17.3.9 execBuildType()

`void SimpleDisplay::execBuildType ( )  [static]`

Prints second section of ExecBuilding.

Definition at line 267 of file SimpleDisplay.cpp.

```
00267                                   {
00268     checkAbortStatus();
00269     printf("%s",("   export "+ COLOR + "$name" + RESET_COLOR +" > /dev/sda1 | \\\n").c_str());
00270     printf(R"(                  grep -E '.*[\.dev,\.data]' | \)");
00271 printf("%s",("\n                   xarg -I touch "+ COLOR + "$slot" + RESET_COLOR + "." + COLOR +
      "$name"+ RESET_COLOR + "\n").c_str());
00272     printf("fi\n\n");
00273     printf("%s",("cd /dev/sda1/" + COLOR + "$slot\n" + RESET_COLOR).c_str());
00274     printf("%s",(GREY_COLOR + "\n# Select the type of device.  Write \'data\' to build a DataDevice or
      \'kernel\'").c_str());
00275     printf("%s",(GREY_COLOR + "\n# to build a KernelDevice.  Remember:  Don't use quotation marks." +
      RESET_COLOR).c_str());
00276     printf("%s",("\nwrite -t $(hex " + COLOR +"/usr/bin/gparted/sda1" + RESET_COLOR +" | grep -F
      'dev_type') < type.setDevicetype=").c_str());
00277 }
```

References checkAbortStatus(), COLOR, GREY_COLOR, and RESET_COLOR.

Referenced by AI::buildDevice().

4.17.3.10 jump()

```
void SimpleDisplay::jump ( ) [static]
```

Cleans the display using '\n'.

Definition at line 93 of file SimpleDisplay.cpp.

```
{
    for (int i = 0; i < 80; i++){
        std::cout « std::endl;
    };
}
```

Referenced by cleanDevi(), execBuildName(), printAIInterface(), printDeviHeader(), printLoginInterface(), printMainInterface(), printPresentation(), and printRequestPassword().

4.17.3.11 operator<<()

```
void SimpleDisplay::operator<< (
            SimpleDisplay display )
```

Inherit configuration from other display.

Parameters

display	Display to be inherited from

Definition at line 492 of file SimpleDisplay.cpp.

```
{

    this->securityStatus = display.securityStatus;
    this->securityConnected = display.securityConnected;
    this->user = display.user;
    this->group = display.group;

}
```

References group, securityConnected, securityStatus, and user.

4.17.3.12 printAIInterface()

```
void SimpleDisplay::printAIInterface ( ) [static]
```

Prints AI interface.

Definition at line 206 of file SimpleDisplay.cpp.

```
{

    checkAbortStatus();
    jump();

printf("#===================================================================================================#\n");
    printf("|                                         SETTINGS
|\n");
    printf("|
|\n");
```

```
    printf("|    Type a number for using one of the following Options
    |\n");
    printf("|
    |\n");
    printf("|    1)  Turn off/on all devices
    |\n");
    printf("|    2)  Turn off/on all security
    |\n");
    printf("|    3)  Check Security Status
    |\n");
    printf("|    4)  Use Microphone
    |\n");
    printf("|    5)  [EXP] Force a Security Failure
    |\n");
    printf("|    6)  [EXP] Build a New Device
    |\n");
    printf("|    7)  Add new User to System
    |\n");
    printf("|    8)  Remove User from System
    |\n");
    printf("|    9)  Return Security to safe status
    |\n");
    printf("|    10) [EXP] Force Bad_Alloc Error
    |\n");
    printf("|    11) Show all users
    |\n");
    printf("|    12) About Us
    |\n");
    printf("|
    |\n");
    printf("|    -1) Exit
    |\n");
    printf("|
    |\n");
printf("#=================================================================================================#\n");

}
```

References checkAbortStatus(), and jump().

Referenced by AI::forceError(), main(), and AI::recoverSecurity().

4.17.3.13 printChecking()

void SimpleDisplay::printChecking ( ) [static]

Prints text "Checking...".

Definition at line 77 of file SimpleDisplay.cpp.

```
00077                                  {
00078     std::cout « "\n\t\t\t\t\t    Checking...\n\n";
00079 }
```

Referenced by Login::startLogin().

4.17.3.14 printDeviHeader()

void SimpleDisplay::printDeviHeader (
 std::string *deviceName*) [static]

Prints Devi++ initial interface.

Parameters

deviceName	Name of current device

Definition at line 325 of file SimpleDisplay.cpp.

```
00325                                                        {
00326     jump();
00327
      printf("#==========================================================================================#\n");
00328     printf("|                                          %-20s
      |\n",deviceName.c_str());
00329
      printf("#==========================================================================================#\n\n");
00330     printf("~                               DEVi++ - Device VIM Based Terminal
      \n");
00331     printf("~
      \n");
00332     printf("~                                         version %-9s
      \n","DEV.0.2");
00333     printf("~                                    by Sebastian Mayorquin
      \n");
00334     printf("~
      \n");
00335     printf("~                                  Sponsor DEVi++ Development!
      \n");
00336     printf("~
      \n");
00337     printf("%s", ("~                                type       :wq" + COLOR +"<Enter>" + RESET_COLOR +"
      to exit                                            \n").c_str());
00338     printf("%s", ("~                                type       :help" + COLOR +"<Enter>" + RESET_COLOR +"
      for DEVi help                                      \n").c_str());
00339     printf("%s", ("~                                type       :dev" + COLOR +"<Enter>" + RESET_COLOR +"
      for skills help                                    \n").c_str());
00340     printf("%s", ("~                                type       :clean" + COLOR +"<Enter>" + RESET_COLOR +"
      for cleaning terminal                                      \n").c_str());
00341
00342     printf("~
      \n");
00343     printf("~                             Thanks for trusting in JVH Systems!
      \n");
00344     printf("~\n");
00345     printf("~\n");
00346     printf("~\n");
00347     printf("~\n");
00348     printf("~\n");
00349     printf("~\n");
00350     printf("~\n");
00351
00352
00353 }
```

References COLOR, jump(), and RESET_COLOR.

Referenced by main().

4.17.3.15 printDeviHelp()

`void SimpleDisplay::printDeviHelp ( )  [static]`

Prints Devi++ help manual.

Definition at line 357 of file SimpleDisplay.cpp.

```
00357                                   {
00358
00359     printf("~ -------------------- DEVi++ Help --------------------\n");
00360     printf("~ Devi++ is a VIM based terminal that will allow you to\n");
00361     printf("~ interact with your devices with commands.  There are \n");
00362     printf("%s", ("~ two types of commands " + COLOR + "Global Commands" + RESET_COLOR + " and " +
      COLOR + "Device Skills"+ RESET_COLOR + "\n").c_str());
00363     printf("~ \n");
00364     printf("~ Global commands are available in every device and let\n");
00365     printf("~ you interact with DEVi or with your Device status.\n");
```

```
    printf("~ For using them you only have to type any of the following\n");
    printf("~ orders:\n");
    printf("~ \n");
    printf("%s", ("~ -> " + COLOR + ":wq"+ RESET_COLOR + "       Exit the terminal (Do not use Ctrl + C)\n").c_str());
    printf("%s", ("~ -> " + COLOR + ":help"+ RESET_COLOR + "     Displays DEVi manual \n").c_str());
    printf("%s", ("~ -> " + COLOR + ":clean"+ RESET_COLOR + "    Clean DEVi display \n").c_str());
    printf("~ \n");
    printf("%s", ("~ -> " + COLOR + "data"+ RESET_COLOR + "      Displays collected data since starting/1h\n").c_str());
    printf("%s", ("~ -> " + COLOR + "forceref"+ RESET_COLOR + "  Forces current device to refresh data\n").c_str());
    printf("%s", ("~ -> " + COLOR + "turnon"+ RESET_COLOR + "    Turns the device ON\n").c_str());
    printf("%s", ("~ -> " + COLOR + "turnoff"+ RESET_COLOR + "   Turns the device OFF\n").c_str());
    printf("%s", ("~ -> " + COLOR + "reboot"+ RESET_COLOR + "    Reset the device\n").c_str());
    printf("%s", ("~ -> " + COLOR + "who"+ RESET_COLOR + "       Prints the name of the user\n").c_str());
    printf("%s", ("~ -> " + COLOR + "pwd"+ RESET_COLOR + "       Prints current path\n").c_str());

    printf("~ \n");
    printf("~ Note:  While DEVi terminal is active, all devices will be\n");
    printf("~ stopped until applying changes\n");
    printf("~ \n");
    printf("~ Each device has its own commands.  Those commands are \n");
    printf("~ called as skills and are automatically detected when \n");
    printf("~ starting the program.  You can look at the self-generated\n");
    printf("%s", ("~ skills manual by executing "+ COLOR + ":dev"+ RESET_COLOR + "\n").c_str());
    printf("~ -----------------------------------------------------\n");
}
```

References COLOR, and RESET_COLOR.

Referenced by IODi::requestSkill().

4.17.3.16 printLoginInterface()

```
void SimpleDisplay::printLoginInterface (
            int user,
            int pass,
            int INVALID_USER = -1,
            int INVALID_PASS = -1 )  [static]
```

Prints the login interface (encrypted)

Parameters

user	username to be printed in screen
password	Password to encrypt and print in screen
INVALID_USER	User to do NOT print user in screen. -1 as default.
INVALID_PASS	Pass to do NOT print user in screen. -1 as default.

Definition at line 30 of file SimpleDisplay.cpp.

```
00030                                                                                   {
00031 
00032     // ----- Declarations ------
00033 
00034     std::string current_user;
00035     std::string current_pass;
00036 
00037 
00038     // ----- Checking if variables are printable ------
00039 
00040     if (user == INVALID_USER) current_user = "";
00041     else current_user = std::to_string(user);
00042 
00043     if (pass == INVALID_PASS) current_pass = "";
00044     else current_pass= std::to_string(pass);
```

```


    // ----- Printing Interface ------
    jump();
    std::cout « "    _____  __     __  __    __         ______                          __   \n";
    std::cout « "   |      \\|  \\    |  \\|  \\  |  \\        /       \\                          |  \\ \n";
    std::cout « "    \\$$$$$| $$    | $$| $$   | $$       |  $$$$$$\\ __    __   _______  _| $$_ ______   ______ _____    _______ \n";
    std::cout «"      | $$| $$    | $$| $$__| $$       | $$___\\$$|  \\  |  \\ /       \\|   $$ \\   / \\ |      \\    \\  /       \\\n";
    std::cout « " __   | $$ \\$$\\ /  $$| $$    $$        \\$$    \\ | $$  | $$|  $$$$$$$ \\$$$$$$  | $$$$$$\\| $$$$$$\\$$$$\\|  $$$$$$$\n";
    std::cout « "|  \\   | $$  \\$$\\  $$ | $$$$$$$$        _\\$$$$$$\\| $$  | $$ \\$$    \\    | $$ __ | $$    $$| $$ | $$ | $$ \\$$    \\ \n";
    std::cout « "| $$__| $$   \\$$ $$  | $$  | $$       |  \\__| $$| $$__/ $$ _\\$$$$$$\\  | $$|  \\| $$$$$$$$| $$ | $$ | $$ _\\$$$$$$\\\n";
    std::cout « " \\$$    $$    \\$$$   | $$  | $$        \\$$    $$ \\$$    $$|       $$   \\$$  $$ \\$$     \\| $$ | $$ | $$|       $$\n";
    std::cout « "  \\$$$$$$      \\$     \\$$   \\$$         \\$$$$$$  _\\$$$$$$$ \\$$$$$$$     \\$$$$ \\$$$$$$$ \\$$  \\$$  \\$$ \\$$$$$$$ \n";
    std::cout « "                                                |  \\__| $$                                                       \n";
    std::cout « "                                                 \\$$    $$                                                       \n";
    std::cout « "                                                  \\$$$$$$                                                        \n\n\n";
    std::cout « "\t\t\t#---------------------------------------------------#\n";
    std::cout « "\n\t\t\t\t\t-> User:" « current_user « std::endl;
    std::cout « "\t\t\t\t\t-> Password:";


    // ----- Encrypting Password ------

    for (int i = 0; i < current_pass.size(); i++) std::cout « '*';


    // ----- Printing Interface ------

    std::cout « "\n\n\t\t\t#---------------------------------------------------#\n";
    std::cout « "\n\t\t\t\t       Type -1 to exit\n";
}
```

References jump(), and user.

Referenced by Login::startLogin().

4.17.3.17 printMainInterface()

```
void SimpleDisplay::printMainInterface (
            std::string * deviceNames,
            bool * deviceStatus )
```

Prints main interface to screen using SimpleDisplay properties.

Parameters

deviceNames	Names of every dataDevice to be printedç
deviceStatus	

Definition at line 105 of file SimpleDisplay.cpp.

```
00105                                                                              {
00106     std::string directory[SimpleDisplay::MAXIMUM_OPTIONS];
00107     std::string option[SimpleDisplay::MAXIMUM_OPTIONS];
00108     std::string status[SimpleDisplay::MAXIMUM_OPTIONS];
00109
00110
00111     for (int i = 0; i < this->dataDevices; i++) {
```

```
        directory[i] = deviceNames[i];
        option[i] = std::to_string(i) + ")";
        if (deviceStatus[i])
            status[i] = "Running";
        else
            status[i] = "Inactive";
    }


    for (int i = this->dataDevices; i < SimpleDisplay::MAXIMUM_OPTIONS; i++) {
        directory[i] = "";
        option[i] = "";
        status[i] = "-";
    }

    std::string conected;
    if (this->securityConnected)
        conected = "Connected";
    else
        conected = "Disconnected";

    std::string secStatus;
    if (this->securityStatus)
        secStatus = "Secure";
    else
        secStatus = "WARNING";

    std::string username = (this->user == 0) ?  "root" :  std::to_string(this->user);
    std::string group = (this->user == -1) ?  "DEVMODE" :  this->group;

    jump();

printf("#=========================##=======================================================================#\n")
    printf("|  %-23s ||
|\n","---- [DIRECTORY] ----");
    printf("| %-3s %-20s ||       #------------------     JVH - Systems     --------------------#
|\n",option[0].c_str(),directory[0].c_str());
    printf("| %-3s %-20s ||
|\n",option[1].c_str(),directory[1].c_str());
    printf("| %-3s %-20s ||       Welcome to the Main Interface of JVH! Select an option from the
|\n",option[2].c_str(),directory[2].c_str());
    printf("| %-3s %-20s ||       directory by writting its name or position and pressing ENTER.
|\n",option[3].c_str(),directory[3].c_str());
    printf("| %-3s %-20s ||
|\n",option[4].c_str(),directory[4].c_str());
    printf("| %-3s %-20s ||                       [Current Status]
|\n",option[5].c_str(),directory[5].c_str());
    printf("| %-3s %-20s ||
|\n",option[6].c_str(),directory[6].c_str());
    printf("| %-3s %-20s ||       [Security Alarms]:  %-12s      [Security Status]:  %-8s
|\n",option[7].c_str(),directory[7].c_str(),conected.c_str(),secStatus.c_str());
    printf("| %-3s %-20s ||
|\n",option[8].c_str(),directory[8].c_str());
    printf("| %-3s %-20s ||         Device 1:     %-11s                Device 7:     %-11s
|\n",option[9].c_str(),directory[9].c_str(),status[0].c_str(),status[6].c_str());
    printf("| %-3s %-20s ||         Device 2:     %-11s                Device 8:     %-11s
|\n",option[10].c_str(),directory[10].c_str(),status[1].c_str(),status[7].c_str());
    printf("| %-3s %-20s ||         Device 3:     %-11s                Device 9:     %-11s
|\n",option[11].c_str(),directory[11].c_str(),status[2].c_str(),status[8].c_str());
    printf("|  %-23s ||         Device 4:     %-11s                Device 10:  %-11s
|\n","",status[3].c_str(),status[9].c_str());
    printf("|  %-23s ||         Device 5:     %-11s                Device 11:  %-11s
|\n","",status[4].c_str(),status[10].c_str());
    printf("|  %-23s ||         Device 6:     %-11s                Device 12:  %-11s
|\n","",status[5].c_str(),status[11].c_str());
    printf("|  %-23s ||
|\n","");
    printf("| %-2d) %-20s ||       [User]:  %-5s                       [Kernel Devices]:  %-2d
|\n",-1,"EXIT",username.c_str(),this->kernelDevices);
    printf("| %-2d) %-20s ||       [Group]:  %-7s                  [Data Devices]:  %-2d
|\n",-2,"SETTINGS",group.c_str(),this->dataDevices);
    printf("|  %-23s ||
|\n","");

printf("#=========================##=======================================================================#\n")

}
```

References dataDevices, group, jump(), MAXIMUM_OPTIONS, securityConnected, securityStatus, and user.

Referenced by main().

4.17.3.18 printPresentation()

```
void SimpleDisplay::printPresentation ( )  [static]
```

Prints presentation text before main program.

Definition at line 169 of file SimpleDisplay.cpp.

```
00169                                              {
00170 
00171     jump();
00172 
00173     printf("#==================================       JVH-Systems  V.0.3.1
      ========================================#\n\n");
00174     printf("       Bienvenido Software Developer
      \n");
00175     printf("\n");
00176     printf("       Esta pantalla es puramente informativa y NO aparecera en la version FINAL del
      programa.\n");
00177     printf("       A continuacion vas a acceder al programa JVH-Systems version V.0.3.1 Antes de
      lanzarte\n");
00178     printf("       al programa te recomendamos que compruebes los siguientes parametros:\n");
00179     printf("\n");
00180     printf("    1.  Comprueba que tu terminal es capaz de ver el recuadro en el que se encuentra
      incluido\n");
00181     printf("       este texto EN SU TOTALIDAD.\n");
00182     printf("\n");
00183     printf("%s", ("    2.  Comprueba que tu terminal colorea " + COLOR + "estas palabras" +
      RESET_COLOR + " de color azul.  Si tu terminal no acepta\n").c_str());
00184     printf("       colores ANSI cierra este programa con Ctrl+C y ejecuta ./main --no-colors\n");
00185     printf("\n");
00186     printf("    3.  Una vez hayas comprobado todo estaras listo para testear este programa
      adecuadamente.\n");
00187     printf("\n");
00188     printf("       Hemos dejado a tu disposicion dos cuentas.  Puedes agregar tras iniciar el
      programa:\n");
00189     printf("
      \n");
00190     printf("                          root:  0                  GUEST: 99999\n");
00191     printf("                        Password:  0                Password:  99999999\n");
00192     printf("\n");
00193     printf("       Tambien puedes utilizar ./main --developer para solucionar problemas con la base de
      datos\n\n");
00194 #ifndef _WIN32
00195     printf("%s", ("       " + WARNING_COLOR + "[WARNING]" + RESET_COLOR + " El sistema ha detectado
      que no estas utilizando Windowsx86 o que\n").c_str());
00196     printf("       estas utilizando Linux.  Se han desativado las opciones --developer y --no-colors
      \n");
00197     printf("       en versiones anteriores a V.0.3.0 por razones de seguridad.  PD: Versión en el
      titulo.\n\n");
00198 #endif
00199 
      printf("#=========================================================================================================#\n");
00200 
00201 }
```

References COLOR, jump(), RESET_COLOR, and WARNING_COLOR.

Referenced by main().

4.17.3.19 printRequestPassword()

```
void SimpleDisplay::printRequestPassword ( )  [static]
```

Prints Authentication Screen from AI.

Definition at line 235 of file SimpleDisplay.cpp.

```
00235                                                 {
00236 
00237     jump();
00238     std::cout « "#---------------------------------------------------------------------#\n";
00239     std::cout « "|                                                                     |\n";
00240     std::cout « "|      The command requested needs administrator/developer            |\n";
```

```
    std::cout « "|      access.  Please verify your account typing root password       |\n";
    std::cout « "|                                                                     |\n";
    std::cout « "#---------------------------------------------------------------------#\n";
}
```

References jump().

Referenced by AI::ensureAdminAccess().

4.17.3.20 printSkillManual()

```
void SimpleDisplay::printSkillManual (
            std::string * skillnames,
            int count ) [static]
```

Prints a self-generated manual based in current SimpleDisplay.

Parameters

skillnames	array of names of current device skills
number	of skills detected

Definition at line 393 of file SimpleDisplay.cpp.

```
00393                                                                             {
00394
00395     printf("~ -------- DEVi++ Self-Generated Skill Manual ---------\n");
00396     printf("~ Every device has its own commands, JVH implements a  \n");
00397     printf("~ builder to install high-level functions into the     \n");
00398     printf("~ interface.  This device functionalities are called    \n");
00399     printf("%s", ("~ " + COLOR + "skills" + RESET_COLOR + ".  In this version, we only allow
      high-level     \n").c_str());
00400     printf("~ developers to integrate the name of the skill without  \n");
00401     printf("~ description.\n");
00402     printf("~ \n");
00403     printf("~ This manual will show all skills detected for the\n");
00404     printf("~ selected device, for more help, you can search in\n");
00405     printf("~ our wiki:  \n");
00406     printf("~ \n");
00407     printf("%s", ("~ " + COLOR +  "https://github.com/clases-julio/p4-implclases-SamthinkGit" +
      RESET_COLOR + "\n").c_str());
00408     printf("~ \n");
00409     printf("~ [DEVi++] Searching Skills...\n");
00410     printf("%s", ("~ [DEVi++] " + std::to_string(count+7) + " skills detected.\n").c_str());
00411     printf("~ [DEVi++] Generating Manual:\n");
00412     printf("~ \n");
00413     printf("%s",("~ [GLOBAL] -> " + COLOR + "data" + RESET_COLOR + "\n").c_str());
00414     printf("%s",("~ [GLOBAL] -> " + COLOR + "forceref" + RESET_COLOR + "\n").c_str());
00415     printf("%s",("~ [GLOBAL] -> " + COLOR + "turnon" + RESET_COLOR + "\n").c_str());
00416     printf("%s",("~ [GLOBAL] -> " + COLOR + "turnoff" + RESET_COLOR + "\n").c_str());
00417     printf("%s",("~ [GLOBAL] -> " + COLOR + "reboot" + RESET_COLOR + "\n").c_str());
00418
00419
00420     for (int i = 0; i < count; i++){
00421         printf("%s",("~ [DETECTED] -> " + COLOR + skillnames[i] + RESET_COLOR + "\n").c_str());
00422     }
00423     printf("~ \n");
00424     printf("~ NOTE: For using skills only type the name in DEVi++ and press <ENTER>\n");
00425     printf("~ -----------------------------------------------------\n");
00426 }
```

References COLOR, and RESET_COLOR.

Referenced by IODi::requestSkill().

4.17.3.21 setDataDevices()

```
void SimpleDisplay::setDataDevices (
            int dataDevices )
```

Sets number of Data Devices.

Parameters

dataDevices	

Definition at line 480 of file SimpleDisplay.cpp.

```
00480                                               {
00481     SimpleDisplay::dataDevices = dataDevices;
00482 }
```

References dataDevices.

Referenced by DeviceBuilder::buildDisplay().

4.17.3.22 setGroup()

```
void SimpleDisplay::setGroup (
            const std::string & group )
```

Sets current user group.

Parameters

group	

Definition at line 488 of file SimpleDisplay.cpp.

```
00488                                                {
00489     SimpleDisplay::group = group;
00490 }
```

References group.

Referenced by main().

4.17.3.23 setKernelDevices()

```
void SimpleDisplay::setKernelDevices (
            int kernelDevices )
```

Sets number of Kernel Devices.

Parameters

kernelDevices	

Definition at line 476 of file SimpleDisplay.cpp.

```
00476                                                    {
00477     SimpleDisplay::kernelDevices = kernelDevices;
00478 }
```

References kernelDevices.

Referenced by DeviceBuilder::buildDisplay().

4.17.3.24 setSecurityConnected()

```
void SimpleDisplay::setSecurityConnected (
            bool securityConnected )
```

Sets the connection status of the alarms.

Parameters

securityConnected	Status of alarms: (True) Connected. (False) Disconnected

Definition at line 468 of file SimpleDisplay.cpp.

```
00468                                                            {
00469     SimpleDisplay::securityConnected = securityConnected;
00470 }
```

References securityConnected.

Referenced by DeviceBuilder::buildDisplay(), and AI::turnSecurity().

4.17.3.25 setSecurityStatus()

```
void SimpleDisplay::setSecurityStatus (
            bool securityStatus )
```

Sets the security status of the alarms.

Parameters

securityStatus	Status of alarms: (True) Secure. (False) Insecure

Definition at line 472 of file SimpleDisplay.cpp.

```
00472                                                      {
00473     SimpleDisplay::securityStatus = securityStatus;
00474 }
```

References securityStatus.

Referenced by DeviceBuilder::buildDisplay(), AI::forceError(), AI::recoverSecurity(), and AI::refreshData().

4.17.3.26 setUser()

```
void SimpleDisplay::setUser (
            int user )
```

Sets current user identificator.

Parameters

user	

Definition at line 484 of file SimpleDisplay.cpp.

```
00484                                               {
00485     SimpleDisplay::user = user;
00486 }
```

References user.

Referenced by main().

4.17.4 Member Data Documentation

4.17.4.1 COLOR

```
std::string SimpleDisplay::COLOR = "\033[36m"  [static]
```

String to call bold font.

Warning

Can vary between terminals

Definition at line 155 of file SimpleDisplay.h.

Referenced by deviPrintData(), execBuildFinish(), execBuildGenerator(), execBuildName(), execBuildType(), main(), printDeviHeader(), printDeviHelp(), printPresentation(), and printSkillManual().

4.17.4.2 currentDisplay

```
SimpleDisplay * SimpleDisplay::currentDisplay  [static]
```

Pointer to current Display.

Definition at line 170 of file SimpleDisplay.h.

Referenced by AI::forceError(), main(), AI::recoverSecurity(), AI::refreshData(), SimpleDisplay(), and AI::turnSecurity().

4.17.4.3 dataDevices

```
int SimpleDisplay::dataDevices  [private]
```

Number of Data Devices detected.

Definition at line 200 of file SimpleDisplay.h.

Referenced by printMainInterface(), and setDataDevices().

4.17.4.4 GREY_COLOR

```
std::string SimpleDisplay::GREY_COLOR = "\033[32m"  [static]
```

String to call secondary font.

Warning

Can vary between terminals

Definition at line 159 of file SimpleDisplay.h.

Referenced by execBuildGenerator(), execBuildName(), execBuildType(), and main().

4.17.4.5 group

```
std::string SimpleDisplay::group  [private]
```

Group of current user.

Definition at line 206 of file SimpleDisplay.h.

Referenced by operator<<(), printMainInterface(), and setGroup().

4.17.4.6 kernelDevices

```
int SimpleDisplay::kernelDevices  [private]
```

Number of Kernel Devices detected.

Definition at line 197 of file SimpleDisplay.h.

Referenced by setKernelDevices().

4.17.4.7 MAXIMUM_OPTIONS

```
int SimpleDisplay::MAXIMUM_OPTIONS = 12 [static], [private]
```

Maximum options to display in main interface.

Warning

Display can make unestable if changing variable

Definition at line 184 of file SimpleDisplay.h.

Referenced by printMainInterface().

4.17.4.8 RESET_COLOR

```
std::string SimpleDisplay::RESET_COLOR = "\033[0m" [static]
```

String to call default font.

Warning

Can vary between terminals

Definition at line 163 of file SimpleDisplay.h.

Referenced by deviPrintData(), execBuildFinish(), execBuildGenerator(), execBuildName(), execBuildType(), main(), printDeviHeader(), printDeviHelp(), printPresentation(), and printSkillManual().

4.17.4.9 security_abort

```
bool SimpleDisplay::security_abort = false [static]
```

Variable to jump if program fail. Default in False.

Definition at line 173 of file SimpleDisplay.h.

Referenced by checkAbortStatus(), and AI::signalHandler().

4.17.4.10 securityConnected

```
bool SimpleDisplay::securityConnected [private]
```

Conection status of alarms.

Definition at line 191 of file SimpleDisplay.h.

Referenced by operator<<(), printMainInterface(), and setSecurityConnected().

4.17.4.11 securityStatus

```
bool SimpleDisplay::securityStatus [private]
```

Security status of alarms.

Definition at line 194 of file SimpleDisplay.h.

Referenced by operator<<(), printMainInterface(), and setSecurityStatus().

4.17.4.12 user

```
int SimpleDisplay::user [private]
```

Identifier of current user.

Definition at line 203 of file SimpleDisplay.h.

Referenced by operator<<(), printLoginInterface(), printMainInterface(), and setUser().

4.17.4.13 WARNING_COLOR

```
std::string SimpleDisplay::WARNING_COLOR = "\033[31m" [static]
```

String to call warning color.

Warning

Can vary between terminals

Definition at line 167 of file SimpleDisplay.h.

Referenced by main(), and printPresentation().

The documentation for this class was generated from the following files:

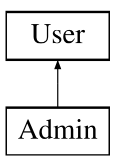
- SimpleDisplay.h
- SimpleDisplay.cpp

4.18 User Class Reference

This class will be the responsible for storing and managing the user's data. Also is able to create an abstract user object to ease data access.

```
#include <User.h>
```

Inheritance diagram for User:



Public Types

- enum {
INVALID_USER = -2 , INVALID_PASS = -2 , USER_COLUMN = 0 , PASS_COLUMN = 1 ,
ADMIN_COLUMN = 2 , ADMIN_IDENTIFIER = 1 , TOTAL_COLUMNS = 3 , DEVELOPER_USER = -1 }

Public Member Functions

- User ()
  Empty constructor (not recommended)
- User (int id)
  Build a new ACTIVE user. Only use to ease user information access. Auto sets the admin role according to data stored in file.
- int getUser ()
  Returns User's username.
- bool getAdminStatus ()
  Returns User's admin status.

Static Public Member Functions

- static bool check (int id, int pass)
  Check if id and password are valid. Needs to access to datafile.
- static void updateUserData ()
  Reads the datafile, then store users in a 2 dimensional array (allUsers) setting each column according to the columns setted in User.h class. Sorted by id for efficiency.
- static void saveUserData ()
  Saves all data saved in the set vector to a .dat file.
- static void addUser (int id, int pass, bool isAdmin)
  Adds new user to allUsers.
- static void rmUser (int id)
  Removes a user from allUsers.
- static void printUsers ()
  Prints all registered users.

Static Protected Member Functions

- static int ∗ find (int id)
  Finds a user by giving the id as a key . Maximum complexity of O(n). Returns the properties of the user as a pointer to array.

Protected Attributes

- int id
  User identifier (5-digit-int)
- bool isAdmin
  Admin Status. False by defect.
- int pass
  Password to access the account.

Static Protected Attributes

- static char SEPARATOR = ','

  Character to separate tokens in file.
- static std::string TAG = "User"

  Name of the Class (for exception calling)
- static std::set< int * > allUsers

  Set of integers to store all users.
- static std::string DATAFILE = "data/userdata.txt"

  [DEPRECATED] Path of userdata file
- static std::string USERFILE = "data/userdata.dat"

  Path to userdata file.

4.18.1 Detailed Description

This class will be the responsible for storing and managing the user's data. Also is able to create an abstract user object to ease data access.

Definition at line 16 of file User.h.

4.18.2 Member Enumeration Documentation

4.18.2.1 anonymous enum

`anonymous enum`

Enumerator

INVALID_USER	
INVALID_PASS	
USER_COLUMN	
PASS_COLUMN	
ADMIN_COLUMN	
ADMIN_IDENTIFIER	
TOTAL_COLUMNS	
DEVELOPER_USER	

Definition at line 24 of file User.h.

```
00024          {
00025         INVALID_USER = -2,
00026         INVALID_PASS = -2,
00027         USER_COLUMN = 0,
00028         PASS_COLUMN = 1,
00029         ADMIN_COLUMN = 2,
00030         ADMIN_IDENTIFIER = 1,
00031         TOTAL_COLUMNS = 3,
00032         DEVELOPER_USER = -1
00033     };
```

4.18.3 Constructor & Destructor Documentation

4.18.3.1 User() [1/2]

```
User::User ( )
```

Empty constructor (not recommended)

Definition at line 24 of file User.cpp.

```
00024 : id(INVALID_USER), isAdmin(false) {};
```

Referenced by saveUserData().

4.18.3.2 User() [2/2]

```
User::User (
              int id )
```

Build a new ACTIVE user. Only use to ease user information access. Auto sets the admin role according to data stored in file.

Parameters

id	user identification (5-digit num)

Definition at line 28 of file User.cpp.

```
00028                 : id(id){
00029 
00030     if (id == DEVELOPER_USER ){
00031         this->pass = 0;
00032         this->isAdmin = true;
00033         return;
00034     }
00035 
00036     // ----- Initializing User info ------
00037     try {
00038 
00039         //  Find and Set admin status from data
00040         int* userData = find(id);
00041         this->pass = userData[PASS_COLUMN];
00042         this->isAdmin = userData[ADMIN_COLUMN];
00043 
00044 
00045     // ----- Catching invalid users ------
00046     }catch(InvalidUserException &e){
00047 
00048         this->id = INVALID_USER;
00049         this->isAdmin = 0;
00050 
00051     }
00052 };
```

References ADMIN_COLUMN, DEVELOPER_USER, find(), INVALID_USER, isAdmin, pass, and PASS_COLUMN.

4.18.4 Member Function Documentation

4.18.4.1 addUser()

```
void User::addUser (
            int id,
            int pass,
            bool isAdmin )  [static]
```

Adds new user to allUsers.

Parameters

id	user identification (5-digit num)
pass	user password (8-digit num)
isAdmin	Admin Status. (True) Admin. (False) Guest.

Definition at line 182 of file User.cpp.

```
                                             {

    int* newUser = new int[TOTAL_COLUMNS];
    newUser[USER_COLUMN] = user;
    newUser[PASS_COLUMN] = pass;
    newUser[ADMIN_COLUMN] = (isAdmin) ?  ADMIN_IDENTIFIER :  0;

    allUsers.insert(newUser);

}
```

References ADMIN_COLUMN, ADMIN_IDENTIFIER, allUsers, isAdmin, pass, PASS_COLUMN, TOTAL_COLUMNS, and USER_COLUMN.

Referenced by AI::addUser().

4.18.4.2 check()

```
bool User::check (
            int id,
            int pass )  [static]
```

Check if id and password are valid. Needs to access to datafile.

Parameters

id	user identification (5-digit num)
pass	user password (8-digit num)

Returns

True if id and pass are valid

Definition at line 68 of file User.cpp.

```
                                {
    try{
        int* userData = find(id);
        return (userData[PASS_COLUMN] == pass);
```

```
00072
00073     // User not found
00074     }catch(UserNotFoundException &e){
00075         return false;
00076     }
00077 }
```

References find(), pass, and PASS_COLUMN.

Referenced by Login::startLogin().

4.18.4.3 find()

```
int * User::find (
              int id )  [static], [protected]
```

Finds a user by giving the id as a key . Maximum complexity of O(n). Returns the properties of the user as a pointer to array.

Parameters

id	key for searching user

Note

Complexity of O(log(n)) is possible, not implemented yet.

Exceptions

UserNotFoundException	User not found

Definition at line 163 of file User.cpp.

```
00163                    {
00164
00165     // ----- Creates iterator
00166     std::set<int*>::iterator it;
00167
00168     // ----- Travel the set until find
00169     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00170
00171         if ((*iterator)[USER_COLUMN] == id)
00172             return *iterator;
00173     }
00174
00175     // ----- User not found, send error
00176
00177     // [DEPRECADTED] throw std::runtime_error("User not Found");
00178     throw UserNotFoundException(TAG);
00179
00180 }
```

References allUsers, TAG, and USER_COLUMN.

Referenced by Admin::adminStatus(), check(), rmUser(), and User().

4.18.4.4 getAdminStatus()

```
bool User::getAdminStatus ( )
```

Returns User's admin status.

Definition at line 62 of file User.cpp.

```
00062                    {
00063     return this->isAdmin;
00064 }
```

References isAdmin.

Referenced by main().

4.18.4.5 getUser()

```
int User::getUser ( )
```

Returns User's username.

Definition at line 57 of file User.cpp.

```
00057               {
00058     return this->id;
00059 }
```

References id.

Referenced by main().

4.18.4.6 printUsers()

```
void User::printUsers ( )  [static]
```

Prints all registered users.

Definition at line 209 of file User.cpp.

```
00209                       {
00210 
00211     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00212 
00213             std::cout « ((*iterator)[ADMIN_COLUMN] == 0 ?  "User:  " :  "Admin:  ");
00214             std::cout « (*iterator)[USER_COLUMN] « std::endl;
00215 
00216     }
00217 
00218 }
```

References ADMIN_COLUMN, allUsers, and USER_COLUMN.

Referenced by AI::showAllUsers().

4.18.4.7 rmUser()

```
void User::rmUser (
             int id )  [static]
```

Removes a user from allUsers.

Parameters

id	user identification

Definition at line 194 of file User.cpp.

```
                             {

    try {

        int *user = find(id);
        allUsers.erase(user);

    }catch(InvalidUserException &e){

        throw;

    };

}
```

References allUsers, and find().

Referenced by AI::removeUser().

4.18.4.8 saveUserData()

`void User::saveUserData ( ) [static]`

Saves all data saved in the set vector to a .dat file.

Note

Path file can be changed with USERFILE

This function should be only called at the end of the program

Definition at line 220 of file User.cpp.

```
                             {

    // ---- Opening file ----
    std::ofstream file (USERFILE);

    // ---- Testing if file is correct ----
    if (!file)
        throw InvalidFileException(TAG);

    // ----- Creates iterator -----
    std::set<int*>::iterator it;

    // ----- Saves all user data in allUsers to file  -----
    for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){

        User newUser = User((*iterator)[USER_COLUMN]);
        file.write(reinterpret_cast <const char *> (&newUser),sizeof (User));

    }

    // ----- Close the file -----
    file.close();

}
```

References allUsers, TAG, User(), USER_COLUMN, and USERFILE.

Referenced by AI::signalHandler().

4.18.4.9 updateUserData()

`void User::updateUserData ( ) [static]`

Reads the datafile, then store users in a 2 dimensional array (allUsers) setting each column according to the columns setted in User.h class. Sorted by id for efficiency.

Exceptions

InvalidUserException	User not valid/found
InvalidFileException	File not valid/not Accessible

Definition at line 79 of file User.cpp.

```
00079                           {
00080
00081     // ----- Declarations -----
00082     User user;
00083
00084     // ---- Opening file ----
00085     std::ifstream file (USERFILE);
00086
00087     // ---- Testing if file is correct ----
00088     if (!file)
00089         throw InvalidFileException(TAG);
00090
00091     // ----- Reading every user in file -----
00092     while (file.read(reinterpret_cast <char *>(&user),sizeof (User))){
00093
00094         int* newEntry = new int[TOTAL_COLUMNS];
00095         newEntry[USER_COLUMN] = user.id;
00096         newEntry[PASS_COLUMN] = user.pass;
00097         newEntry[ADMIN_COLUMN] = user.isAdmin;
00098
00099         allUsers.insert(newEntry);
00100     }
00101     // ----- Closing file -----
00102     file.close();
00103 }
```

References ADMIN_COLUMN, allUsers, id, isAdmin, pass, PASS_COLUMN, TAG, TOTAL_COLUMNS, USER_COLUMN, and USERFILE.

Referenced by main().

4.18.5 Member Data Documentation

4.18.5.1 allUsers

`std::set< int * > User::allUsers [static], [protected]`

Set of integers to store all users.

Definition at line 133 of file User.h.

Referenced by addUser(), find(), printUsers(), rmUser(), saveUserData(), and updateUserData().

4.18.5.2 DATAFILE

`std::string User::DATAFILE = "data/userdata.txt" [static], [protected]`

[DEPRECATED] Path of userdata file

Definition at line 135 of file User.h.

4.18.5.3 id

```
int User::id [protected]
```

User identifier (5-digit-int)

Note

root and developer don't need to use 5-digit numbers

Definition at line 105 of file User.h.

Referenced by getUser(), and updateUserData().

4.18.5.4 isAdmin

```
bool User::isAdmin [protected]
```

Admin Status. False by defect.

Definition at line 107 of file User.h.

Referenced by addUser(), Admin::Admin(), getAdminStatus(), updateUserData(), and User().

4.18.5.5 pass

```
int User::pass [protected]
```

Password to access the account.

Definition at line 109 of file User.h.

Referenced by addUser(), check(), updateUserData(), and User().

4.18.5.6 SEPARATOR

```
char User::SEPARATOR = ',' [static], [protected]
```

Character to separate tokens in file.

Definition at line 129 of file User.h.

4.18.5.7 TAG

```
std::string User::TAG = "User"  [static], [protected]
```

Name of the Class (for exception calling)

Definition at line 131 of file User.h.

Referenced by find(), saveUserData(), and updateUserData().

4.18.5.8 USERFILE

```
std::string User::USERFILE = "data/userdata.dat"  [static], [protected]
```

Path to userdata file.

Definition at line 137 of file User.h.

Referenced by saveUserData(), and updateUserData().

The documentation for this class was generated from the following files:

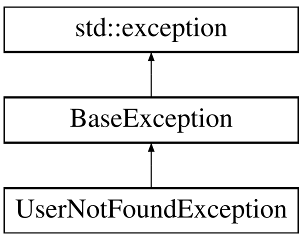
- User.h
- User.cpp

4.19 UserNotFoundException Class Reference

This class lets the program throw an exception when the selected user has not been found.

```
#include <UserNotFoundException.h>
```

Inheritance diagram for UserNotFoundException:



Public Member Functions

- UserNotFoundException (std::string &tag)

    Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "User not Found"

  Message to display in what()

Additional Inherited Members

4.19.1 Detailed Description

This class lets the program throw an exception when the selected user has not been found.

Definition at line 16 of file UserNotFoundException.h.

4.19.2 Constructor & Destructor Documentation

4.19.2.1 UserNotFoundException()

```
UserNotFoundException::UserNotFoundException (
            std::string & tag ) [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file UserNotFoundException.cpp.

```
00004                                                                    {
00005
00006     this->tag = tag;
00007     this->exception = ERROR_MESSAGE;
00008
00009 };
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.19.3 Member Data Documentation

4.19.3.1 ERROR_MESSAGE

```
std::string UserNotFoundException::ERROR_MESSAGE = "User not Found" [static], [private]
```

Message to display in what()

Definition at line 33 of file UserNotFoundException.h.

Referenced by UserNotFoundException().

The documentation for this class was generated from the following files:

- UserNotFoundException.h
- UserNotFoundException.cpp

Chapter 5

File Documentation

5.1 kb.cpp File Reference

```
#include "kb.h"
```

5.2 kb.cpp

Go to the documentation of this file.

```
#include "kb.h"

int kb::kbhit(){
  struct termios oldt, newt;
  int ch;
  int oldf;

  tcgetattr(STDIN_FILENO, &oldt);
  newt = oldt;
  newt.c_lflag &= ~(ICANON | ECHO);
  tcsetattr(STDIN_FILENO, TCSANOW, &newt);
  oldf = fcntl(STDIN_FILENO, F_GETFL, 0);
  fcntl(STDIN_FILENO, F_SETFL, oldf | O_NONBLOCK);

  ch = getchar();

  tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
  fcntl(STDIN_FILENO, F_SETFL, oldf);

  if(ch != EOF){
    ungetc(ch, stdin);
    return 1;
  }

  return 0;

}


```

5.3 kb.h File Reference

```
#include <stdio.h>
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
```

Classes

- class kb

5.4 kb.h

Go to the documentation of this file.

```
#ifndef KBHIT_H
#define KBHIT_H

#include <stdio.h>
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

class kb {

    public:
    static int kbhit();
};
#endif
```

5.5 Admin.h File Reference

```
#include "User.h"
```

Classes

- class Admin

    This class is the responsible for instantiating an Admin object to distinguish between Users and Admins.

5.5.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file Admin.h.

5.6 Admin.h

Go to the documentation of this file.

```
// ------------------- Admin - JVH Systems -------------------------------
#ifndef JVH_SYSTEMS_ADMIN_H
#define JVH_SYSTEMS_ADMIN_H
#include "User.h"

class Admin :  public User {

// -----------------------------------------------
//                    PUBLIC                    |
// -----------------------------------------------
public:

    // -----------------  DYNAMIC  ----------------------

    Admin(int id);

    // -----------------  STATIC  ----------------------

    static bool adminStatus(int id);

};

#endif //JVH_SYSTEMS_ADMIN_H
```

5.7 AI.h File Reference

```
#include "Login.h"
#include "lib.h"
#include "DataDevice.h"
#include "KernelDevice.h"
#include "IODi.h"
#include "../src/deviceDataset.cpp"
```

Classes

- class AI

    This class will be the responsible for interpreting global commands requested from the main class and checking the proper functioning of the devices. Thus, it will be implemented by the main class and can't live without Device class. Will be settled by the builder.

5.7.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

16/11/2022

Definition in file AI.h.

5.8 AI.h

Go to the documentation of this file.

```
// ------------------- Automatic Interface - JVH Systems -------------------------------
#ifndef AI_H
#define AI_H

#include "Login.h"
#include "lib.h"
#include "DataDevice.h"
#include "KernelDevice.h"
#include "IODi.h"
#include "../src/deviceDataset.cpp"
class AI {

// -----------------------------------------------
//                   PUBLIC                      |
// -----------------------------------------------
public:

    // -----------------  DYNAMIC  ----------------------

    // [Methods]

    AI();

    void startCollectingData();

    void refreshData();

    void throwDemon();

    // -----------------  STATIC  ----------------------

    // [Function]

    static void requestOption(std::string opt,bool isAdmin);

// -----------------------------------------------
//                  PRIVATE                      |
// -----------------------------------------------
private:

    enum {
        TURNDEVICES = 1,
        TURNSECURITY = 2,
        CHECKSECURITY = 3,
        MICROPHONE = 4,
        FORCEERROR = 5,
        BUILDDEVICE = 6,
        ADDUSER = 7,
        REMOVEUSER = 8,
        RETURNSECURITY = 9,
        FORCEBADALLOC = 10,
        SHOWUSERS = 11,
        MORE = 12,
        EXIT = -1
    };


    // -----------------  STATIC  ----------------------

    // [Functions]

    static void turnDevices();

    static void turnSecurity();

    static void checkSecurity();

    static void microphone();

    static void forceError();

    static void buildDevice();

    static void recoverSecurity();

    static void addUser();

    static void removeUser();

    static void forceBadAlloc();

    static void showAllUsers();
```

```
    static void ensureAdminAccess(bool isAdmin);

    static void signalHandler(int signum);

    // [Variables]

    static std::string TAG;
    static bool abort;
    static int MAX_DATA;
    static int SLEEPTIME;
    int time_counter;
};


#endif //AI_H
```

5.9 DataDevice.h File Reference

```
#include "Device.h"
```

Classes

- class DataDevice

  This class will contain specific properties of output devices and a set of functionalities for managing and checking their data. It inherits properties from Device class.

5.9.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file DataDevice.h.

5.10 DataDevice.h

Go to the documentation of this file.

```
// ------------------- DataDevice - JVH Systems ------------------------------
#ifndef DATADEVICE_H
#define DATADEVICE_H
#include "Device.h"

class DataDevice :  public Device{

    // This class will build all DataDevices
    friend class DeviceBuilder;
    friend class AI;
    friend class IODi;

// ---------------------------------------------
//                    PUBLIC                  |
// ---------------------------------------------
public:
```

```

    // -----------------  DYNAMIC  ----------------------

    // [Methods]

    DataDevice();

    DataDevice(std::string name, int id,std::vector<void(*)()>* skill, std::string* skillNames);

    // [Variables]

    void appendNewData();

    // -----------------  STATIC  ----------------------

    // [Functions]

    static DataDevice* allDataDevices;
    static std::string* allDataDeviceNames;

    // [Variables]

    static int getNumSensors();


// -----------------------------------------------
//                   PRIVATE                     |
// -----------------------------------------------
private:

    // -----------------  DYNAMIC  ----------------------

    // [Methods]

    void setLimit(int* limitArray);

    // [Variables]

    std::vector<int> collectedData;
    std::vector<std::string> collectedTimes;
    int(*dataGenerator)();
    bool needsLimit = false;
    int* limit;

    // -----------------  STATIC  ----------------------

    // [Functions]

    static void setAllDataDevices(DataDevice* allDevices);

    // [Variables]

    static int numSensors;
    static int numSensorsActive;

};


#endif //DATADEVICE_H
```

5.11 Device.h File Reference

```
#include "lib.h"
```

Classes

- class Device

  This class will contain every device as a general low-level definition, containing the skills, properties and a set of functionalities for managing all devices or modifying them individually. Every device will be built from the builder and the data managed will be managed from AI and IODi. This class will also be the base for constructing DataDevices and KernelDevices.

5.11.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file Device.h.

5.12 Device.h

Go to the documentation of this file.

```
// ------------------- Device - JVH Systems -------------------------------
#ifndef DEVICE_H
#define DEVICE_H
#include "lib.h"

class Device {

    friend class DeviceBuilder;
    friend class IODi;
    friend class AI;

// -----------------------------------------------
//                   PUBLIC                      |
// -----------------------------------------------
public:

    // -----------------  DYNAMIC  ----------------------

    // [Methods]

    Device();

    Device(std::string deviceName, int deviceId);

    // [Operators]

    void operator[](int index);

    void operator[](std::string skillName);

    bool operator~();

    // [Variables]

    std::string getName();

    int getNumSkills();

    // -----------------  STATIC  ----------------------

    static Device* allDevices;

// -----------------------------------------------
//                  PRIVATE                      |
// -----------------------------------------------
private:

    // -----------------  STATIC  ----------------------

    // [Functions]

    static std::string TAG;
    static std::string* allDeviceNames;
    static int numDevices;

    static void setAllDevices(Device* allNewDevices, std::string* allNewDeviceNames, int numDevices);

    static void setDeviceCounter(int num);

```

```
// ----------------------------------------------
//                  PROTECTED                   |
// ----------------------------------------------
protected:

    // -----------------  DYNAMIC  ---------------------

    // [Enum]

    enum{
        INVALID_DEVICE = -1  // Change to set invalid ID of devices
    };

    // [Variables]

    bool isDataDevice;
    bool isActive;
    int id;
    int numSkills;
    std::string* skillNames;
    std::string name;
    std::vector<void(*)()>* skills;

};

#endif //DEVICE_H
```

5.13 DeviceBuilder.h File Reference

```
#include "lib.h"
#include "DataDevice.h"
#include "KernelDevice.h"
```

Classes

- class DeviceBuilder

  This class lets a high-level programmer add easily more devices to the main program. You can use the implemented public functions wherever you need. For adding a new device you can follow this steps:

5.13.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file DeviceBuilder.h.

5.14 DeviceBuilder.h

Go to the documentation of this file.

```
// ------------------- DeviceBuilder - JVH Systems -------------------------------
#ifndef DEVICEBUILDER_H
#define DEVICEBUILDER_H
#include "lib.h"
#include "DataDevice.h"
#include "KernelDevice.h"

class DeviceBuilder {

// -----------------------------------------------
//                    PUBLIC                     |
// -----------------------------------------------
public:

    // -----------------  DYNAMIC  ----------------------

    DeviceBuilder(std::string name = "STD_DEVICE");

    void setSkill(void(*skill)(), std::string skillName);

    void setLimit(int* newLimit);

    void setAsDataDevice();

    void setAsKernelDevice();

    void setDataGenerator(int(*dataGenerator)());

    void setSecurityGenerator(bool(*securityGenerator)());

    // -----------------  STATIC  ----------------------

    static void buildAllDevices();

    static SimpleDisplay* buildDisplay();

    static void clearAll();

// -----------------------------------------------
//                   PRIVATE                     |
// -----------------------------------------------
private:

    // -----------------  DYNAMIC  ----------------------

    // [Methods]

    void buildDKDevice();

    // [Variables]

    std::string name = "STD Device";
    std::vector<void(*)()> skillVector;
    std::vector<std::string> skillNamesVector;
    int (*dataGenerator)();
    bool (*securityGenerator)();
    int id;
    int* limit = nullptr;
    bool isDataDevice;

    // -----------------  STATIC  ----------------------

    // [Variables]

    static std::vector<DeviceBuilder*> allBuilders;
    static std::vector<Device> allDevices;
    static std::vector<DataDevice> allDataDevices;
    static std::vector<KernelDevice> allKernelDevices;
    static int deviceCounter;


};


#endif //DEVICEBUILDER_H
```

5.15 BaseException.h File Reference

```
#include <string>
#include <exception>
```

Classes

- class BaseException

  This class establishes a basic format for JVH exceptions.

5.15.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file BaseException.h.

5.16 BaseException.h

Go to the documentation of this file.

```
// ------------------- BaseException - JVH Systems -------------------------------
#ifndef BASE_EXCEPTION_H
#define BASE_EXCEPTION_H
#include <string>
#include <exception>

class BaseException :  public std::exception {

// -----------------------------------------------
//                   PUBLIC                      |
// -----------------------------------------------
public:

    BaseException();

    const char* what();

// -----------------------------------------------
//                 PROTECTED                     |
// -----------------------------------------------
protected:

    std::string tag;
    std::string exception;


};

#endif //BASE_EXCEPTION_H
```

5.17 InsufficientPermissionsException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InsufficientPermissionsException

    This class lets the program throw an exception when the permissions of the user are not valid.

5.18 InsufficientPermissionsException.h

Go to the documentation of this file.

```
// ------------------- InsufficientPermissionException - JVH Systems -------------------------------
#ifndef INSUFFICIENTPERMISSIONSEXCEPTION_H
#define INSUFFICIENTPERMISSIONSEXCEPTION_H
#include "BaseException.h"
class InsufficientPermissionsException :    BaseException {

// -----------------------------------------------
//                    PUBLIC                     |
// -----------------------------------------------
public:

    explicit InsufficientPermissionsException(std::string &tag);

// -----------------------------------------------
//                   PRIVATE                     |
// -----------------------------------------------
private:

    static std::string ERROR_MESSAGE;

};


#endif //INSUFFICIENTPERMISSIONSEXCEPTION_H
```

5.19 InvalidDeviceException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidDeviceException

    This class lets the program throw an exception when the selected device is not valid.

5.19.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidDeviceException.h.

5.20 InvalidDeviceException.h

Go to the documentation of this file.

```
// ------------------- InvalidDeviceException - JVH Systems -------------------------------
#ifndef INVALIDDEVICE_H
#define INVALIDDEVICE_H
#include "BaseException.h"

class InvalidDeviceException :  public BaseException{

// -----------------------------------------------
//                   PUBLIC                      |
// -----------------------------------------------
public:

    explicit InvalidDeviceException(std::string &tag);

// -----------------------------------------------
//                  PRIVATE                      |
// -----------------------------------------------
private:

    static std::string ERROR_MESSAGE;

};


#endif //INVALIDDEVICE_H
```

5.21 InvalidFileException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidFileException

  This class lets the program throw an exception when the File to be read is not valid.

5.21.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidFileException.h.

5.22 InvalidFileException.h

Go to the documentation of this file.

```
// ------------------- InvalidFileException - JVH Systems ------------------------------
#ifndef INVALIDFILEEXCEPTION_H
#define INVALIDFILEEXCEPTION_H
#include "BaseException.h"

class InvalidFileException :  BaseException {

// -----------------------------------------------
//                    PUBLIC                      |
// -----------------------------------------------
public:

    explicit InvalidFileException(std::string &tag);

// -----------------------------------------------
//                   PRIVATE                      |
// -----------------------------------------------
private:

    static std::string ERROR_MESSAGE;

};


#endif //INVALIDFILEEXCEPTION_H
```

5.23 InvalidSkillException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidSkillException

  This class lets the program throw an exception when the selected skill is not valid.

5.23.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidSkillException.h.

5.24 InvalidSkillException.h

Go to the documentation of this file.

```
// ------------------- InvalidSkillException - JVH Systems ------------------------------
#ifndef INVALIDSKILLEXCEPTION_H
#define INVALIDSKILLEXCEPTION_H
#include "BaseException.h"

class InvalidSkillException :  BaseException {

// -----------------------------------------------
//                    PUBLIC                     |
// -----------------------------------------------
public:

    explicit InvalidSkillException(std::string &tag);

// -----------------------------------------------
//                   PRIVATE                     |
// -----------------------------------------------
private:

    static std::string ERROR_MESSAGE;

};


#endif //INVALIDSKILLEXCEPTION_H
```

5.25 InvalidUserException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidUserException

    This class lets the program throw an exception when the selected user is not valid.

5.25.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidUserException.h.

5.26 InvalidUserException.h

Go to the documentation of this file.

```
// ------------------- InvalidUserException - JVH Systems -------------------------------
#ifndef INVALIDUSEREXCEPTION_H
#define INVALIDUSEREXCEPTION_H
#include "BaseException.h"

class InvalidUserException :  BaseException{

// -----------------------------------------------
//                   PUBLIC                      |
// -----------------------------------------------
public:

    explicit InvalidUserException(std::string &tag);

// -----------------------------------------------
//                  PRIVATE                      |
// -----------------------------------------------
private:

    static std::string ERROR_MESSAGE;

};


#endif //INVALIDUSEREXCEPTION_H
```

5.27 UserNotFoundException.h File Reference

```
#include "BaseException.h"
```

Classes

- class UserNotFoundException

  This class lets the program throw an exception when the selected user has not been found.

5.27.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file UserNotFoundException.h.

5.28 UserNotFoundException.h

Go to the documentation of this file.

```
// ------------------- UserNotFoundException - JVH Systems ------------------------------
#ifndef USERNOTFOUND_H
#define USERNOTFOUND_H
#include "BaseException.h"
class UserNotFoundException :  public BaseException {
// ------------------------------------------------
//                   PUBLIC                       |
// ------------------------------------------------
public:
    explicit UserNotFoundException(std::string &tag);
// ------------------------------------------------
//                  PRIVATE                       |
// ------------------------------------------------
private:
    static std::string ERROR_MESSAGE;
};
#endif //USERNOTFOUND_H
```

5.29 exceptionsLib.h File Reference

```
#include "except/BaseException.h"
#include "except/UserNotFoundException.h"
#include "except/InvalidUserException.h"
#include "except/InvalidFileException.h"
#include "except/InvalidDeviceException.h"
#include "except/InvalidSkillException.h"
#include "except/InsufficientPermissionsException.h"
```

5.29.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

21/11/2022

Definition in file exceptionsLib.h.

5.30 exceptionsLib.h

Go to the documentation of this file.

```
#ifndef EXCEPTIONSLIB_H
#define EXCEPTIONSLIB_H
#include "except/BaseException.h"
#include "except/UserNotFoundException.h"
#include "except/InvalidUserException.h"
#include "except/InvalidFileException.h"
#include "except/InvalidDeviceException.h"
#include "except/InvalidSkillException.h"
#include "except/InsufficientPermissionsException.h"
#endif //EXCEPTIONSLIB_H
```

5.31 IODi.h File Reference

```
#include "DataDevice.h"
```

Classes

- class IODi

  This class will be the responsible for interpreting the status of the devices and sending proper information, and executing the requested skills for the main class when it needs to interact with any device. Thus, it will be implemented by the main class and can't exist without Device class.

5.31.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file IODi.h.

5.32 IODi.h

Go to the documentation of this file.

```
// ------------------- Input/Output Device Interface - JVH Systems -------------------------------
#ifndef IODI_H
#define IODI_H
#include "DataDevice.h"
class IODi {
// -----------------------------------------------
//                   PUBLIC                      |
// -----------------------------------------------
public:
    enum{
        DEFAULT_ID = 33
    };
    // -----------------  DYNAMIC  ---------------------
    IODi();
    std::string* requestDeviceNames();
    std::string* requestDeviceSkills();
    std::string requestCurrentDeviceName();
    int requestNumberOfDataDevices();
    int requestNumberOfSkills();
    bool* requestAllStatus();
    void requestSkill(std::string skill);
    void requestSkill(int skill);
```

```
    void setCurrentDevice(std::string name);

    void setCurrentDevice(int pos);

// -----------------------------------------------
//                  PRIVATE                      |
// -----------------------------------------------
private:

    // -----------------  DYNAMIC  ----------------------

    // [Variables]

    int id;
    std::string currentDeviceName;
    int currentDevicePos;

    // -----------------  STATIC  ----------------------

    // [Functions]

    static std::string TAG;
    static int maxID;

};


#endif// IODI_H
```

5.33 KernelDevice.h File Reference

```
#include "Device.h"
```

Classes

- class KernelDevice

  This class will contain specific properties of only kernel interacting devices and a set of functionalities for managing and checking their data. It also inherits properties from Device class.

5.33.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file KernelDevice.h.

5.34 KernelDevice.h

Go to the documentation of this file.

```
// ------------------- KernelDevice - JVH Systems -------------------------------
#ifndef KERNELDEVICE_H
#define KERNELDEVICE_H
#include "Device.h"

class KernelDevice :  public Device {

    // This class will build all DataDevices
    friend class DeviceBuilder;
    friend class AI;

// -----------------------------------------------
//                    PUBLIC                     |
// -----------------------------------------------
public:

    // -----------------  DYNAMIC  ----------------------

    KernelDevice();

    KernelDevice(std::string name, int id);

// -----------------------------------------------
//                   PRIVATE                     |
// -----------------------------------------------
private:

    // -----------------  DYNAMIC  ----------------------

    // [Variables]

    bool(*securityGenerator)();
    bool securityHasBeenBroken;

    // -----------------  STATIC  ----------------------

    // [Variables]

    static KernelDevice* allKernelDevices;
    static int numKerns;

    // [Methods]

    static void setAllKernelDevices(KernelDevice* allDevices);

};


#endif //KERNELDEVICE_H
```

5.35 lib.h File Reference

```
#include "SimpleDisplay.h"
#include "exceptionsLib.h"
#include <vector>
#include <cstdio>
#include <ctime>
#include <algorithm>
#include <fstream>
#include <limits>
#include <chrono>
#include <set>
#include <csignal>
#include "../ext/kb.h"
```

5.35.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

21/11/2022

Definition in file lib.h.

5.36 lib.h

Go to the documentation of this file.

```

#ifndef LIB_H
#define LIB_H

#include "SimpleDisplay.h"
#include "exceptionsLib.h"
#include <vector>
#include <cstdio>
#include <ctime>
#include <algorithm>
#include <fstream>
#include <limits>
#include <chrono>
#include <set>
#include <csignal>

#endif //LIB_H

#ifdef _WIN32
#include <windows.h>
#include <conio.h>
#else
#include "../ext/kb.h"
#endif

```

5.37 Login.h File Reference

```
#include "Admin.h"
```

Classes

- class Login

  This class is the responsible for superimposing a login interface to make the user log in as user. This will be a graphical interface belonging to the User class.

5.37.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

04/11/2022

Definition in file Login.h.

5.38 Login.h

Go to the documentation of this file.

```
// ------------------- Login - JVH Systems -------------------------------
#ifndef LOGIN_H
#define LOGIN_H
#include "Admin.h"

class Login {

// ------------------------------------------------
//                   PUBLIC                      |
// ------------------------------------------------
public:

    // -----------------  DYNAMIC  ---------------------

    Login();

    User startLogin();

// ------------------------------------------------
//                  PRIVATE                      |
// ------------------------------------------------
private:

    // -----------------  STATIC  ---------------------

    static int SLEEP_TIME;
    static int EXIT_REQUEST;

    // -----------------  DYNAMIC  ---------------------

    int user;
    int pass;

};


#endif //LOGIN_H
```

5.39 SimpleDisplay.h File Reference

```
#include <string>
#include <iostream>
#include <vector>
#include <ctime>
#include <unistd.h>
```

Classes

- class SimpleDisplay

  This library will contain a set of methods implemented by the main for graphicating in screen the main interface. This class can be changed by any other library whose functionalities are named in the same way.

5.39.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

04/11/2022

Definition in file SimpleDisplay.h.

5.40 SimpleDisplay.h

Go to the documentation of this file.

```
// ------------------- SimpleDisplay - JVH Systems ------------------------------
#ifndef SIMPLE_DISPLAY_H
#define SIMPLE_DISPLAY_H
#include <string>
#include <iostream>
#include <vector>
#include <ctime>
#include <unistd.h>

class SimpleDisplay {

// ----------------------------------------------
//                    PUBLIC                    |
// ----------------------------------------------
public:

    // ----------------  DYNAMIC  ---------------------

    // [Methods]

    SimpleDisplay();

    // [Setters]

    void setSecurityConnected(bool securityConnected);

    void setSecurityStatus(bool securityStatus);

    void setKernelDevices(int kernelDevices);

    void setDataDevices(int dataDevices);

    void setUser(int user);

    void setGroup(const std::string &group);

    void operator«(SimpleDisplay display);

    static void printLoginInterface(int user, int pass, int INVALID_USER=-1, int INVALID_PASS=-1);

    static void printChecking();

    static void printPresentation();

    // [MAIN]

    void printMainInterface(std::string* deviceNames, bool* deviceStatus);

    // ----------------  STATIC  ---------------------

    // [DEVi++]

    static void deviCout(std::string text);

    static void printDeviHeader(std::string deviceName);

    static void cleanDevi(std::string deviceName);

    static void printDeviHelp();

    static void deviPrintData(std::vector<int> data ,std::vector<std::string> time);

    static void printSkillManual(std::string* skillnames, int count);

    // [STD]

    static void jump();

    static void cout(std::string text);

    // [AI]

    static void printAIInterface();

    static void printRequestPassword();

    static void execBuildName();

    static void execBuildType();

    static void execBuildGenerator(bool isDataDevice);

```

```
    static void execBuildFinish();

    // [Security Rescue]

    static void checkAbortStatus();

    // [Variables]

    static std::string COLOR;

    static std::string GREY_COLOR;

    static std::string RESET_COLOR;

    static std::string WARNING_COLOR;

    static SimpleDisplay* currentDisplay;

    static bool security_abort;

// ----------------------------------------------
//                  PRIVATE                     |
// ----------------------------------------------
private:

    // -----------------  STATIC  ----------------------

    static int MAXIMUM_OPTIONS;

    // -----------------  DYNAMIC  ----------------------

    // [Variables]

    bool securityConnected;

    bool securityStatus;

    int kernelDevices;

    int dataDevices;

    int user;

    std::string group;

};


#endif //SIMPLE_DISPLAY_H
```

5.41 User.h File Reference

`#include "../headers/lib.h"`

Classes

- class User

  This class will be the responsible for storing and managing the user's data. Also is able to create an abstract user object to ease data access.

5.41.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

04/11/2022

Definition in file User.h.

5.42 User.h

Go to the documentation of this file.

```
// ------------------- User - JVH Systems ------------------------------
#ifndef USER_H
#define USER_H
#include "../headers/lib.h"

class User {

// -----------------------------------------------
//                   PUBLIC                      |
// -----------------------------------------------
public:

    // [Enum]
    enum {
        INVALID_USER = -2,
        INVALID_PASS = -2,
        USER_COLUMN = 0,
        PASS_COLUMN = 1,
        ADMIN_COLUMN = 2,
        ADMIN_IDENTIFIER = 1,
        TOTAL_COLUMNS = 3,
        DEVELOPER_USER = -1
    };

    // -----------------  DYNAMIC  ---------------------

    // [Methods]

    User();

    User(int id);

    // [Variables]

    int getUser();

    bool getAdminStatus();

    // -----------------  STATIC  ---------------------

    // [Functions]

    static bool check(int id, int pass);

    static void updateUserData();

    static void saveUserData();

    static void addUser(int id, int pass, bool isAdmin);

    static void rmUser(int id);

    static void printUsers();

// -----------------------------------------------
//                  PROTECTED                    |
// -----------------------------------------------
protected:

    // -----------------  DYNAMIC  ---------------------

    // [Variables]

    int id;
    bool isAdmin;
    int pass;

    // -----------------  STATIC  ---------------------

    // [Functions]

    static int* find(int id);

    // [DEPRECATED] static std::vector<int> find(int id);

    // [Variables]

    static char SEPARATOR;
    static std::string TAG;
    static std::set<int*> allUsers;
    static std::string DATAFILE;
    static std::string USERFILE;
```

```
};


#endif //USER_H
```

5.43 Admin.cpp File Reference

```
#include "../headers/Admin.h"
```

5.44 Admin.cpp

Go to the documentation of this file.

```
#include "../headers/Admin.h"

// -----------------------------------------------------
//                    PUBLIC FUNCTIONALITIES              |
// -----------------------------------------------------

// --- [Constructor] Admin() ---
Admin::Admin(int id) :  User(id){

    this->isAdmin = true;

}

// --- [Static] adminStatus() ---
bool Admin::adminStatus(int id) {

    try{
        int* userData = find(id);
        return (userData[ADMIN_COLUMN] == ADMIN_IDENTIFIER);

    // User not found
    }catch(UserNotFoundException &e){
        return false;
    }
}
```

5.45 AI.cpp File Reference

```
#include "../headers/AI.h"
```

5.45.1 Detailed Description

Author

Sebastian Mayorquin

Date

16/11/2022 @grade Software Robotics (Software Design)

Definition in file AI.cpp.

5.46 AI.cpp

Go to the documentation of this file.

```
// ------------------- Automatic Interface - JVH Systems ------------------------------
#include "../headers/AI.h"

// ----- Declarations ------
int AI::MAX_DATA = 30;
int AI::SLEEPTIME = 1;
std::string AI::TAG = "AI";

bool AI::abort = false;

// -------------------------------------------------------
//                  PUBLIC FUNCTIONALITIES              |
// -------------------------------------------------------


// --- [Empty Constructor] AI() ---
AI::AI() {
    this->time_counter = 0;
    signal(SIGINT, signalHandler);

}

// --- [Method] startCollectingData ---
void AI::startCollectingData() {

    // ----- Intializing ------
    std::cout « "[AI] Collecting Data" « std::endl;
    int dataCounter = 0;

    // ----- Filling every device ------
    int numDevices = DataDevice::getNumSensors();
    for (int device = 0; device < numDevices; device++)
        for (int data = 0; data < MAX_DATA; data++) {
            DataDevice::allDataDevices[device].appendNewData();
            dataCounter++;
        }

    // ----- Sending Output ------
    std::cout « "[AI] Collected " « dataCounter « " measures" « std::endl;
}


// --- [Method] refreshData ---
void AI::refreshData() {

    // ----- Declarations ------
    int numDevices = DataDevice::getNumSensors();

    // ----- Refreshing every data device one time ------
    for (int i = 0; i < DataDevice::numSensors; i++) {

        DataDevice* target = &DataDevice::allDataDevices[i];

        // Check if it's active
        if (!~(*target)) continue;

        // Check if it is full
        if (target->collectedData.size() >= MAX_DATA) {

            // Remove oldest measurement
            target->collectedData.erase(target->collectedData.begin());

            // Remove oldest time
            target->collectedTimes.erase(target->collectedTimes.begin());

        }
        // Append new measurement and time
        target->appendNewData();
    }

    // ----- Refreshing every kernel device one time -----
    for (int i = 0; i < KernelDevice::numKerns; i++) {

        KernelDevice* k = &KernelDevice::allKernelDevices[i];
        // Check if it's active
        if (!~(*k)) continue;

        if (!(*k).securityGenerator() || k->securityHasBeenBroken) {
            k->securityHasBeenBroken = true;
            SimpleDisplay::currentDisplay->setSecurityStatus(false);
        }
    }
```

```
}

// --- [Method] throwDemon ---
void AI::throwDemon() {

#ifdef _WIN32

    while (true) {
        try {

            if (kbhit()) break;
            if (abort){
                SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1
instead.\n");
                exit(0);
            }
            refreshData();
            sleep(SLEEPTIME);

        } catch (std::exception &e) {
            sleep(SLEEPTIME);
            continue;
        }
    }
#else
    while (true){
    try{

        if (kb::kbhit()) break;
        if (abort){
            SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1
instead.\n");
            exit(0);
        }
        refreshData();
        sleep(SLEEPTIME);

    }catch (std::exception e){
        sleep(SLEEPTIME);
    }
}
#endif
}

void AI::requestOption(std::string opt, bool isAdmin) {
    try {
        int num = atoi(opt.c_str());

        switch (num) {

            case TURNDEVICES:
                turnDevices();
                break;

            case TURNSECURITY:
                turnSecurity();
                break;

            case CHECKSECURITY:
                checkSecurity();
                break;

            case MICROPHONE:
                microphone();
                break;

            case FORCEERROR:  {
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch (InsufficientPermissionsException &e){
                    break;
                }

                forceError();
                break;
            }

            case BUILDDEVICE:{
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch (InsufficientPermissionsException &e){
                    break;
                }

                buildDevice();
                break;
```

```
            }

            case ADDUSER:{
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch(InsufficientPermissionsException &e){
                    break;
                }
                addUser();
                break;
            }

            case REMOVEUSER:{
                try{
                    AI::ensureAdminAccess(isAdmin);
                }catch(InsufficientPermissionsException &e){
                    break;
                }
                removeUser();
                break;
            }

            case RETURNSECURITY:  {
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch (InsufficientPermissionsException &e){
                    break;
                }
                recoverSecurity();
                break;
            }

            case FORCEBADALLOC:{
                try {
                    AI::ensureAdminAccess(isAdmin);
                }catch (InsufficientPermissionsException &e){
                    break;
                }
                forceBadAlloc();
                break;
            }

            case SHOWUSERS:  {
                try {
                    AI::ensureAdminAccess(isAdmin);
                } catch (InsufficientPermissionsException &e){
                    break;
                }
                showAllUsers();
                break;
            }

            case EXIT:
                break;
        }


    }catch (std::exception &e){
        SimpleDisplay::cout("Option NOT valid");
        sleep(1);
    }

}

void AI::ensureAdminAccess(bool isAdmin) {

    // ----- Checking Admin Status -----
    if (!isAdmin) {
        SimpleDisplay::cout("THIS ACTION REQUIRES ADMINISTRATOR ACCESS");
        sleep(SLEEPTIME*2);
        throw InsufficientPermissionsException(TAG);
    }

    // ----- Entering a console and requesting password -----
    try {
        std::string message;
        SimpleDisplay::printRequestPassword();
        SimpleDisplay::cout("Keyboard:  ");
        std::getline(std::cin,message);

        // Checking input
        if (message != "0")
            throw InsufficientPermissionsException(TAG);

        // Invalid Input
        }catch(std::exception e){

```

```
            SimpleDisplay::cout("INVALID PASSWORD");
            sleep(SLEEPTIME);
            throw InsufficientPermissionsException(TAG);
        }

}

void AI::turnDevices() {

    // ----- Instantiating IODi for easy device control -----
    IODi iodi;

    // ----- Turning all Devices ON/OFF -----
    for (int i = 0; i < DataDevice::numSensors; i++) {
        iodi.setCurrentDevice(i);

        if (~DataDevice::allDataDevices[i])
            iodi.requestSkill("turnoff");
        else
            iodi.requestSkill("turnon");

    }

    // ----- Exiting...  -----
    std::string none;
    SimpleDisplay::cout("Press ENTER to continue");
    getline(std::cin,none);
}

void AI::turnSecurity() {

    for (int i = 0; i < KernelDevice::numKerns; i++) {

        // Accessing Device
        KernelDevice *k = &KernelDevice::allKernelDevices[i];

        // Checking Status
        k->isActive = !~(*k);

        // Turning ON/OFF
        if (~(*k)) {
            SimpleDisplay::cout(k->getName() + " turned ON\n");
            SimpleDisplay::currentDisplay->setSecurityConnected(true);
        }
        else {
            SimpleDisplay::cout(k->getName() + " turned OFF\n");
            SimpleDisplay::currentDisplay->setSecurityConnected(false);
        }
    }

    // ----- Exiting -----
    std::string none;
    SimpleDisplay::cout("Press ENTER to continue");
    getline(std::cin,none);
}

void AI::checkSecurity(){

    for (int i = 0; i < KernelDevice::numKerns; i++) {

        // Acessing Device
        KernelDevice* k = &KernelDevice::allKernelDevices[i];

        // Displaying chart
        SimpleDisplay::cout("[AI] [" + k->getName() + "] ");

        if (~*(k))
            SimpleDisplay::cout("[RUNNING]: ");
        else
            SimpleDisplay::cout("[DISCONNECTED]: ");

        if (!k->securityHasBeenBroken)
            SimpleDisplay::cout("Security status correct");
        else
            SimpleDisplay::cout("[WARNING] Security FAILURE! Please check the logs.");

        SimpleDisplay::cout("\n");

    }
    // ----- Exiting...  -----
    std::string none;
    SimpleDisplay::cout("Press ENTER to continue");
    getline(std::cin,none);

}

void AI::microphone(){
```

```


    try {
        std::string message;

        SimpleDisplay::cout("\nEnter a message:  ");
        getline(std::cin, message);

        SimpleDisplay::cout("[AI] SENDING MESSAGE:\n");
        SimpleDisplay::cout("|| -> " + message);

        sleep(SLEEPTIME);

    }catch (std::exception &e){
        SimpleDisplay::cout("Input not valid");
    }

}

void AI::forceError(){

    SimpleDisplay::printAIInterface();
    SimpleDisplay::cout("Forcing FAILURE in all security devices");

    for (int i = 0; i < KernelDevice::numKerns; i++) {
        KernelDevice::allKernelDevices[i].securityHasBeenBroken = true;
    }
    SimpleDisplay::currentDisplay->setSecurityStatus(false);

    sleep(SLEEPTIME*2);
}

void AI::buildDevice(){

    SimpleDisplay::execBuildName();
    std::string name;
    std::getline(std::cin,name);
    if (name == "EXIT") {

        SimpleDisplay::cout("\n[AI] Exit Request");
        sleep(SLEEPTIME);
        return;
    }

    SimpleDisplay::execBuildType();
    std::string type;
    std::getline(std::cin,type);

    if (type != "data" && type != "kernel"){
        SimpleDisplay::cout("\n[AI] Exit Request");
        sleep(SLEEPTIME);
        return;
    }

    std::string generator;
    if (type == "data"){
        SimpleDisplay::execBuildGenerator(true);
        std::getline(std::cin,generator);
    }else{
        SimpleDisplay::execBuildGenerator(false);
        std::getline(std::cin,generator);
    }

    SimpleDisplay::execBuildFinish();
    sleep(SLEEPTIME*2);
    SimpleDisplay::cout("[AI] Testing Building...\n");
    sleep(SLEEPTIME*2);
    SimpleDisplay::cout("[AI] Testing Device...\n");
    sleep(SLEEPTIME*2);

    DeviceBuilder::clearAll();
    DeviceBuilder* newDevice = new DeviceBuilder(name);
    DataDevice::numSensors = 0;
    KernelDevice::numKerns = 0;
    Device::numDevices = 0;
    if (type == "data"){
        (*newDevice).setAsDataDevice();
        (*newDevice).setDataGenerator(generateRandomData);
        DeviceBuilder::buildAllDevices();
    }else{
        (*newDevice).setAsKernelDevice();
        (*newDevice).setSecurityGenerator(exampleTest);
        DeviceBuilder::buildAllDevices();

    }
    SimpleDisplay::cout("[AI] Building SUCCESS\n");
    std::cout « "\nPress ENTER to start";
```

```
    std::cin.get();
    std::cin.clear();

}

void AI::recoverSecurity() {

    SimpleDisplay::printAIInterface();
    SimpleDisplay::cout("Recovering all Devices to Secure Status...");

    for (int i = 0; i < KernelDevice::numKerns; i++) {
        KernelDevice::allKernelDevices[i].securityHasBeenBroken = false;
    }
    SimpleDisplay::currentDisplay->setSecurityStatus(true);

    sleep(SLEEPTIME*2);
}

void AI::addUser() {

    int user, password;
    bool isAdmin;
    std::string entry;

    SimpleDisplay::cout("[AI] Insert a NIF id:  ");
    std::getline(std::cin, entry);

    try {
        user = std::stoi(entry);
    }catch (std::exception e){
        SimpleDisplay::cout("[AI] Input not valid");
        sleep(SLEEPTIME);
        return;
    }

    SimpleDisplay::cout("[AI] Insert a password:  ");

    try {
        password = std::stoi(entry);
    }catch (std::exception e) {
        SimpleDisplay::cout("[AI] Input not valid");
        sleep(SLEEPTIME);
        return;
    }

    std::getline(std::cin, entry);
    do {
        SimpleDisplay::cout("[AI] Give the user Admin Status?  [y/n]:  ");
        std::getline(std::cin, entry);
    }while(entry!= "y" && entry!= "n");

    isAdmin = (entry == "y" ?  1 :  0);
    User::addUser(user,password,isAdmin);

}

void AI::removeUser() {

    std::string id, confirmation;

    SimpleDisplay::cout("[" + TAG + "] Insert the id of the user to be removed:  ");
    std::getline(std::cin,id);
    do {
        SimpleDisplay::cout("[" + TAG + "] Are you sure to remove " + id
        + " from the User list?\nThis can't be undone [y/n]:");
        std::getline(std::cin,confirmation);
    }while(confirmation != "y" && confirmation != "n");

    if (confirmation == "y") {
        try {
            User::rmUser(std::stoi(id));
        } catch (InvalidUserException &e) {
            SimpleDisplay::cout("[" + TAG + "] User requested not found");
        }
    }
    sleep(SLEEPTIME);

}

void AI::forceBadAlloc() {

    SimpleDisplay::cout("\nBuilding Secure Environment...");
    sleep(SLEEPTIME);
    SimpleDisplay::cout("\nGenerating Error...");
    try {
        std::vector<int> vector;
        for (int i = 0; i > -1; i++)
```

```
            vector.push_back(i);
    }catch(std::exception &e) {
        SimpleDisplay::cout("\n[WARNING] Environment Failure:  \n-> what():  ");
        SimpleDisplay::cout(e.what());
        SimpleDisplay::cout("\nReturning...");
        sleep(SLEEPTIME*4);
    }
}

void AI::signalHandler(int signum) {

    // ----- Save all data -----
    User::saveUserData();

    // ----- Send abortation signal -----
    abort = true;
    SimpleDisplay::security_abort = true;

    // ----- Display EXIT CODE -----
    std::cout « "[" + TAG + "] Exit call detected.  Exiting JVH Systems...";
    sleep(SLEEPTIME);
    std::cout « "\n[" + TAG + "] Thank you for trusting us!\n\n";
    exit(0);

}

void AI::showAllUsers() {
    User::printUsers();

    std::cout « "\nPress ENTER to return";
    std::cin.get();

    //  Clear bad inputs
    std::cin.clear();

}

```

5.47 BaseException.cpp File Reference

```
#include "../headers/except/BaseException.h"
```

5.48 BaseException.cpp

Go to the documentation of this file.

```
#include "../headers/except/BaseException.h"
BaseException::BaseException() {}

const char* BaseException::what() {

    return ("[" + tag + "] " + exception).c_str();

}

```

5.49 DataDevice.cpp File Reference

```
#include "../headers/DataDevice.h"
```

5.49.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file DataDevice.cpp.

5.50 DataDevice.cpp

Go to the documentation of this file.

```
// ------------------- DataDevice - JVH Systems -------------------------------
#include "../headers/DataDevice.h"

// ----- Declarations ------
DataDevice* DataDevice::allDataDevices;
std::string* DataDevice::allDataDeviceNames;
int DataDevice::numSensors = 0;
int DataDevice::numSensorsActive = 0;


// -------------------------------------------------------
//                  PUBLIC FUNCTIONALITIES               |
// -------------------------------------------------------


// --- [Empty Constructor] DataDevice ---
DataDevice::DataDevice() {
    this->name = "";
    this->id = INVALID_DEVICE;
    this->isDataDevice = true;

};

// --- [Constructor] DataDevice ---
DataDevice::DataDevice(std::string name, int id,std::vector<void(*)()>* skillVector, std::string* skillNames) {

    // ----- Counting Devices ------
    std::cout « "[DataDevice] New Device Detected:  0x" « id « std::endl;
    numSensors++;

    // ----- Property assignation ------
    this->name = name;
    this->id = id;
    this->skills = skillVector;
    this->skillNames = skillNames;
    this->isDataDevice = true;
    this->numSkills = skillVector->size();
    this->isActive = true;
}

// --- [Getter] getNumSensors ---
int DataDevice::getNumSensors(){
    return numSensors;
};


// --- [Method] appendNewData ---
void DataDevice::appendNewData() {
    auto clock = std::chrono::system_clock::now();
    time_t time = std::chrono::system_clock::to_time_t(clock);
    std::string time_stamp = std::ctime(&time);
    this->collectedData.push_back(this->dataGenerator());
    this->collectedTimes.push_back(time_stamp);
}


// -------------------------------------------------------
//                  PRIVATE FUNCTIONALITIES              |
```

```
// ------------------------------------------------------

// --- [Method] setLimit ---
void DataDevice::setLimit(int* limitArray) {
    this->limit = limitArray;
    this->needsLimit = true;
}


// --- [Setter] setAllDataDevices ---
void DataDevice::setAllDataDevices(DataDevice* allDevices){
    allDataDevices = allDevices;
    allDataDeviceNames = new std::string[numSensors];

    for (int i = 0; i < numSensors; i++){
        allDataDeviceNames[i] = allDevices[i].getName();
    }
}
```

5.51 Device.cpp File Reference

```
#include "../headers/Device.h"
```

5.51.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file Device.cpp.

5.52 Device.cpp

Go to the documentation of this file.

```
// ------------------- Device - JVH Systems -------------------------------
#include "../headers/Device.h"

// ----- Declarations ------
int Device::numDevices;
Device* Device::allDevices;
std::string* Device::allDeviceNames;
std::string Device::TAG = "Device";

// ------------------------------------------------------
//                   PUBLIC FUNCTIONALITIES               |
// ------------------------------------------------------

// --- [Empty Constructor] Device ---
Device::Device() :  name(""),id(INVALID_DEVICE){};

// --- [Constructor] Device ---
Device::Device(std::string deviceName, int deviceId):  name(deviceName),id(deviceId){}


// --- [Getter] getName ---
std::string Device::getName() {
    return this->name;
}

int Device::getNumSkills(){
```

```
    return this->numSkills;
}

// --- [Method] operator[] (int) ---
void Device::operator[](int index){
    try{
        (*this->skills)[index]();

    }catch (std::exception){
        throw InvalidSkillException(TAG);
    }
}

// --- [Method] operator[] (string) ---
void Device::operator[](std::string skillName){
    // ----- Set number of skills ------
    int skillSize = this->numSkills;

    // ----- Find and use Skill ------
    for (int i = 0; i < skillSize; i++){
        if (skillName == skillNames[i])
            try{
                (*this->skills)[i]();
                break;
            }catch (std::exception){
                throw InvalidSkillException(TAG);
            }

    }
};

// --- [Operator] ~ ---
bool Device::operator~() {
    return this->isActive;
}

// ------------------------------------------------------
//                  PRIVATE FUNCTIONALITIES             |
// ------------------------------------------------------


// --- [Setter] setAllDevices ---
void Device::setAllDevices(Device* allNewDevices, std::string* allNewDeviceNames, int numDevices) {

    // ----- Setting Device pointers and names ------
    std::cout « "[DEVICES] Collecting all devices:  " « std::endl;
    Device::allDevices = allNewDevices;
    Device::allDeviceNames = allNewDeviceNames;
    Device::numDevices = numDevices;

    // ----- Log Success Device ------
    for (int i = 0; i < numDevices; i++){
        std::cout « "-> (SUCCESS)  " « allNewDeviceNames[i] « std::endl;
    }

}

// --- [Setter] setDeviceCounter ---
void Device::setDeviceCounter(int num) {
    Device::numDevices = num;
}
```

5.53 DeviceBuilder.cpp File Reference

`#include "../headers/DeviceBuilder.h"`

5.53.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file DeviceBuilder.cpp.

5.54 DeviceBuilder.cpp

Go to the documentation of this file.

```
// ------------------- DeviceBuilder - JVH Systems -------------------------------
#include "../headers/DeviceBuilder.h"


// ----- Declarations ------
std::vector<DeviceBuilder*> DeviceBuilder::allBuilders;
std::vector<Device> DeviceBuilder::allDevices;
std::vector<DataDevice> DeviceBuilder::allDataDevices;
std::vector<KernelDevice> DeviceBuilder::allKernelDevices;
int DeviceBuilder::deviceCounter;

// ------------------------------------------------------
//                  PUBLIC FUNCTIONALITIES              |
// ------------------------------------------------------

// --- [Static] buildDisplay ---
SimpleDisplay* DeviceBuilder::buildDisplay(){

    SimpleDisplay* display = new SimpleDisplay();
    display->setDataDevices(allDataDevices.size());
    display->setKernelDevices(allKernelDevices.size());
    display->setSecurityConnected(true);
    display->setSecurityStatus(true);
    return display;

}


// --- [Static] buildAllDevices ---
void DeviceBuilder::buildAllDevices() {

    // ----- Initializing ------
    std::cout « "[BUILDER] Sending " « allBuilders.size() « " Devices to Kernel" « std::endl;

    // ----- Declarations ------
    std::string *deviceNames = new std::string[allBuilders.size()]; // Array to Store Device names
    int numBuilders = allBuilders.size();

    // ----- Building All Devices Saved as Kernel or Data Device ------
    for (int i = 0; i < numBuilders; i++){

        // Saving device name in array
        deviceNames[i] = (*allBuilders[i]).name;

        // Building as Data or Kernel Device
        (*allBuilders[i]).buildDKDevice();
    }

    std::cout « "[BUILDER] Optimizing property access" « std::endl;
    // ----- All Devices Vector to Array ------
    Device *deviceArray = new Device[allDevices.size()];
    for (int i = 0; i < allDevices.size(); i++){
        deviceArray[i] = allDevices[i];
    }

    // ----- All DataDevices Vector to Array ------
    DataDevice *dataDeviceArray = new DataDevice[allDataDevices.size()];
    for (int i = 0; i < allDataDevices.size(); i++){
        dataDeviceArray[i] = allDataDevices[i];
    }


    // ----- All KernelDevices Vector to Array ------
    KernelDevice *kernelDeviceArray = new KernelDevice[allKernelDevices.size()];
    for (int i = 0; i < allKernelDevices.size(); i++){
        kernelDeviceArray[i] = allKernelDevices[i];
    }

    // ----- Storing Arrays ------
    std::cout « "[BUILDER] Storing General properties to Kernel..." « std::endl;
    Device::setAllDevices(deviceArray, deviceNames,numBuilders);
    DataDevice::setAllDataDevices(dataDeviceArray);
    KernelDevice::setAllKernelDevices(kernelDeviceArray);

}

// --- [Static] clearAll ---
void DeviceBuilder::clearAll(){

    // ----- Cleaning Vectors ------
    std::cout « "[BUILDER] Cleaning Builder..." « std::endl;

```

```
    allDataDevices.clear();
    allDevices.clear();
    allKernelDevices.clear();
}

// --- [Constructor] DeviceBuilder ---
DeviceBuilder::DeviceBuilder(std::string name) {

    // ----- Name Assign ------
    std::cout « "[BUILDER] Building " « name « std::endl;
    this->name = name;

    // ----- Storing Builder Pointer------
    DeviceBuilder* devicePointer = this;
    allBuilders.push_back(devicePointer);

    // ----- Settling Device ID ------
    enum {FANCY_ID = 55324,FANCY_SUM = 37};
    //Shhh..  It looks cooler with strange nums
    this->id = deviceCounter + FANCY_ID;
    deviceCounter += FANCY_SUM;

};

// --- [Method] setSkill ---
void DeviceBuilder::setSkill(void(*skill)(), std::string skillName) {

    // ----- Storing Device Skills ------
    this->skillVector.push_back(skill);
    this->skillNamesVector.push_back(skillName);
}

// --- [Method] setLimit ---
void DeviceBuilder::setLimit(int* newLimit) {

    // ----- Settling Device Limit ------
    this->limit = newLimit;
}

// --- [Setter] setAsDataDevice ---
void DeviceBuilder::setAsDataDevice() {
    this->isDataDevice = true;
}


// --- [Setter] setAsKernelDevice ---
void DeviceBuilder::setAsKernelDevice() {
    this->isDataDevice = false;
}
// --- [Method] setDataGenerator ---
void DeviceBuilder::setDataGenerator(int (*dataGenerator)()) {
    this->dataGenerator = dataGenerator;
}

void DeviceBuilder::setSecurityGenerator(bool (*securityGenerator)()) {
    this->securityGenerator = securityGenerator;
}

// ------------------------------------------------------
//                  PRIVATE FUNCTIONALITIES             |
// ------------------------------------------------------

// --- [Method] buildDKDevice---
void DeviceBuilder::buildDKDevice() {

    // ----- Skills to Dynamic Vector (so it doesn't get deleted when builder dies) ------
    std::vector<void(*)()>* dynamicSkillVector = new std::vector<void(*)()>;
    dynamicSkillVector->assign(skillVector.begin(), skillVector.end());

    // ----- SkillNames to Array (Optimizing data) ------
    std::string *skillNamesArray = new std::string[skillNamesVector.size()];
    for (int i = 0; i < this->skillNamesVector.size(); i++){
        skillNamesArray[i] = skillNamesVector[i];
    }

    // ----- Building DataDevice ------
    if (this->isDataDevice) {
        DataDevice *dataDev = new DataDevice(this->name, this->id, dynamicSkillVector, skillNamesArray);

        // Setting DataDevice limit (optional)
        if (this->limit != nullptr)
            (*dataDev).setLimit(this->limit);

        (*dataDev).dataGenerator = this->dataGenerator;

        // Saving Device Pointers
```

```
        allDevices.push_back(*dataDev);
        allDataDevices.push_back(*dataDev);

    // ----- Building KernelDevice ------
    }else {

        KernelDevice* kernDev = new KernelDevice(this->name, this->id);

        (*kernDev).securityGenerator = this->securityGenerator;

        // Saving Device Pointers
        allDevices.push_back(*kernDev);
        allKernelDevices.push_back(*kernDev);
    }

}


```

5.55 deviceDataset.cpp File Reference

```
#include "../headers/DeviceBuilder.h"
#include "exampleSkillsDataset.cpp"
#include "exampleSecurityDataset.cpp"
```

Classes

- class deviceDataset

Macros

- #define DEVICE_DATASET

5.55.1 Macro Definition Documentation

5.55.1.1 DEVICE_DATASET

```
#define DEVICE_DATASET
```

Definition at line 2 of file deviceDataset.cpp.

5.56 deviceDataset.cpp

Go to the documentation of this file.

```
#ifndef DEVICE_DATASET
#define DEVICE_DATASET
#include "../headers/DeviceBuilder.h"
#include "exampleSkillsDataset.cpp"
#include "exampleSecurityDataset.cpp"

class deviceDataset {
public:

    static void initDataset(){

        // ----- Building All Devices ------
        std::cout « "[BUILDER] Calling Default builders..." « std::endl;
        DeviceBuilder* thermometer = new DeviceBuilder("Thermometer");
        DeviceBuilder* humiditySensor = new DeviceBuilder("Humidity Sensor");
        DeviceBuilder* airSensor = new DeviceBuilder("Air Sensor");
        DeviceBuilder* lightSensor = new DeviceBuilder("Light Sensor");
        DeviceBuilder* rgbCamera = new DeviceBuilder("RGB Camera");
        DeviceBuilder* thermalCamera = new DeviceBuilder("Thermal Camera");

        DeviceBuilder* securityCamera1 = new DeviceBuilder("Security Camera 1");
        DeviceBuilder* securityCamera2 = new DeviceBuilder("Security Camera 2");
        DeviceBuilder* laserDetector = new DeviceBuilder("Laser Detector");

        // ----- Building Device Skills ------
        std::cout « "[BUILDER] Building skills" « std::endl;

        // ----- Building Main Data Generator ------
        (*thermometer).setDataGenerator(generateRandomData);
        (*humiditySensor).setDataGenerator(generateRandomData);
        (*airSensor).setDataGenerator(generateRandomData);
        (*lightSensor).setDataGenerator(generateRandomData);
        (*rgbCamera).setDataGenerator(generateRandomData);
        (*thermalCamera).setDataGenerator(generateRandomData);

        (*securityCamera1).setSecurityGenerator(exampleTest);
        (*securityCamera2).setSecurityGenerator(exampleTest);
        (*laserDetector).setSecurityGenerator(exampleTest);

        // ----- Building Skills (Only for DataDevices) -----
        (*thermometer).setSkill(exampleSkill1, "calculate");
        (*thermometer).setSkill(exampleSkill2, "set celsius");

        (*humiditySensor).setSkill(exampleSkill3,"hum decrease");
        (*humiditySensor).setSkill(exampleSkill4,"hum increase");

        (*airSensor).setSkill(exampleSkill3,"air type --british");

        (*lightSensor).setSkill(exampleSkill4,"setmode blind");
        (*lightSensor).setSkill(exampleSkill4,"setmode colorful");
        (*lightSensor).setSkill(exampleSkill4,"setmode hacker");

        (*rgbCamera).setSkill(exampleSkill3,"print camera");
        (*rgbCamera).setSkill(exampleSkill4,"lookfor --myself");

        (*thermalCamera).setSkill(exampleSkill3,"tcconfig up");
        (*thermalCamera).setSkill(exampleSkill4,"tcconfig down");

        // ----- Building Device Limits (optional) ------
        std::cout « "[BUILDER] Building limits" « std::endl;

    //    int lim[3] = {1, 2, 3};
    //    (*test).setLimit(lim);

        // ----- Setting Device Types [Data/Kernel]
        std::cout « "[BUILDER] Settling Types" « std::endl;
        (*thermometer).setAsDataDevice();
        (*humiditySensor).setAsDataDevice();
        (*airSensor).setAsDataDevice();
        (*lightSensor).setAsDataDevice();
        (*rgbCamera).setAsDataDevice();
        (*thermalCamera).setAsDataDevice();

        (*securityCamera1).setAsKernelDevice();
        (*securityCamera2).setAsKernelDevice();
        (*laserDetector).setAsKernelDevice();
    }
};

#endif
```

5.57 exampleSecurityDataset.cpp File Reference

```
#include "../headers/lib.h"
```

Macros

- #define EXAMPLE_SECURITY

Functions

- static bool exampleTest ()

5.57.1 Macro Definition Documentation

5.57.1.1 EXAMPLE_SECURITY

```
#define EXAMPLE_SECURITY
```

Definition at line 2 of file exampleSecurityDataset.cpp.

5.57.2 Function Documentation

5.57.2.1 exampleTest()

```
static bool exampleTest ( ) [static]
```

Definition at line 7 of file exampleSecurityDataset.cpp.

```
00007                          {
00008     return true;
00009
00010 };
```

Referenced by AI::buildDevice(), and deviceDataset::initDataset().

5.58 exampleSecurityDataset.cpp

Go to the documentation of this file.

```
00001 #ifndef EXAMPLE_SECURITY
00002 #define EXAMPLE_SECURITY
00003 #include "../headers/lib.h"
00004
00005 // --- [Function] exampleTest ---
00006 // Usage:  Example Security Skill for developer testing
00007 static bool exampleTest() {
00008     return true;
00009
00010 };
00011
00012 #endif
```

5.59 exampleSkillsDataset.cpp File Reference

```
#include "../headers/lib.h"
```

Macros

- #define EXAMPLE_SKILLS

Functions

- static void exampleSkill1 ()
- static void exampleSkill2 ()
- static void exampleSkill3 ()
- static void exampleSkill4 ()
- static int generateRandomData ()

5.59.1 Macro Definition Documentation

5.59.1.1 EXAMPLE_SKILLS

```
#define EXAMPLE_SKILLS
```

Definition at line 2 of file exampleSkillsDataset.cpp.

5.59.2 Function Documentation

5.59.2.1 exampleSkill1()

```
static void exampleSkill1 ( )  [static]
```

Definition at line 7 of file exampleSkillsDataset.cpp.

```
00007                                {
00008     SimpleDisplay::deviCout("This is the example Skill 1");
00009
00010 };
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.59.2.2 exampleSkill2()

```
static void exampleSkill2 ( ) [static]
```

Definition at line 14 of file exampleSkillsDataset.cpp.

```
00014                                  {
00015     SimpleDisplay::deviCout("This is the example Skill 2" );
00016 };
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.59.2.3 exampleSkill3()

```
static void exampleSkill3 ( ) [static]
```

Definition at line 20 of file exampleSkillsDataset.cpp.

```
00020                                  {
00021     SimpleDisplay::deviCout("This is the example Skill 3");
00022 };
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.59.2.4 exampleSkill4()

```
static void exampleSkill4 ( ) [static]
```

Definition at line 26 of file exampleSkillsDataset.cpp.

```
00026                                  {
00027     SimpleDisplay::deviCout("This is the example Skill 4");
00028 }
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.59.2.5 generateRandomData()

```
static int generateRandomData ( ) [static]
```

Definition at line 30 of file exampleSkillsDataset.cpp.

```
00030                                      {
00031     return rand();
00032 }
```

Referenced by AI::buildDevice(), and deviceDataset::initDataset().

5.60 exampleSkillsDataset.cpp

Go to the documentation of this file.

```
#ifndef EXAMPLE_SKILLS
#define EXAMPLE_SKILLS
#include "../headers/lib.h"

// --- [Function] exampleSkill ---
// Usage:  Example Skill for developer testing
static void exampleSkill1() {
    SimpleDisplay::deviCout("This is the example Skill 1");

};

// --- [Function] exampleSkill ---
// Usage:  Example Skill for developer testing
static void exampleSkill2() {
    SimpleDisplay::deviCout("This is the example Skill 2" );
};

// --- [Function] exampleSkill ---
// Usage:  Example Skill for developer testing
static void exampleSkill3() {
    SimpleDisplay::deviCout("This is the example Skill 3");
};

// --- [Function] exampleSkill ---
// Usage:  Example Skill for developer testing
static void exampleSkill4() {
    SimpleDisplay::deviCout("This is the example Skill 4");
}

static int generateRandomData() {
    return rand();
}


#endif
```

5.61 InsufficientPermissionsException.cpp File Reference

`#include "../headers/except/InsufficientPermissionsException.h"`

5.62 InsufficientPermissionsException.cpp

Go to the documentation of this file.

```
#include "../headers/except/InsufficientPermissionsException.h"
std::string InsufficientPermissionsException::ERROR_MESSAGE = "Invalid User";

InsufficientPermissionsException::InsufficientPermissionsException(std::string &tag) {

    this->exception = ERROR_MESSAGE;
    this->tag = tag;


}
```

5.63 InvalidDeviceException.cpp File Reference

`#include "../headers/except/InvalidDeviceException.h"`

5.64 InvalidDeviceException.cpp

Go to the documentation of this file.

```
#include "../headers/except/InvalidDeviceException.h"
std::string InvalidDeviceException::ERROR_MESSAGE = "Invalid User";

InvalidDeviceException::InvalidDeviceException(std::string &tag) {

    this->exception = ERROR_MESSAGE;
    this->tag = tag;

}
```

5.65 InvalidFileException.cpp File Reference

```
#include "../headers/except/InvalidFileException.h"
```

5.66 InvalidFileException.cpp

Go to the documentation of this file.

```
#include "../headers/except/InvalidFileException.h"
std::string InvalidFileException::ERROR_MESSAGE = "Invalid File";

InvalidFileException::InvalidFileException(std::string &tag) {

    this->exception = ERROR_MESSAGE;
    this->tag = tag;
}
```

5.67 InvalidSkillException.cpp File Reference

```
#include "../headers/except/InvalidSkillException.h"
```

5.68 InvalidSkillException.cpp

Go to the documentation of this file.

```
#include "../headers/except/InvalidSkillException.h"
std::string InvalidSkillException::ERROR_MESSAGE = "Invalid File";

InvalidSkillException::InvalidSkillException(std::string &tag) {

    this->exception = ERROR_MESSAGE;
    this->tag = tag;

}
```

5.69 InvalidUserException.cpp File Reference

```
#include "../headers/except/InvalidUserException.h"
```

5.70 InvalidUserException.cpp

Go to the documentation of this file.

```
#include "../headers/except/InvalidUserException.h"

std::string InvalidUserException::ERROR_MESSAGE = "Invalid User";

InvalidUserException::InvalidUserException(std::string &tag) {

    this->exception = ERROR_MESSAGE;
    this->tag = tag;

}
```

5.71 IODi.cpp File Reference

```
#include "../headers/IODi.h"
#include "../headers/AI.h"
```

5.71.1 Detailed Description

Author

Sebastian Mayorquin

Date

16/11/2022 @grade Software Robotics (Software Design)

Definition in file IODi.cpp.

5.72 IODi.cpp

Go to the documentation of this file.

```
// ------------------- Input/Output Device Interface - JVH Systems -------------------------------
#include "../headers/IODi.h"
#include "../headers/AI.h"

// ----- Declarations ------
int IODi::maxID = 0;
std::string IODi::TAG = "IODi";

// -------------------------------------------------------
//                  PUBLIC FUNCTIONALITIES               |
// -------------------------------------------------------

// --- [Empty Constructor] IODi() ---
IODi::IODi(){
    this->id = DEFAULT_ID;
    maxID++;
};

// --- [Setter] setCurrentDevice ---
void IODi::setCurrentDevice(int pos) {

    // ----- If position is valid ------
    if (pos < IODi::requestNumberOfDataDevices()) {

        // Set position in current device
        this->currentDevicePos = pos;

```

```
        // Search and set name in current device
        this->currentDeviceName = DataDevice::allDataDeviceNames[pos];

    }
    else
        // ----- Argument not valid ------
        throw InvalidDeviceException(TAG);

}

// --- [Setter] setCurrentDevice ---
void IODi::setCurrentDevice(std::string name) {

    // ----- Check type argument is not a number ------
    if (std::isdigit(name[0])){

        this->setCurrentDevice(std::stoi(name));
        return;
    }

    // ----- Declarations ------
    int maxDevices = DataDevice::getNumSensors();

    // ----- Search Device ------
    for (int i = 0; i < maxDevices; i++){

        if (name == DataDevice::allDataDeviceNames[i]){

            // Save position and name in current device
            this->currentDevicePos = i;
            this->currentDeviceName = name;
            return;

        }

    }
    // ----- Device not valid ------
    throw InvalidDeviceException(TAG);

}


// --- [REQUEST] -> Name of all Devices
std::string* IODi::requestDeviceNames(){
    return DataDevice::allDataDeviceNames;
};

// --- [REQUEST] -> Number of all DataDevices
int IODi::requestNumberOfDataDevices(){
    return DataDevice::getNumSensors();
};

// --- [REQUEST] -> Number of Skills of current device
int IODi::requestNumberOfSkills(){
    return DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();
}

// --- [REQUEST] -> Name of Skills of current device
std::string* IODi::requestDeviceSkills(){
    int numSkills = DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();

    std::string* deviceSkills = new std::string[numSkills];

    for (int i = 0; i < numSkills; i++){
        deviceSkills[i] = DataDevice::allDataDevices[this->currentDevicePos].skillNames[i];
    }

    return deviceSkills;
};
// --- [Function] requestAllStatus()
bool* IODi::requestAllStatus() {

    bool* status = new bool[DataDevice::numSensors];

    for (int i = 0; i < DataDevice::numSensors; i++)
        status[i] = ~DataDevice::allDataDevices[i];

    return status;
};

// --- [REQUEST] -> Uses a skill given the position in device array
void IODi::requestSkill(int skill){
    try {
        DataDevice::allDataDevices[this->currentDevicePos][skill];
    }catch(InvalidSkillException &e){};
};

```

```
// --- [REQUEST] -> Name of current device
std::string IODi::requestCurrentDeviceName() {
    return this->currentDeviceName;
}

// --- [REQUEST] -> Uses a skill given the keyword
void IODi::requestSkill(std::string skill){

    // ----- Obtaining current device -----
    DataDevice* target = &DataDevice::allDataDevices[this->currentDevicePos];

    // ----- Switch between possible arguments ------
    if (skill == ":help")
        SimpleDisplay::printDeviHelp();

    else if (skill == ":dev")
        SimpleDisplay::printSkillManual(target->skillNames,\
                                        this->requestNumberOfSkills());
                                        // -> Vector with the name of skills
                                        // -> Int with the number of skills

    else if (skill == ":clean")
        SimpleDisplay::cleanDevi(this->currentDeviceName);

    else if (skill == "data")
        SimpleDisplay::deviPrintData(target->collectedData,\
                                     target->collectedTimes);
                                     // -> Vector with measurements
                                     // -> Vector with times of measurements

    else if(skill == "turnoff"){
        target->isActive = false;
        target->collectedData.resize(1);
        target->collectedData[0] = 0;
        target->collectedTimes.resize(1);
        target->collectedTimes[0] = "REBOOT";
        SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned OFF");
    }

    else if(skill == "turnon"){
        target->isActive = true;
        SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned ON");
    }

    else if (skill == "reboot"){
        this->requestSkill("turnoff");
        this->requestSkill("turnon");
    }

    else if (skill == "pwd")
        SimpleDisplay::deviCout("/home/devices/" + std::to_string(this->currentDevicePos));

    // ----- Frequent Invalid Commands ------

    else if (skill == "help")
        SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :help");

    else if (skill == "exit")
        SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :wq");

    else if (skill == "clear")
        SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :clean");

    else if (skill[0] == '\"' || skill[0] == '\'') {
        SimpleDisplay::deviCout("Are you trying to Inject SQL? Lol ok");
        SimpleDisplay::deviCout("[YOU HAVE BEEN BANNED]");
        exit(1);
    }

    // ----- Not Global command, request to current device ------
    else {
        try{
            (*target)[skill];
        }catch(InvalidSkillException &e){};
    }
}
```

5.73 KernelDevice.cpp File Reference

`#include "../headers/KernelDevice.h"`

5.73.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file KernelDevice.cpp.

5.74 KernelDevice.cpp

Go to the documentation of this file.

```
// ------------------- KernelDevice - JVH Systems -------------------------------
#include "../headers/KernelDevice.h"


// ----- Declarations ------
KernelDevice* KernelDevice::allKernelDevices;
int KernelDevice::numKerns = 0;

// ------------------------------------------------------
//                  PUBLIC FUNCTIONALITIES              |
// ------------------------------------------------------

// --- [Empty Constructor] KernelDevice ---
KernelDevice::KernelDevice() {
    this->name = "";
    this->isActive = false;
    this->id = INVALID_DEVICE;
    this->isDataDevice = false;
    this->securityHasBeenBroken = false;
};

// --- [Constructor] KernelDevice ---
KernelDevice::KernelDevice(std::string name, int id) {

    std::cout « "[KernelDevice] New Device Detected:  0x" « id « std::endl;

    numKerns++;

    this->name = name;
    this->id = id;
    this->isDataDevice = false;
    this->isActive = true;
    this->securityHasBeenBroken = false;
}

// ------------------------------------------------------
//                  PRIVATE FUNCTIONALITIES             |
// ------------------------------------------------------

// --- [Setter] setAllKernelDevices ---
void KernelDevice::setAllKernelDevices(KernelDevice* allDevices){
    allKernelDevices = allDevices;
}

```

5.75 Login.cpp File Reference

```
#include "../headers/Login.h"
```

5.75.1 Detailed Description

Author

Sebastian Mayorquin

Date

03/11/2022 @grade Software Robotics (Software Design)

Definition in file Login.cpp.

5.76 Login.cpp

Go to the documentation of this file.

```
// ------------------- Login - JVH Systems -------------------------------
#include "../headers/Login.h"
// ----- Declarations ------
int Login::SLEEP_TIME = 1;
int Login::EXIT_REQUEST = -1;

// -------------------------------------------------------
//                  PUBLIC FUNCTIONALITIES              |
// -------------------------------------------------------

// --- [Constructor] Login() ---
Login::Login() {};



// --- [Method] startLogin() ---
User Login::startLogin() {


    // ----- Declarations ------
    bool badAccess = false;          // Bool to detect if input is not correct.

    // ----- Logger Bucle ------
    do {

        //  Checking Bad Inputs
        if (badAccess) {
            SimpleDisplay::cout("Bad Access");
            sleep(SLEEP_TIME);
        }

        //  Reset Variables
        user = User::INVALID_USER;
        pass = User::INVALID_PASS;

        //  Prompt user
        SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
        SimpleDisplay::cout("\nKeyboard:  ");
        std::cin » user;

        if (user == EXIT_REQUEST) raise(SIGINT);

        //  Prompt password
        SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
        SimpleDisplay::cout("\nKeyboard:  ");
        std::cin » pass;

        if (pass == EXIT_REQUEST) raise(SIGINT);

        //  Print checking interface
        SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
        SimpleDisplay::printChecking();

        //  Prepare next logger (if needed)
        badAccess = true;
        sleep(SLEEP_TIME);

        //  Clear bad inputs
        std::cin.clear();
```

```
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        //  Loop until the user and password are valid
    } while (!User::check(user, pass));

    // Return Admin (If needed)
    if (Admin::adminStatus(user))
        return Admin(user);

    // ----- Returning User ------
    return User(user);
};


```

5.77 main.cpp File Reference

This is the main Test Program, Running for testing snapshot.

`#include "../headers/AI.h"`

main

Author

Sebastian Mayorquin

- enum { MAIN_SCREEN = 0 , DEVICE_SCREEN = 1 , SETTINGS_SCREEN = 2 }
- int main (int argc, char **argv)

  Main function to start and keep running JVH systems.

5.77.1 Detailed Description

This is the main Test Program, Running for testing snapshot.

Date

04/11/2022

Version

v.0.2.1

Definition in file main.cpp.

5.77.2 Enumeration Type Documentation

5.77.2.1 anonymous enum

`anonymous enum`

Defining the different screens that the program can have.

Enumerator

MAIN_SCREEN	
DEVICE_SCREEN	
SETTINGS_SCREEN	

Definition at line 14 of file main.cpp.

```
00014     {
00015     MAIN_SCREEN = 0,
00016     DEVICE_SCREEN = 1,
00017     SETTINGS_SCREEN = 2
00018 };
```

5.77.3 Function Documentation

5.77.3.1 main()

```
int main (
            int argc,
            char ** argv )
```

Main function to start and keep running JVH systems.

Parameters

argc	Number of arguments passed to the program.
argv	Argument vector.

Definition at line 27 of file main.cpp.

```
00027                                  {
00028 
00029     // ----- Checking Arguments ------
00030     bool IS_DEVELOPER = false;
00031 
00032 #ifdef _WIN32
00033     for (int i = 0; i < argc; i++){
00034 
00035         if (strcmp(argv[i], "--no-colors") == 0){
00036             SimpleDisplay::COLOR = "";
00037             SimpleDisplay::GREY_COLOR = "";
00038             SimpleDisplay::WARNING_COLOR = "";
00039             SimpleDisplay::RESET_COLOR = "";
00040         }
00041         if (strcmp(argv[i], "--developer") == 0)
00042             IS_DEVELOPER = true;
00043     }
00044 #else
00045         // Code for Linux
00046         for (int i = 0; i < argc; i++){
00047             if (strncmp(argv[i], "--no-colors", 11) == 0){
00048                 SimpleDisplay::COLOR = "";
00049                 SimpleDisplay::GREY_COLOR = "";
00050                 SimpleDisplay::WARNING_COLOR = "";
00051                 SimpleDisplay::RESET_COLOR = "";
00052             }
00053             if (strncmp(argv[i], "--developer", 11) == 0)
00054                 IS_DEVELOPER = true;
00055         }
00056 #endif
00057 
00058     // ----- Initializing Builder + Display ------
00059     std::cout « "-------- [STARTING BUILDING] --------" « std::endl;
```

```
    deviceDataset::initDataset();
    DeviceBuilder::buildAllDevices();
    SimpleDisplay* display = DeviceBuilder::buildDisplay();

    // ----- Initializing AI------
    AI* ai = new AI();
    ai->startCollectingData();


    std::cout « "-------- [BUILDING SUCCESS] --------" « std::endl;

    // ----- Initializing IODi ------
    int screen_status = MAIN_SCREEN;
    std::string answer = "";
    IODi* iodi = new IODi();

    // ----- Initializing User Data------
    try {
        User::updateUserData();
    }catch (InvalidFileException &e){
        std::cout « "\n[Login] There has been an error updating user file, exiting...";
        exit(1);
    }catch (std::exception &e){
        std::cout « "\n[Login] There has been an error accessing the file, exiting...";
        exit(1);
    }
    // ----- Initializing Login ------
    Login* l = new Login();

    std::cout « "\nPress ENTER to start";
    std::cin.get();

    //  Clear bad inputs
    std::cin.clear();

    // ----- Checking Developer Status ------
    if (!IS_DEVELOPER) {
        SimpleDisplay::printPresentation();
        std::cout « "\nPress ENTER to start";
        std::cin.get();
    }

    //  Clear bad inputs
    std::cin.clear();

    // ----- Sending to Logger ------
    User user;
    if (!IS_DEVELOPER)
        user = l->startLogin();
    else {
        user = User(User::DEVELOPER_USER);
    }
    // ----- Setting User ------
    display->setUser(IS_DEVELOPER ? User::DEVELOPER_USER :  user.getUser());
    display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));

    // ----- [MAIN BUCLE] ------
    while(true){

        // ----- MAIN SCREEN ------
        if (screen_status == MAIN_SCREEN) {

            // Print Display
            std::string *deviceNames = iodi->requestDeviceNames();
            display->printMainInterface(deviceNames,iodi->requestAllStatus());

            // Request an entry
            SimpleDisplay::cout("Keyboard:  ");
            ai->throwDemon(); // Update while not entry
            std::cin » answer;

            // Exit if requested
            if (answer == "exit" || answer == "-1") {

                // Generating Logger
                user = l->startLogin();
                display->setUser(user.getUser());
                display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));

            }

            // Go to Settings if requested
            else if (answer == "settings" || answer == "-2") {
                screen_status = SETTINGS_SCREEN;
            }

            // Entry not recognized, sending to IODi
```

```
            else
                try {

                    // Send to IODi and switch screen
                    iodi->setCurrentDevice(answer);
                    screen_status = DEVICE_SCREEN;

                }catch (InvalidDeviceException &e){

                    //IODi hadn't found valid option, reset
                    SimpleDisplay::cout("INVALID OPTION");
                    sleep(1);
                }

        // ----- DEVi++ Terminal ------
        }else if (screen_status == DEVICE_SCREEN){

            // Print heeader and request entry
            SimpleDisplay::printDeviHeader(iodi->requestCurrentDeviceName());
            std::getline(std::cin,answer);

            // Don't exit the terminal until exit
            while (answer != ":wq") {

                // Request an entry
                SimpleDisplay::cout("~ ");
                std::getline(std::cin, answer);

                // Force refresh the devices if requested
                if (answer == "forceref") {
                    ai->refreshData();
                    SimpleDisplay::deviCout("[AI] Forced current Device to Refresh");
                }

                // Show the user if requested
                else if (answer == "who"){
                    SimpleDisplay::deviCout(std::to_string(user.getUser()));
                }

                // Command not recognized, sending to IODi
                else
                    iodi->requestSkill(answer);
            }

            // When DEVi++ finish, switch to main screen
            screen_status = MAIN_SCREEN;

        }else if (screen_status == SETTINGS_SCREEN){

            while (answer != "-1") {

                SimpleDisplay::printAIInterface();
                SimpleDisplay::cout("Keyboard:  ");
                std::getline(std::cin, answer);
                AI::requestOption(answer,user.getAdminStatus());
            }
            SimpleDisplay copy = *display;
            display = DeviceBuilder::buildDisplay();
            *display « copy;

            SimpleDisplay::currentDisplay = display;
            screen_status = MAIN_SCREEN;

        }
    }
}
```

References DeviceBuilder::buildAllDevices(), DeviceBuilder::buildDisplay(), SimpleDisplay::COLOR, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, User::DEVELOPER_USER, DEVICE_SCREEN, SimpleDisplay::deviCout(), User::getAdminStatus(), User::getUser(), SimpleDisplay::GREY_COLOR, deviceDataset::initDataset(), MAIN_SCREEN, SimpleDisplay::printAIInterface(), SimpleDisplay::printDeviHeader(), SimpleDisplay::printMainInterface(), SimpleDisplay::printPresenta AI::refreshData(), IODi::requestAllStatus(), IODi::requestCurrentDeviceName(), IODi::requestDeviceNames(), AI::requestOption(), IODi::requestSkill(), SimpleDisplay::RESET_COLOR, IODi::setCurrentDevice(), SimpleDisplay::setGroup(), SETTINGS_SCREEN, SimpleDisplay::setUser(), AI::startCollectingData(), Login::startLogin(), AI::throwDemon(), User::updateUserData(), and SimpleDisplay::WARNING_COLOR.

5.78 main.cpp

Go to the documentation of this file.

```
// ---------------------------- main - JVH Systems -------------------------------
#include "../headers/AI.h"
enum{
    MAIN_SCREEN = 0,
    DEVICE_SCREEN = 1,
    SETTINGS_SCREEN = 2
};


int main(int argc, char** argv)  {

    // ----- Checking Arguments ------
    bool IS_DEVELOPER = false;

#ifdef _WIN32
    for (int i = 0; i < argc; i++){

        if (strcmp(argv[i], "--no-colors") == 0){
            SimpleDisplay::COLOR = "";
            SimpleDisplay::GREY_COLOR = "";
            SimpleDisplay::WARNING_COLOR = "";
            SimpleDisplay::RESET_COLOR = "";
        }
        if (strcmp(argv[i], "--developer") == 0)
            IS_DEVELOPER = true;
    }
#else
        // Code for Linux
        for (int i = 0; i < argc; i++){
            if (strncmp(argv[i], "--no-colors", 11) == 0){
                SimpleDisplay::COLOR = "";
                SimpleDisplay::GREY_COLOR = "";
                SimpleDisplay::WARNING_COLOR = "";
                SimpleDisplay::RESET_COLOR = "";
            }
            if (strncmp(argv[i], "--developer", 11) == 0)
                IS_DEVELOPER = true;
        }
#endif

    // ----- Initializing Builder + Display ------
    std::cout « "-------- [STARTING BUILDING] --------" « std::endl;
    deviceDataset::initDataset();
    DeviceBuilder::buildAllDevices();
    SimpleDisplay* display = DeviceBuilder::buildDisplay();

    // ----- Initializing AI------
    AI* ai = new AI();
    ai->startCollectingData();


    std::cout « "-------- [BUILDING SUCCESS] --------" « std::endl;

    // ----- Initializing IODi ------
    int screen_status = MAIN_SCREEN;
    std::string answer = "";
    IODi* iodi = new IODi();

    // ----- Initializing User Data------
    try {
        User::updateUserData();
    }catch (InvalidFileException &e){
        std::cout « "\n[Login] There has been an error updating user file, exiting...";
        exit(1);
    }catch (std::exception &e){
        std::cout « "\n[Login] There has been an error accessing the file, exiting...";
        exit(1);
    }
    // ----- Initializing Login ------
    Login* l = new Login();

    std::cout « "\nPress ENTER to start";
    std::cin.get();

    //  Clear bad inputs
    std::cin.clear();

    // ----- Checking Developer Status ------
    if (!IS_DEVELOPER) {
        SimpleDisplay::printPresentation();
        std::cout « "\nPress ENTER to start";
        std::cin.get();
    }

    //  Clear bad inputs
    std::cin.clear();

```

```
    // ----- Sending to Logger ------
    User user;
    if (!IS_DEVELOPER)
        user = l->startLogin();
    else {
        user = User(User::DEVELOPER_USER);
    }
    // ----- Setting User ------
    display->setUser(IS_DEVELOPER ? User::DEVELOPER_USER :  user.getUser());
    display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));

    // ----- [MAIN BUCLE] ------
    while(true){

        // ----- MAIN SCREEN ------
        if (screen_status == MAIN_SCREEN) {

            // Print Display
            std::string *deviceNames = iodi->requestDeviceNames();
            display->printMainInterface(deviceNames,iodi->requestAllStatus());

            // Request an entry
            SimpleDisplay::cout("Keyboard:  ");
            ai->throwDemon(); // Update while not entry
            std::cin » answer;

            // Exit if requested
            if (answer == "exit" || answer == "-1") {

                // Generating Logger
                user = l->startLogin();
                display->setUser(user.getUser());
                display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));

            }

            // Go to Settings if requested
            else if (answer == "settings" || answer == "-2") {
                screen_status = SETTINGS_SCREEN;
            }

            // Entry not recognized, sending to IODi
            else
                try {

                    // Send to IODi and switch screen
                    iodi->setCurrentDevice(answer);
                    screen_status = DEVICE_SCREEN;

                }catch (InvalidDeviceException &e){

                    //IODi hadn't found valid option, reset
                    SimpleDisplay::cout("INVALID OPTION");
                    sleep(1);
                }

        // ----- DEVi++ Terminal ------
        }else if (screen_status == DEVICE_SCREEN){

            // Print heeader and request entry
            SimpleDisplay::printDeviHeader(iodi->requestCurrentDeviceName());
            std::getline(std::cin,answer);

            // Don't exit the terminal until exit
            while (answer != ":wq") {

                // Request an entry
                SimpleDisplay::cout("~ ");
                std::getline(std::cin, answer);

                // Force refresh the devices if requested
                if (answer == "forceref") {
                    ai->refreshData();
                    SimpleDisplay::deviCout("[AI] Forced current Device to Refresh");
                }

                // Show the user if requested
                else if (answer == "who"){
                    SimpleDisplay::deviCout(std::to_string(user.getUser()));
                }

                // Command not recognized, sending to IODi
                else
                    iodi->requestSkill(answer);
            }

            // When DEVi++ finish, switch to main screen
```

```
                screen_status = MAIN_SCREEN;

            }else if (screen_status == SETTINGS_SCREEN){

                while (answer != "-1") {

                    SimpleDisplay::printAIInterface();
                    SimpleDisplay::cout("Keyboard:  ");
                    std::getline(std::cin, answer);
                    AI::requestOption(answer,user.getAdminStatus());
                }
                SimpleDisplay copy = *display;
                display = DeviceBuilder::buildDisplay();
                *display « copy;

                SimpleDisplay::currentDisplay = display;
                screen_status = MAIN_SCREEN;

            }
        }
}
```

5.79 SimpleDisplay.cpp File Reference

`#include "../headers/SimpleDisplay.h"`

5.79.1 Detailed Description

Author

Sebastian Mayorquin

Date

03/11/2022 @grade Software Robotics (Software Design)

Definition in file SimpleDisplay.cpp.

5.80 SimpleDisplay.cpp

Go to the documentation of this file.

```
// ------------------- SimpleDisplay - JVH Systems -------------------------------
#include "../headers/SimpleDisplay.h"

int SimpleDisplay::MAXIMUM_OPTIONS = 12;
std::string SimpleDisplay::COLOR = "\033[36m";
std::string SimpleDisplay::WARNING_COLOR = "\033[31m";
std::string SimpleDisplay::RESET_COLOR = "\033[0m";
std::string SimpleDisplay::GREY_COLOR = "\033[32m";
SimpleDisplay* SimpleDisplay::currentDisplay;
bool SimpleDisplay::security_abort = false;

// --- [Function] SimpleDisplay()
SimpleDisplay::SimpleDisplay() {

    SimpleDisplay::currentDisplay = this;

};

// ------------------------------------------------------
//                        LOGIN                        |
// ------------------------------------------------------

// --- [Static] printLoginInterface ---
```

```
void SimpleDisplay::printLoginInterface(int user, int pass, int INVALID_USER, int INVALID_PASS){

    // ----- Declarations ------

    std::string current_user;
    std::string current_pass;


    // ----- Checking if variables are printable ------

    if (user == INVALID_USER) current_user = "";
    else current_user = std::to_string(user);

    if (pass == INVALID_PASS) current_pass = "";
    else current_pass= std::to_string(pass);


    // ----- Printing Interface ------
    jump();
    std::cout « "    ______  __     __  __    __          _______                        __
\n";
    std::cout « "   |      \\|  \\    |  \\|  \\  |  \\        /      \\                      |  \\
\n";
    std::cout « "    \\$$$$$| $$    | $$| $$  | $$       |  $$$$$$\\ __    __   ________  _| $$_
_______   _______ _____    ________ \n";
    std::cout «"      | $$| $$    | $$| $$__| $$       | $$___\\$$|  \\  |  \\ /       \\|   $$ \\   /
\\ |      \\    \\  /        \\\n";
    std::cout « " __   | $$ \\$$\\ /  $$| $$    $$        \\$$    \\ | $$  | $$|  $$$$$$$ \\$$$$$$  |
$$$$$$\\| $$$$$$\\$$$$\\|  $$$$$$$\n";
    std::cout « "|  \\  | $$  \\$$\\  $$ | $$$$$$$$        _\\$$$$$$\\| $$  | $$ \\$$    \\   | $$ __ |
$$    $$| $$ | $$ | $$ \\$$    \\ \n";
    std::cout « "| $$__| $$   \\$$ $$  | $$  | $$       |  \\__| $$| $$__/ $$ _\\$$$$$$\\  | $$|  \\|
$$$$$$$$| $$ | $$ | $$ _\\$$$$$$\\\n";
    std::cout « " \\$$    $$    \\$$$   | $$  | $$        \\$$    $$ \\$$    $$|        $$   \\$$  $$
\\$$     \\| $$ | $$ | $$|        $$\n";
    std::cout « "  \\$$$$$$      \\$      \\$$   \\$$         \\$$$$$$  _\\$$$$$$$ \\$$$$$$$     \\$$$$
\\$$$$$$$ \\$$  \\$$  \\$$ \\$$$$$$$ \n";
    std::cout « "                                                     |  \\__| $$
\n";
    std::cout « "                                                      \\$$    $$
\n";
    std::cout « "                                                       \\$$$$$$
\n\n\n";
    std::cout « "\t\t\t#---------------------------------------------------#\n";
    std::cout « "\n\t\t\t\t\t-> User:" « current_user « std::endl;
    std::cout « "\t\t\t\t\t-> Password:";


    // ----- Encrypting Password ------

    for (int i = 0; i < current_pass.size(); i++) std::cout « '*';


    // ----- Printing Interface ------

    std::cout « "\n\n\t\t\t#---------------------------------------------------#\n";
    std::cout « "\n\t\t\t\t       Type -1 to exit\n";
}
// --- [Function] printChecking()
void SimpleDisplay::printChecking(){
    std::cout « "\n\t\t\t\t\t    Checking...\n\n";
}

// -------------------------------------------------------
//                          STD COUT                    |
// -------------------------------------------------------


// --- [Static] cout ---
void SimpleDisplay::cout(std::string text){
    checkAbortStatus();
    std::cout « text;
}

// --- [Static] jump ---
void SimpleDisplay::jump(){
    for (int i = 0; i < 80; i++){
        std::cout « std::endl;
    };
}

// -------------------------------------------------------
//                            MAIN                      |
// -------------------------------------------------------


// --- [Static] printMainInterface ---
```

```
void SimpleDisplay::printMainInterface(std::string* deviceNames, bool* deviceStatus) {
    std::string directory[SimpleDisplay::MAXIMUM_OPTIONS];
    std::string option[SimpleDisplay::MAXIMUM_OPTIONS];
    std::string status[SimpleDisplay::MAXIMUM_OPTIONS];


    for (int i = 0; i < this->dataDevices; i++) {
        directory[i] = deviceNames[i];
        option[i] = std::to_string(i) + ")";
        if (deviceStatus[i])
            status[i] = "Running";
        else
            status[i] = "Inactive";
    }


    for (int i = this->dataDevices; i < SimpleDisplay::MAXIMUM_OPTIONS; i++) {
        directory[i] = "";
        option[i] = "";
        status[i] = "-";
    }

    std::string conected;
    if (this->securityConnected)
        conected = "Connected";
    else
        conected = "Disconnected";

    std::string secStatus;
    if (this->securityStatus)
        secStatus = "Secure";
    else
        secStatus = "WARNING";

    std::string username = (this->user == 0) ?  "root" :  std::to_string(this->user);
    std::string group = (this->user == -1) ?  "DEVMODE" :  this->group;

    jump();

printf("#==========================##=================================================================================#\n");
    printf("|  %-23s ||                                                                                   |\n","---- [DIRECTORY] ----");
    printf("| %-3s %-20s ||       #------------------     JVH - Systems     --------------------#         |\n",option[0].c_str(),directory[0].c_str());
    printf("| %-3s %-20s ||                                                                                   |\n",option[1].c_str(),directory[1].c_str());
    printf("| %-3s %-20s ||       Welcome to the Main Interface of JVH! Select an option from the             |\n",option[2].c_str(),directory[2].c_str());
    printf("| %-3s %-20s ||       directory by writting its name or position and pressing ENTER.              |\n",option[3].c_str(),directory[3].c_str());
    printf("| %-3s %-20s ||                                                                                   |\n",option[4].c_str(),directory[4].c_str());
    printf("| %-3s %-20s ||                              [Current Status]                                     |\n",option[5].c_str(),directory[5].c_str());
    printf("| %-3s %-20s ||                                                                                   |\n",option[6].c_str(),directory[6].c_str());
    printf("| %-3s %-20s ||       [Security Alarms]:  %-12s      [Security Status]:  %-8s             |\n",option[7].c_str(),directory[7].c_str(),conected.c_str(),secStatus.c_str());
    printf("| %-3s %-20s ||                                                                                   |\n",option[8].c_str(),directory[8].c_str());
    printf("| %-3s %-20s ||         Device 1:    %-11s                Device 7:    %-11s                |\n",option[9].c_str(),directory[9].c_str(),status[0].c_str(),status[6].c_str());
    printf("| %-3s %-20s ||         Device 2:    %-11s                Device 8:    %-11s                |\n",option[10].c_str(),directory[10].c_str(),status[1].c_str(),status[7].c_str());
    printf("| %-3s %-20s ||         Device 3:    %-11s                Device 9:    %-11s                |\n",option[11].c_str(),directory[11].c_str(),status[2].c_str(),status[8].c_str());
    printf("|  %-23s ||         Device 4:    %-11s                Device 10:  %-11s                |\n","",status[3].c_str(),status[9].c_str());
    printf("|  %-23s ||         Device 5:    %-11s                Device 11:  %-11s                |\n","",status[4].c_str(),status[10].c_str());
    printf("|  %-23s ||         Device 6:    %-11s                Device 12:  %-11s                |\n","",status[5].c_str(),status[11].c_str());
    printf("|  %-23s ||                                                                                   |\n","");
    printf("| %-2d) %-20s ||       [User]:  %-5s                       [Kernel Devices]:  %-2d            |\n",-1,"EXIT",username.c_str(),this->kernelDevices);
    printf("| %-2d) %-20s ||       [Group]:  %-7s                    [Data Devices]:  %-2d              |\n",-2,"SETTINGS",group.c_str(),this->dataDevices);
    printf("|  %-23s ||                                                                                   |\n","");

printf("#==========================##=================================================================================#\n");

}

// --- [Function] printPresentation()
void SimpleDisplay::printPresentation() {
```

```
    jump();

    printf("#==================================       JVH-Systems  V.0.3.1
========================================#\n\n");
    printf("       Bienvenido Software Developer
\n");
    printf("\n");
    printf("       Esta pantalla es puramente informativa y NO aparecera en la version FINAL del
programa.\n");
    printf("       A continuacion vas a acceder al programa JVH-Systems version V.0.3.1 Antes de
lanzarte\n");
    printf("       al programa te recomendamos que compruebes los siguientes parametros:\n");
    printf("\n");
    printf("    1.  Comprueba que tu terminal es capaz de ver el recuadro en el que se encuentra
incluido\n");
    printf("       este texto EN SU TOTALIDAD.\n");
    printf("\n");
    printf("%s", ("    2.  Comprueba que tu terminal colorea " + COLOR + "estas palabras" +
RESET_COLOR + " de color azul.  Si tu terminal no acepta\n").c_str());
    printf("       colores ANSI cierra este programa con Ctrl+C y ejecuta ./main --no-colors\n");
    printf("\n");
    printf("    3.  Una vez hayas comprobado todo estaras listo para testear este programa
adecuadamente.\n");
    printf("\n");
    printf("       Hemos dejado a tu disposicion dos cuentas.  Puedes agregar tras iniciar el
programa:\n");
    printf("
\n");
    printf("                           root:  0                 GUEST: 99999\n");
    printf("                         Password:  0               Password:  99999999\n");
    printf("\n");
    printf("       Tambien puedes utilizar ./main --developer para solucionar problemas con la base de
datos\n\n");
#ifndef _WIN32
    printf("%s", ("       " + WARNING_COLOR + "[WARNING]" + RESET_COLOR + " El sistema ha detectado
que no estas utilizando Windowsx86 o que\n").c_str());
    printf("       estas utilizando Linux.  Se han desativado las opciones --developer y --no-colors
\n");
    printf("       en versiones anteriores a V.0.3.0 por razones de seguridad.  PD: Versión en el
titulo.\n\n");
#endif

printf("#==============================================================================================================#\n");

}

// -----------------------------------------------------
//                        AI                          |
// -----------------------------------------------------
void SimpleDisplay::printAIInterface() {

    checkAbortStatus();
    jump();

printf("#==========================================================================================================#\n");
    printf("|                                              SETTINGS
|\n");
    printf("|
|\n");
    printf("|    Type a number for using one of the following Options
|\n");
    printf("|
|\n");
    printf("|    1)  Turn off/on all devices
|\n");
    printf("|    2)  Turn off/on all security
|\n");
    printf("|    3)  Check Security Status
|\n");
    printf("|    4)  Use Microphone
|\n");
    printf("|    5)  [EXP] Force a Security Failure
|\n");
    printf("|    6)  [EXP] Build a New Device
|\n");
    printf("|    7)  Add new User to System
|\n");
    printf("|    8)  Remove User from System
|\n");
    printf("|    9)  Return Security to safe status
|\n");
    printf("|    10) [EXP] Force Bad_Alloc Error
|\n");
    printf("|    11) Show all users
|\n");
    printf("|    12) About Us
```

```
|\n");
    printf("|                                                                      |\n");
    printf("|    -1) Exit                                                          |\n");
    printf("|                                                                      |\n");
printf("#==================================================================================================#\n");

}

// --- [Function] printRequestPassword()
void SimpleDisplay::printRequestPassword() {

    jump();
    std::cout « "#-------------------------------------------------------------------#\n";
    std::cout « "|                                                                   |\n";
    std::cout « "|      The command requested needs administrator/developer          |\n";
    std::cout « "|      access.  Please verify your account typing root password       |\n";
    std::cout « "|                                                                   |\n";
    std::cout « "#-------------------------------------------------------------------#\n";
}

// --- [Function] execBuildName()
void SimpleDisplay::execBuildName(){
    checkAbortStatus();
    jump();
    printf("%s",(COLOR + "#!/bin/bash" + RESET_COLOR + "\n\n").c_str());
    printf("%s",(GREY_COLOR + "# ------------------------------ DEVICE BUILDER -------------------------------------\n").c_str());
    printf("%s",(GREY_COLOR + "# Welcome to the Device Builder in Execution Time.  We will guide you for easily adding\n").c_str());
    printf("%s",(GREY_COLOR + "# more devices to JVH_Systems without modifying the program.  This tool is experimental\n").c_str());
    printf("%s",(GREY_COLOR + "# can cause SEVERAL DAMAGE to your program.  Please, if you have any doubt about the \n").c_str());
    printf("%s",(GREY_COLOR + "# building process, type EXIT (anytime) and read more in our github:\n").c_str());
    printf("%s",(GREY_COLOR + "# https://github.com/clases-julio/p6-setstl-usuarios-SamthinkGit\n\n" + RESET_COLOR).c_str());
    printf("%s",("cd " + COLOR + "$HOME" + RESET_COLOR + "/src/deviceDataset.cpp\n").c_str());
    printf("%s",("slot=find("+COLOR+"\'DeviceNameSlot\'"+RESET_COLOR+")\n\n").c_str());
    printf("if [ slot == \"VALID_SLOT\" ]; then\n\n");
    printf("%s",(GREY_COLOR + " # Set the name of your device after 'name=', you can use both simbols a-zA-Z0-9 and spaces.\n").c_str());
    printf("%s",(GREY_COLOR + " # When you have written the name press ENTER. PD: You don't need to use Quoation marks\n" + RESET_COLOR).c_str());
    printf("%s",(GREY_COLOR + " # Example:  «stablish name=My Device 2»\n" + RESET_COLOR).c_str());
    printf("   stablish name=");
}

// --- [Function] execBuildType()
void SimpleDisplay::execBuildType(){
    checkAbortStatus();
    printf("%s",("   export "+ COLOR + "$name" + RESET_COLOR +" > /dev/sda1 | \\\n").c_str());
    printf(R"(                 grep -E '.*[\.dev,\.data]' | \)");
printf("%s",("\n                   xarg -I touch "+ COLOR + "$slot" + RESET_COLOR + "." + COLOR + "$name"+ RESET_COLOR + "\n").c_str());
    printf("fi\n\n");
    printf("%s",("cd /dev/sda1/" + COLOR + "$slot\n" + RESET_COLOR).c_str());
    printf("%s",(GREY_COLOR + "\n# Select the type of device.  Write \'data\' to build a DataDevice or \'kernel\'").c_str());
    printf("%s",(GREY_COLOR + "\n# to build a KernelDevice.  Remember:  Don't use quotation marks." + RESET_COLOR).c_str());
    printf("%s",("\nwrite -t $(hex " + COLOR +"/usr/bin/gparted/sda1" + RESET_COLOR +" | grep -F 'dev_type') < type.setDevicetype=").c_str());
}

// --- [Function] execBuildGenerator()
void SimpleDisplay::execBuildGenerator(bool isDataDevice){

    checkAbortStatus();
    printf("%s",("\nif [ \"" + COLOR + "$EUID" + RESET_COLOR + "\" -ne 0 ]; then\n").c_str());
    printf("   nc -e /bin/bash $(ifconfig | head -1 6)\n");
    printf("   nxforce sudo su dev\n");
    printf("\nmkdir /tmp/new_generator\n");
    printf("touch /tmp/new_generator/config.txt\n");
    printf("echo Device Generator \\n:  $(hex /config/dev/sda1) > config.txt\n");
    printf("chmod +x /tmp/new_generator/config_dev.sh\n");
    printf("(sh /tmp/new_generator/config_dev.sh)\n");
    printf("%s",("\nlog "+ COLOR + "$EXECUTION" + RESET_COLOR + " complete\n\n").c_str());

    if (isDataDevice){
        printf("%s",(GREY_COLOR + "# Select one main generator for the data device.  This will be the function \n").c_str());
        printf("# responsible for generating all measurements.  We provided one example of data
```

```
generators.  \n");
        printf("# Feel free for adding more by yourself in /src/exampleSkillsDataset.cpp\n");
        printf("# -> exampleGenerator1\n");
        printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
").c_str());
    }else{
        printf("%s",(GREY_COLOR + "# Select one main generator for the kernel device.  This will be
the function \n").c_str());
        printf("# responsible for checking the security status.  We provided 1 example of main
generator.  \n");
        printf("# Feel free for adding more by yourself in /src/exampleSecurityDataset.cpp\n");
        printf("# -> exampleSecurity1\n");
        printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
").c_str());
    }

}
// --- [Function] execBuildFinish()
void SimpleDisplay::execBuildFinish(){
    checkAbortStatus();
    printf("\nsystemctl grub turnoff\n");
    printf("\nfinal_device=$(sh /tmp/*.dev.sh)");
    printf("%s",("\necho " + COLOR + "$BUILD_LOG" + RESET_COLOR + " >
/var/JVH/building.log").c_str());
    printf("\nsystemctl grub turnon");
    printf("\n\nexec building\n\n");
    printf("------ Output ------\n");
}

// ------------------------------------------------------
//                         DEVi++                       |
// ------------------------------------------------------


// --- [Static] printDeviHeader ---
void SimpleDisplay::printDeviHeader(std::string deviceName) {
    jump();

printf("#===========================================================================================#\n");
    printf("|                                                 %-20s
|\n",deviceName.c_str());

printf("#===========================================================================================#\n\n");
    printf("~                               DEVi++ - Device VIM Based Terminal
\n");
    printf("~
\n");
    printf("~                                         version %-9s
\n","DEV.0.2");
    printf("~                                    by Sebastian Mayorquin
\n");
    printf("~
\n");
    printf("~                                  Sponsor DEVi++ Development!
\n");
    printf("~
\n");
    printf("%s", ("~                             type       :wq" + COLOR +"<Enter>" + RESET_COLOR +"
to exit                                             \n").c_str());
    printf("%s", ("~                             type       :help" + COLOR +"<Enter>" + RESET_COLOR +"
for DEVi help                                       \n").c_str());
    printf("%s", ("~                             type       :dev" + COLOR +"<Enter>" + RESET_COLOR +"
for skills help                                     \n").c_str());
    printf("%s", ("~                             type       :clean" + COLOR +"<Enter>" + RESET_COLOR +"
for cleaning terminal                                       \n").c_str());

    printf("~
\n");
    printf("~                           Thanks for trusting in JVH Systems!
\n");
    printf("~\n");
    printf("~\n");
    printf("~\n");
    printf("~\n");
    printf("~\n");
    printf("~\n");
    printf("~\n");


}


// --- [Static] cleanDevi ---
void SimpleDisplay::printDeviHelp() {

    printf("~ -------------------- DEVi++ Help --------------------\n");
    printf("~ Devi++ is a VIM based terminal that will allow you to\n");
```

```
    printf("~ interact with your devices with commands.  There are \n");
    printf("%s", ("~ two types of commands " + COLOR + "Global Commands" + RESET_COLOR + " and " +
COLOR + "Device Skills"+ RESET_COLOR + "\n").c_str());
    printf("~ \n");
    printf("~ Global commands are available in every device and let\n");
    printf("~ you interact with DEVi or with your Device status.\n");
    printf("~ For using them you only have to type any of the following\n");
    printf("~ orders:\n");
    printf("~ \n");
    printf("%s", ("~ -> " + COLOR + ":wq"+ RESET_COLOR + "       Exit the terminal (Do not use Ctrl +
C)\n").c_str());
    printf("%s", ("~ -> " + COLOR + ":help"+ RESET_COLOR + "     Displays DEVi manual \n").c_str());
    printf("%s", ("~ -> " + COLOR + ":clean"+ RESET_COLOR + "    Clean DEVi display \n").c_str());
    printf("~ \n");
    printf("%s", ("~ -> " + COLOR + "data"+ RESET_COLOR + "      Displays collected data since
starting/1h\n").c_str());
    printf("%s", ("~ -> " + COLOR + "forceref"+ RESET_COLOR + "  Forces current device to refresh
data\n").c_str());
    printf("%s", ("~ -> " + COLOR + "turnon"+ RESET_COLOR + "    Turns the device ON\n").c_str());
    printf("%s", ("~ -> " + COLOR + "turnoff"+ RESET_COLOR + "   Turns the device OFF\n").c_str());
    printf("%s", ("~ -> " + COLOR + "reboot"+ RESET_COLOR + "    Reset the device\n").c_str());
    printf("%s", ("~ -> " + COLOR + "who"+ RESET_COLOR + "       Prints the name of the
user\n").c_str());
    printf("%s", ("~ -> " + COLOR + "pwd"+ RESET_COLOR + "       Prints current path\n").c_str());

    printf("~ \n");
    printf("~ Note:  While DEVi terminal is active, all devices will be\n");
    printf("~ stopped until applying changes\n");
    printf("~ \n");
    printf("~ Each device has its own commands.  Those commands are \n");
    printf("~ called as skills and are automatically detected when \n");
    printf("~ starting the program.  You can look at the self-generated\n");
    printf("%s", ("~ skills manual by executing "+ COLOR + ":dev"+ RESET_COLOR + "\n").c_str());
    printf("~ -----------------------------------------------------\n");
}

// --- [Static] printSkillManual ---
void SimpleDisplay::printSkillManual(std::string* skillnames, int count) {

    printf("~ -------- DEVi++ Self-Generated Skill Manual ---------\n");
    printf("~ Every device has its own commands, JVH implements a  \n");
    printf("~ builder to install high-level functions into the     \n");
    printf("~ interface.  This device functionalities are called    \n");
    printf("%s", ("~ " + COLOR + "skills" + RESET_COLOR + ".  In this version, we only allow
high-level     \n").c_str());
    printf("~ developers to integrate the name of the skill without  \n");
    printf("~ description.\n");
    printf("~ \n");
    printf("~ This manual will show all skills detected for the\n");
    printf("~ selected device, for more help, you can search in\n");
    printf("~ our wiki:  \n");
    printf("~ \n");
    printf("%s", ("~ " + COLOR +  "https://github.com/clases-julio/p4-implclases-SamthinkGit" +
RESET_COLOR + "\n").c_str());
    printf("~ \n");
    printf("~ [DEVi++] Searching Skills...\n");
    printf("%s", ("~ [DEVi++] " + std::to_string(count+7) + " skills detected.\n").c_str());
    printf("~ [DEVi++] Generating Manual:\n");
    printf("~ \n");
    printf("%s",("~ [GLOBAL] -> " + COLOR + "data" + RESET_COLOR + "\n").c_str());
    printf("%s",("~ [GLOBAL] -> " + COLOR + "forceref" + RESET_COLOR + "\n").c_str());
    printf("%s",("~ [GLOBAL] -> " + COLOR + "turnon" + RESET_COLOR + "\n").c_str());
    printf("%s",("~ [GLOBAL] -> " + COLOR + "turnoff" + RESET_COLOR + "\n").c_str());
    printf("%s",("~ [GLOBAL] -> " + COLOR + "reboot" + RESET_COLOR + "\n").c_str());


    for (int i = 0; i < count; i++){
        printf("%s",("~ [DETECTED] -> " + COLOR + skillnames[i] + RESET_COLOR + "\n").c_str());
    }
    printf("~ \n");
    printf("~ NOTE: For using skills only type the name in DEVi++ and press <ENTER>\n");
    printf("~ -----------------------------------------------------\n");
}

// --- [Static] cleanDevi ---
void SimpleDisplay::cleanDevi(std::string deviceName) {

    jump();

printf("#==========================================================================================#\n");
    printf("|                                           %-20s
|\n",deviceName.c_str());

printf("#==========================================================================================#\n\n");
    for (int i = 0; i < 20;i++){
        printf("~\n");
    }
```

```
}


// --- [Static] deviCout ---
void SimpleDisplay::deviCout(std::string text) {

    checkAbortStatus();
    printf("%s", ("~ " + text + "\n").c_str());
}

// --- [Static] printDeviData ---
void SimpleDisplay::deviPrintData(std::vector<int> data, std::vector<std::string> time) {

    int date_length = 19;
    int size = data.size();
    deviCout("-----------------  Collected Data  ------------------");
    for (int i = 0; i < size; i++){
        printf("%s",("~ " + COLOR + "[").c_str());
        printf("%.*s", date_length, time[i].c_str());
        printf("%s",("]" + RESET_COLOR + ":  " + std::to_string(data[i]) + "\n").c_str());
    }
    deviCout("-----------------------------------------------------");

}

// ------------------------------------------------------
//                          SETTERS                     |
// ------------------------------------------------------
// --- [Function] setSecurityConnected()
void SimpleDisplay::setSecurityConnected(bool securityConnected) {
    SimpleDisplay::securityConnected = securityConnected;
}
// --- [Function] setSecurityStatus()
void SimpleDisplay::setSecurityStatus(bool securityStatus) {
    SimpleDisplay::securityStatus = securityStatus;
}
// --- [Function] setKernelDevices()
void SimpleDisplay::setKernelDevices(int kernelDevices) {
    SimpleDisplay::kernelDevices = kernelDevices;
}
// --- [Function] setDataDevices()
void SimpleDisplay::setDataDevices(int dataDevices) {
    SimpleDisplay::dataDevices = dataDevices;
}
// --- [Function] setUser()
void SimpleDisplay::setUser(int user) {
    SimpleDisplay::user = user;
}
// --- [Function] setGroup()
void SimpleDisplay::setGroup(const std::string &group) {
    SimpleDisplay::group = group;
}
// --- [Function] operator()
void SimpleDisplay::operator«(SimpleDisplay display) {

    this->securityStatus = display.securityStatus;
    this->securityConnected = display.securityConnected;
    this->user = display.user;
    this->group = display.group;

}

// ------------------------------------------------------
//                          SECURITY                    |
// ------------------------------------------------------

// --- [Function] checkAbortStatus()
void SimpleDisplay::checkAbortStatus() {

    if (security_abort) {
        std::cout « "\n----------------- [RESCUE PROTOCOL THREAD] --------------------------";
        std::cout « "\n[WARNING] Abort signal called from insecure process, if you are trying to\n";
        std::cout « "exit the program, please use -1 to return to login interface.\n";
        std::cout « "[AI] Executing Secure Exit...";
        exit(1);
    }
}
```

5.81 User.cpp File Reference

```
#include "../headers/User.h"
```

5.81.1 Detailed Description

Author

Sebastian Mayorquin

Date

03/11/2022 @grade Software Robotics (Software Design)

Definition in file User.cpp.

5.82 User.cpp

Go to the documentation of this file.

```
// ------------------- User - JVH Systems ------------------------------
#include "../headers/User.h"

// ----- Declarations ------
std::set<int*> User::allUsers;
char User::SEPARATOR = ',';
std::string User::TAG = "User";

//File for retrieving user info.
std::string User::DATAFILE = "data/userdata.txt";
std::string User::USERFILE = "data/userdata.dat";

// ------------------------------------------------------
//                  PUBLIC FUNCTIONALITIES              |
// ------------------------------------------------------

// --- [Empty Constructor] User() ---
User::User() :  id(INVALID_USER), isAdmin(false) {};


// --- [Constructor] User() ---
User::User(int id) :  id(id){

    if (id == DEVELOPER_USER ){
        this->pass = 0;
        this->isAdmin = true;
        return;
    }

    // ----- Initializing User info ------
    try {

        //  Find and Set admin status from data
        int* userData = find(id);
        this->pass = userData[PASS_COLUMN];
        this->isAdmin = userData[ADMIN_COLUMN];


    // ----- Catching invalid users ------
    }catch(InvalidUserException &e){

        this->id = INVALID_USER;
        this->isAdmin = 0;

    }
};


```

```

// --- [Getter] getUser() ---
int User::getUser(){
    return this->id;
}

// --- [Getter] getAdminStatus() ---
bool User::getAdminStatus(){
    return this->isAdmin;
}


// --- [Static] check() ---
bool User::check(int id, int pass){
    try{
        int* userData = find(id);
        return (userData[PASS_COLUMN] == pass);

    // User not found
    }catch(UserNotFoundException &e){
        return false;
    }
}
// --- [Static] updateUserData() ---
void User::updateUserData() {

    // ----- Declarations -----
    User user;

    // ---- Opening file ----
    std::ifstream file (USERFILE);

    // ---- Testing if file is correct ----
    if (!file)
        throw InvalidFileException(TAG);

    // ----- Reading every user in file -----
    while (file.read(reinterpret_cast <char *>(&user),sizeof (User))){

        int* newEntry = new int[TOTAL_COLUMNS];
        newEntry[USER_COLUMN] = user.id;
        newEntry[PASS_COLUMN] = user.pass;
        newEntry[ADMIN_COLUMN] = user.isAdmin;

        allUsers.insert(newEntry);
    }
    // ----- Closing file -----
    file.close();
}
/*
// --- [Static] updateUserData() ---
void User::updateUserData() {

// ----- Declarations ------
std::ifstream file(User::DATAFILE);
std::string word;

int line = 0; // Info for exception

// ----- Reading File ------
try {
while (file.good()) {

try {
int *userdata = new int(TOTAL_COLUMNS);

// Get user data delimited by commas
for (int i = 0; i < TOTAL_COLUMNS - 1; i++) {

getline(file, word, SEPARATOR);
userdata[i] = std::stoi(word);
}

// Get last user element
getline(file, word);
userdata[TOTAL_COLUMNS - 1] = std::stoi(word);

// Store user data
allUsers.insert(userdata);
line++;
}catch (std::exception &e){
std::cout « "\n[User] User in line " + std::to_string(line) + " is NOT valid,";
std::cout « "\n[Info] Make sure there are no spaces or ENTER between or after the data.";
throw InvalidUserException(TAG);
}
}

```

```
// ----- Close file ------
file.close();

// ----- Sort Users ------
// [DEPRECATED] std::sort(data.begin(), data.end());
}catch (std::exception &e){
std::cout « "\n[User] File located in " + DATAFILE + " is not valid as an User list.";
throw InvalidFileException(TAG);
}
}
*/

// -------------------------------------------------------
//                  PRIVATE FUNCTIONALITIES              |
// -------------------------------------------------------

// --- [Static] find()
// Usage:  Find a user by giving the id as a key
// Maximum complexity of O(n)
// Note:  Complexity of O(log(n)) is possible, not
// implemented yet.
int* User::find(int id){

    // ----- Creates iterator
    std::set<int*>::iterator it;

    // ----- Travel the set until find
    for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){

        if ((*iterator)[USER_COLUMN] == id)
            return *iterator;
    }

    // ----- User not found, send error

    // [DEPRECADTED] throw std::runtime_error("User not Found");
    throw UserNotFoundException(TAG);

}
// --- [Function] addUser()
void User::addUser(int user, int pass, bool isAdmin) {

    int* newUser = new int[TOTAL_COLUMNS];
    newUser[USER_COLUMN] = user;
    newUser[PASS_COLUMN] = pass;
    newUser[ADMIN_COLUMN] = (isAdmin) ?  ADMIN_IDENTIFIER :  0;

    allUsers.insert(newUser);

}

// --- [Function] rmUser()
void User::rmUser(int id) {

    try {

        int *user = find(id);
        allUsers.erase(user);

    }catch(InvalidUserException &e){

        throw;

    };

}
// --- [Function] printUsers()
void User::printUsers() {

    for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){

            std::cout « ((*iterator)[ADMIN_COLUMN] == 0 ?  "User:  " :  "Admin:  ");
            std::cout « (*iterator)[USER_COLUMN] « std::endl;

    }

}

void User::saveUserData() {

    // ---- Opening file ----
    std::ofstream file (USERFILE);

    // ---- Testing if file is correct ----
    if (!file)
        throw InvalidFileException(TAG);

```

```
    // ----- Creates iterator -----
    std::set<int*>::iterator it;

    // ----- Saves all user data in allUsers to file  -----
    for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){

        User newUser = User((*iterator)[USER_COLUMN]);
        file.write(reinterpret_cast <const char *> (&newUser),sizeof (User));

    }

    // ----- Close the file -----
    file.close();

}

// --- [Static] find() [DEPRECATED] ---
// Usage:  Finds a user from the data array using
// binary search.  Throws a running_time exception
// if user doesn't exist.
// Note:  This method can be used only with data
// contained into a double vector
/*
std::vector<int> User::find(int id) {

// ----- Declarations ------
int begin = 0;
int end = data.size() - 1;
int mid;

// ----- Searching ------
while (begin < end){

//  Fix for odd positions
if (mid == (begin + end) / 2)
begin++;

//  Pointer to next analysis
mid = (begin + end) / 2;

//  Check if target is lower
if (id < data[mid][USER_COLUMN])
end = mid;

//  Check if target is higher
else if (id > data[mid][USER_COLUMN])
begin = mid;

//  Else, it is the target
else
return data[mid];
};

//  Throw exception if user is not found
throw std::runtime_error("Not Valid User");
}
*/
```

5.83 UserNotFoundException.cpp File Reference

```
#include "../headers/except/UserNotFoundException.h"
```

5.84 UserNotFoundException.cpp

Go to the documentation of this file.

```
#include "../headers/except/UserNotFoundException.h"
std::string UserNotFoundException::ERROR_MESSAGE = "User not Found";

UserNotFoundException::UserNotFoundException(std::string &tag) {

    this->tag = tag;
    this->exception = ERROR_MESSAGE;

};


```

Index